

BAN CƠ YẾU CHÍNH PHỦ
HỌC VIỆN KỸ THUẬT MẬT MÃ



BÁO CÁO CHUYÊN ĐỀ CƠ SỞ

**Xây Dựng Hệ Thống Splunk Phát Hiện Hành Vi Thông Qua
Rules Và Cảnh Báo Tự Động**

Khoa: Công nghệ thông tin

Mã nhóm: 42

Sinh viên thực hiện:

Nguyễn Trung Kiên - AT200432

Hoàng Hải Dương - AT200415

Nguyễn Văn Khánh - AT200430

Người hướng dẫn:

TS. Lê Anh Tiến

Khoa công nghệ thông tin – Học viện Kỹ thuật mật mã

Hà Nội - 2026

MỤC LỤC

MỤC LỤC	ii
DANH MỤC HÌNH ẢNH.....	v
DANH MỤC BẢNG BIỂU.....	vi
DANH MỤC TỪ VIẾT TẮT	vii
LỜI MỞ ĐẦU	1
CHƯƠNG 1. TỔNG QUAN VỀ SIEM VÀ ỨNG DỤNG AI TRONG SPLUNK	4
1.1. HỆ THỐNG SIEM	4
1.1.1. Tổng quan về SIEM.....	4
1.1.2. Cơ chế phát hiện mối đe dọa	5
1.2. Kiến trúc hệ thống Splunk.....	6
1.2.1. Tổng quan về Splunk.....	6
1.2.2. Các thành phần trong Splunk	8
1.2.3. Luồng xử lý dữ liệu trong Splunk	12
1.2.4. Xây dựng Alert và Dashboard giám sát	13
1.3. Tìm hiểu về MITRE ATT&CK và nguồn dữ liệu.....	14
1.3.1. Cấu trúc Tactic, Technique, Sub-technique	14
1.3.2. Ứng dụng MITRE ATT&CK trong phát hiện tấn công.....	15
1.3.3. Windows Event Log và Sysmon	15
1.3.4. Linux Syslog và Audit Log	18
1.3.5. Syslog từ thiết bị mạng và tường lửa	19
1.3.6. Các phương thức thu thập log	20
1.4. Tìm hiểu về ngôn ngữ truy vấn Sigma Rule.....	21
1.4.1. Giới thiệu và vai trò của Sigma	21
1.4.2. Cú pháp và cấu trúc rule Sigma.....	22
1.5. Ngôn ngữ SPL	22
1.5.1. Tổng quan về SPL	22
1.5.2. Tương quan sự kiện với SPL.....	23
1.5.3. Lookup và Enrichment	24
1.6. Chuẩn hoá dữ liệu trong Splunk.....	25
1.6.1. Tổng quan về chuẩn hoá dữ liệu logs.....	26
1.6.2. Ứng dụng Data Model trong Splunk	26
1.6.3. Chuẩn hoá dữ liệu với Common Information Model.....	27

1.6.4. Chuẩn hóa dữ liệu với TA trên forwarder	28
1.7. Ứng dụng AI trong Splunk	29
1.7.1. Tổng quan về mô hình ngôn ngữ lớn	29
1.7.2. Kỹ thuật hỗ trợ độ chính xác của LLM	33
1.7.3. Định hướng ứng dụng LLM hỗ trợ vận hành bảo mật	35
1.8. Tổng kết chương 1	38
CHƯƠNG 2. TRIỂN KHAI TÍCH HỢP AI TRONG SPLUNK	40
2.1. Mô hình triển khai	40
2.2. Thu thập và chuẩn hoá dữ liệu	40
2.2.1. Thu thập logs	40
2.2.2. Chuẩn hoá dữ liệu theo CIM	43
2.3. Thiết kế Rule bằng SPL	44
2.3.1. Kỹ thuật phân tích ngữ nghĩa dòng lệnh	44
2.3.2. Kỹ thuật tương quan cây tiến trình	45
2.3.3. Kỹ thuật phân tích ngưỡng và phát hiện dị thường	46
2.3.4. Tích hợp AI hỗ trợ truy vấn log	47
2.4. Cảnh báo tự động	51
2.4.1. Tích hợp AI hỗ trợ phân tích logs	51
2.4.2. Tích hợp AI hỗ trợ điều tra sự cố	59
2.4.3. Cơ chế Scheduled Search trong Splunk	65
2.4.4. Trigger Alert và Add-on Telegram	65
2.5. Trực quan hoá và Dashboard	66
2.5.1. Các loại biểu đồ	66
2.5.2. Thiết kế Dashboard tích hợp AI	67
2.6. Tổng kết chương 2	67
CHƯƠNG 3. THỰC NGHIỆM TÍCH HỢP AI TRONG SPLUNK	68
3.1. Mô hình thực nghiệm	68
3.2. Quy trình thực nghiệm	69
3.2.1. Kịch bản thực nghiệm	69
3.2.2. Mô phỏng tấn công	71
3.2.3. Cấu hình Rules và tạo Alert	73
3.2.4. AI hỗ trợ truy vấn logs	78
3.2.5. AI hỗ trợ phân tích logs	78

3.2.6. AI hỗ trợ điều tra sự cố.....	80
3.3. Đánh giá kết quả.....	81
KẾT LUẬN	83
TÀI LIỆU THAM KHẢO	84
PHỤ LỤC	85

DANH MỤC HÌNH ẢNH

Hình 1.1. Cách thức hoạt động của SIEM.....	5
Hình 1.2. Tổng quan về Splunk.....	7
Hình 1.3. Các thành phần chính trong Splunk	8
Hình 1.4. Cơ chế Bucket trong Indexer.....	10
Hình 1.5. Các thành phần phụ trong Splunk	11
Hình 1.6. Luồng xử lý dữ liệu trong Splunk	12
Hình 1.7. Sơ đồ cơ chế hoạt động của LLM	31
Hình 2.1. Mô hình triển khai hệ thống Splunk tích hợp AI.....	40
Hình 2.2. Mô hình thu thập logs trong Splunk.....	41
Hình 2.3. Cơ chế thu thập logs trong Splunk	42
Hình 2.4. Quy trình chuẩn hóa dữ liệu theo tiêu chuẩn CIM.....	43
Hình 2.7. Luồng hoạt động chức năng hỗ trợ truy vấn log	51
Hình 2.5. Luồng hoạt động chức năng hỗ trợ phân tích log.....	53
Hình 2.6. Luồng hoạt động chức năng hỗ trợ điều tra sự cố.....	61
Hình 3.1. Mô hình thực nghiệm hệ thống Splunk tích hợp AI.....	68

DANH MỤC BẢNG BIỂU

Bảng 1.1. So sánh Universal Forwarder và Heavy Forwarder	10
Bảng 1.2. Đặc điểm của các loại License	12
Bảng 1.3. So sánh Windows Security Event và Sysmon Event	17
Bảng 1.4. So sánh phương thức Push và Pull	21
Bảng 3.1. Yêu cầu cấu hình các thành phần hệ thống	68
Bảng 3.2. Danh sách địa chỉ IP của các thành phần hệ thống Splunk	69
Bảng 3.3. Các cổng mạng sử dụng trong mô hình thực nghiệm Splunk	69
Bảng 3.4. So sánh hiệu quả giữa thực hiện thủ công và sử dụng AI hỗ trợ	82

DANH MỤC TỪ VIẾT TẮT

STT	Từ viết tắt	Tên tiếng anh	Tên tiếng việt
1	AI	Artificial Intelligence	Trí tuệ nhân tạo
2	API	Application Programming Interface	Giao diện lập trình ứng dụng
3	C&C	Command and Control	Máy chủ điều khiển
4	CIM	Common Information Model	Mô hình thông tin chung
5	CLI	Command Line Interface	Giao diện dòng lệnh
6	CPU	Central Processing Unit	Bộ xử lý trung tâm
7	CSRF	Cross-Site Request Forgery	Giả mạo yêu cầu liên trang
8	DDoS	Distributed Denial of Service	Tấn công từ chối dịch vụ phân tán
9	DNS	Domain Name System	Hệ thống phân giải tên miền
10	DS	Deployment Server	Máy chủ triển khai
11	EDR	Endpoint Detection and Response	Phát hiện và phản ứng trên thiết bị đầu cuối
12	GDPR	General Data Protection Regulation	Quy định bảo vệ dữ liệu chung
13	HF	Heavy Forwarder	Bộ chuyển tiếp nâng cao
14	HTTP	Hypertext Transfer Protocol	Giao thức truyền tải siêu văn bản
15	IOC	Indicator of Compromise	Chỉ báo xâm nhập
16	IP	Internet Protocol	Giao thức Internet / địa chỉ mạng
17	ISO 27001	International Organization for Standardization 27001	Tiêu chuẩn quản lý an toàn thông tin
18	JSON	JavaScript Object Notation	Định dạng dữ liệu JSON
19	LLM	Large Language Model	Mô hình ngôn ngữ lớn
20	MTTR	Mean Time to Respond	Thời gian phản hồi trung bình
21	OID	Object Identifiers	Định danh đối tượng
22	PCI DSS	Payment Card Industry Data Security Standard	Tiêu chuẩn bảo mật dữ liệu thẻ thanh toán
23	RAG	Retrieval-Augmented Generation	Sinh tăng cường truy xuất
24	RDP	Remote Desktop Protocol	Giao thức máy tính từ xa
25	SACL	System Access Control List	Danh sách kiểm soát truy cập hệ thống

26	SEM	Security Event Management	Quản lý sự kiện bảo mật
27	SID	Search ID	Mã định danh tìm kiếm
28	SIEM	Security Information and Event Management	Quản lý thông tin và sự kiện bảo mật
29	SIM	Security Information Management	Quản lý thông tin bảo mật
30	SNMP	Simple Network Management Protocol	Giao thức quản lý mạng đơn giản
31	SOC	Security Operations Center	Trung tâm điều hành an ninh mạng
32	SPL	Search Processing Language	Ngôn ngữ xử lý tìm kiếm
33	TA	Technical Add-on	Tiện ích bổ sung kỹ thuật
34	tsidx	Time Series Index	Chỉ mục chuỗi thời gian
35	UF	Universal Forwarder	Bộ chuyển tiếp phổ dụng
36	XSS	Cross-Site Scripting	Tấn công chèn mã lệnh liên trang

LỜI MỞ ĐẦU

1. Tính cấp thiết của đề tài

Trong kỷ nguyên chuyển đổi số, sự gia tăng về quy mô và tính phức tạp của các cuộc tấn công mạng đang trở thành thách thức lớn đối với mọi tổ chức. Các kỹ thuật tấn công hiện đại không còn đơn thuần là phá hoại mà tập trung vào việc ẩn mình, thực hiện các hành vi bất thường để xâm nhập sâu vào hệ thống. Việc quản lý và giám sát thủ công các tệp nhật ký (log) từ hàng trăm thiết bị khác nhau trở nên bất khả thi do khối lượng dữ liệu khổng lồ và sự thiếu tính nhất quán.

Hệ thống quản lý thông tin và sự kiện bảo mật (SIEM) nổi lên như một giải pháp cốt lõi, trong đó Splunk được đánh giá là nền tảng hàng đầu nhờ khả năng thu thập, phân tích và truy vấn dữ liệu theo thời gian thực mạnh mẽ. Tuy nhiên, việc triển khai Splunk một cách hiệu quả không chỉ dừng lại ở việc lưu trữ log, mà quan trọng hơn là khả năng xây dựng các quy tắc phát hiện (rules) dựa trên các khung tham chiếu chuẩn quốc tế như MITRE ATT&CK và chuẩn hóa chúng thông qua Sigma Rule.

Bên cạnh đó, một hệ thống giám sát chỉ thực sự có giá trị khi thông tin về các mối đe dọa được chuyển đến bộ phận phản ứng kịp thời. Việc nghiên cứu cơ chế làm giàu dữ liệu (Data Enrichment) và tự động hóa quy trình cảnh báo qua nền tảng Telegram là vô cùng cần thiết, giúp rút ngắn thời gian phản ứng (MTTR) trước các sự cố an ninh.

Ngoài ra, với sự phát triển của trí tuệ nhân tạo, việc ứng dụng mô hình ngôn ngữ lớn (LLM) vào hỗ trợ phân tích log và điều tra sự cố đang trở thành xu hướng trong các trung tâm vận hành an ninh (SOC), giúp giảm tải cho đội ngũ phân tích và nâng cao hiệu quả phát hiện mối đe dọa.

Vì vậy, đề tài "**Xây dựng hệ thống Splunk phát hiện hành vi thông qua rules và cảnh báo tự động**" có ý nghĩa thực tiễn cao, đáp ứng nhu cầu cấp bách trong công tác bảo mật và quản trị hệ thống hiện nay, đồng thời mở ra hướng tiếp cận mới trong việc ứng dụng AI hỗ trợ vận hành SOC.

2. Mục tiêu nghiên cứu đề tài

Mục tiêu chính của đề tài là xây dựng hoàn thiện một hệ thống giám sát an ninh dựa trên nền tảng Splunk, có khả năng tự động phát hiện các hành vi tấn công đặc trưng và gửi cảnh báo tức thời tới người quản trị.

Các mục tiêu cụ thể bao gồm:

- **Về lý thuyết:** Nghiên cứu sâu về kiến trúc hệ thống SIEM Splunk, ngôn ngữ truy vấn SPL, khung kỹ chiến thuật MITRE ATT&CK và cú pháp xây dựng Sigma Rule và khả năng ứng dụng của mô hình ngôn ngữ lớn (LLM) trong vận hành bảo mật.

- **Về kỹ thuật:** Thiết lập môi trường thực nghiệm thu thập log từ đa dạng nguồn như máy chủ Windows, Linux và tường lửa pfSense.
 - Xây dựng và chuẩn hóa các quy tắc phát hiện hành vi tấn công dựa trên Sigma Rule và chuyển đổi sang SPL.
 - Phát triển module xử lý việc gửi thông báo tự động qua Telegram Bot.
 - Tích hợp mô hình ngôn ngữ lớn (LLM) để hỗ trợ truy vấn logs bằng ngôn ngữ tự nhiên, phân tích ngữ cảnh sự kiện và điều tra sự cố.
- **Về trực quan hóa:** Thiết kế các Dashboard giám sát tổng thể để theo dõi trạng thái an ninh của toàn bộ hệ thống theo thời gian thực.

3. Đối tượng và phạm vi nghiên cứu

- **Đối tượng nghiên cứu:**
 - Hệ thống SIEM Splunk và các thành phần liên quan.
 - Các hành vi, kỹ thuật tấn công mạng dựa trên khung MITRE ATT&CK.
 - Quy trình xây dựng, quản lý rule phát hiện (Sigma Rule, SPL) và cơ chế tự động hóa cảnh báo.
 - Mô hình ngôn ngữ lớn (LLM) và ứng dụng trong vận hành bảo mật.
- **Phạm vi nghiên cứu:**
 - Triển khai thực nghiệm trong môi trường Lab với các thực thể bao gồm: Máy chủ Windows, máy chủ Linux và tường lửa pfSense.
 - Tập trung phát hiện kịch bản tấn công T1219 – Remote Access Software, sử dụng AnyDesk thiết lập kênh điều khiển từ xa.

4. Nhiệm vụ cần thực hiện

Nội dung nghiên cứu tập trung vào các nội dung chính như sau:

- Nghiên cứu lý thuyết về SIEM, Splunk Enterprise, ngôn ngữ SPL, MITRE ATT&CK, Sigma Rule và mô hình ngôn ngữ lớn (LLM).
- Thiết kế mô hình hệ thống Splunk Enterprise với các thành phần: Search Head, Indexer, Forwarder (UF/HF), Deployment Server, Master Cluster. Triển khai trên môi trường ảo hoá với các máy chủ Windows, Linux và pfSense.
- Xây dựng các rule phát hiện dựa trên Sigma Rule, chuyển đổi sang SPL để phát hiện hành vi tấn công đặc trưng.
- Phát triển cơ chế cảnh báo tự động qua Telegram, thiết kế Dashboard giám sát và tích hợp AI hỗ trợ truy vấn, phân tích và điều tra sự cố.

5. Kết quả dự kiến

- **Lý thuyết:**

- Nắm vững kiến trúc và quy trình vận hành của hệ thống SIEM Splunk trong môi trường doanh nghiệp.
 - Hiểu sâu về các kỹ chiến thuật tấn công trong ma trận MITRE ATT&CK để áp dụng vào việc phân tích hành vi.
 - Làm chủ ngôn ngữ Python, ngôn ngữ truy vấn SPL và cú pháp xây dựng Sigma Rule, có khả năng chuyển đổi linh hoạt giữa các định dạng luật phát hiện.
 - Hiểu rõ quy trình chuẩn hóa dữ liệu theo mô hình thông tin chung (CIM) để tối ưu hóa hiệu suất hệ thống.
 - Nắm được nguyên lý hoạt động của mô hình ngôn ngữ lớn (LLM) và các kỹ thuật Prompt Engineering, RAG trong ứng dụng hỗ trợ vận hành bảo mật.
- **Thực nghiệm:**
- Thiết lập thành công hệ thống Splunk thu thập logs từ máy chủ Windows, Linux và tường lửa pfSense về một kho lưu trữ tập trung.
 - Xây dựng được các rule SPL phát hiện hành vi tấn công T1219 - Remote Access Software (AnyDesk) và các kịch bản liên quan.
 - Phát triển cơ chế cảnh báo tự động qua Telegram với nội dung được làm giàu thông tin.
 - Triển khai cơ chế làm giàu dữ liệu thông qua Lookup và Knowledge Objects trên Search Head.
 - Thiết kế Dashboard giám sát trực quan trên Splunk và đánh giá hệ thống thông qua kịch bản tấn công mô phỏng.

CHƯƠNG 1. TỔNG QUAN VỀ SIEM VÀ ỨNG DỤNG AI TRONG SPLUNK

1.1. HỆ THỐNG SIEM

1.1.1. Tổng quan về SIEM

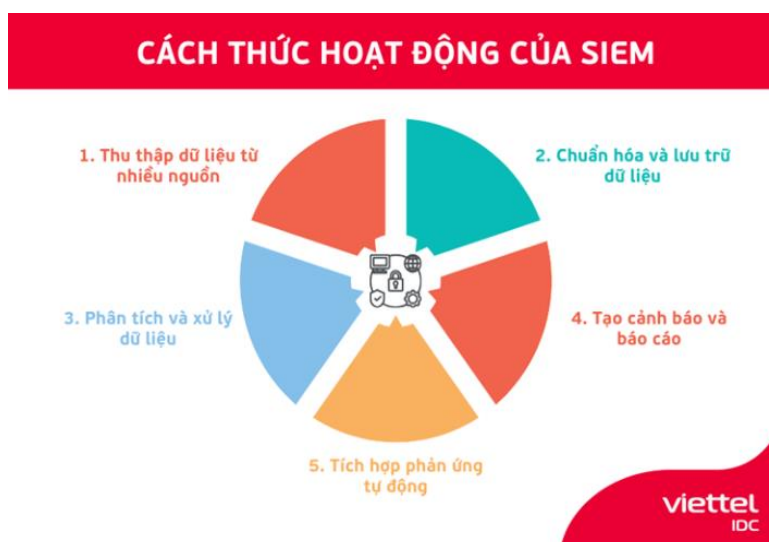
SIEM là một giải pháp công nghệ kết hợp giữa quản lý thông tin bảo mật (SIM) và quản lý sự kiện bảo mật (SEM), được thiết kế để giải quyết những thách thức trong việc thu thập, quản lý và phân tích khối lượng dữ liệu khổng lồ phát sinh từ hạ tầng công nghệ thông tin phức tạp. Về bản chất, SIEM là một nền tảng tập trung hóa dữ liệu máy móc từ đa dạng nguồn trong hệ thống mạng (máy chủ, thiết bị mạng, ứng dụng, thiết bị bảo mật) để tạo ra một kho dữ liệu thống nhất. Bằng cách này, SIEM giúp doanh nghiệp có cái nhìn toàn diện về toàn bộ hạ tầng IT, từ đó nhận diện, theo dõi và phản ứng nhanh với các mối đe dọa.

Hệ thống SIEM đóng vai trò là trung tâm điều hành trong chiến lược an ninh mạng, giúp doanh nghiệp giải quyết bài toán quản trị dữ liệu máy móc khổng lồ và phức tạp. Vai trò đầu tiên của SIEM là tập trung hóa và hiển thị dữ liệu, thực hiện việc thu thập log từ đa dạng nguồn phân tán như tường lửa, thiết bị mạng, máy chủ và ứng dụng để tạo ra một kho lưu trữ thống nhất. Quá trình này giúp loại bỏ các điểm mù trong hệ thống, cho phép quản trị viên có cái nhìn toàn diện về tình trạng hoạt động của toàn bộ hạ tầng hệ thống máy. SIEM đảm nhận vai trò giám sát và phát hiện sự cố theo thời gian thực thông qua các công nghệ so sánh mẫu, phân tích hành vi và học máy để nhận diện sớm các bất thường, các cuộc tấn công hoặc các mối đe dọa nâng cao. Bên cạnh khả năng phát hiện, SIEM còn đóng vai trò then chốt trong Digital Forensics và điều tra sự cố. Nhờ khả năng lưu trữ dữ liệu lịch sử chi tiết, các chuyên gia an ninh có thể tương quan các sự kiện để truy tìm nguyên nhân gốc rễ của cuộc tấn công. Một vai trò chiến lược khác là quản lý tuân thủ, giúp doanh nghiệp tự động hóa báo cáo để đáp ứng các tiêu chuẩn bảo mật quốc tế như ISO 27001, PCI DSS hay GDPR. Cuối cùng, hệ thống này còn hỗ trợ tối ưu hóa vận hành và ra quyết định, bằng cách chuyển đổi dữ liệu thô thành thông tin có ý nghĩa, giúp người quản lý đưa ra các quyết định chiến lược dựa trên dữ liệu thực tế thay vì phỏng đoán.

Quy trình vận hành của SIEM là một chuỗi các bước xử lý dữ liệu nghiêm ngặt nhằm biến các sự kiện thô thành thông tin cảnh báo có giá trị và được sắp xếp phân loại rõ ràng để dễ dàng quản lý. Quy trình bắt đầu bằng bước thu thập dữ liệu từ các nguồn trong hệ thống, ngay sau đó là giai đoạn chuẩn hóa và lưu trữ. Tại đây, dữ liệu được đưa về một định dạng nhất quán để dễ dàng phân tích và áp dụng quản lý lưu trữ phân tầng (hot, warm, cold, frozen) để tiết kiệm tài nguyên lưu trữ. Giai đoạn trọng tâm là phân tích và xử lý, nơi SIEM sử dụng các quy tắc tương quan để đối chiếu dữ liệu với các mẫu tấn công đã biết hoặc sử dụng trí tuệ nhân tạo để dự đoán các rủi ro tiềm ẩn.

Kết quả của quá trình phân tích sẽ dẫn đến việc tạo cảnh báo và báo cáo tự động. Các cảnh báo này được phân loại theo mức độ ưu tiên, giúp đội ngũ bảo mật tập trung nguồn lực vào các vấn đề cấp bách nhất để tối ưu hóa thời gian phản ứng. Trong các giải pháp SIEM hiện đại, bước cuối cùng là tích hợp phản ứng tự động, cho phép hệ thống thực thi ngay lập tức các biện pháp ngăn chặn như:

- Chặn địa chỉ IP độc hại hoặc cô lập các thiết bị đã bị xâm nhập.
- Khóa tài khoản người dùng có những hành vi đáng ngờ.
- Gửi các lệnh khắc phục sự cố trực tiếp tới các hệ thống bảo mật khác trong mạng để giảm thiểu tối đa thiệt hại.



Hình 1.1. Cách thức hoạt động của SIEM

1.1.2. Cơ chế phát hiện mối đe dọa

Để hiện thực hóa vai trò bảo mật chủ động cho doanh nghiệp trong kỷ nguyên số, hệ thống SIEM không chỉ đơn thuần là một kho lưu trữ log mà còn vận hành như bộ máy để phân tích. Sức mạnh cốt lõi của nền tảng này nằm ở khả năng biến khối lượng dữ liệu máy khổng lồ, bao gồm có cấu trúc, bán cấu trúc và phi cấu trúc thành các thông tin chi tiết dễ tiếp cận thông qua việc kết hợp giữa công nghệ hiện đại và tư duy xử lý dữ liệu logic. Quy trình này giúp tổ chức không chỉ nhìn thấy những hành vi bất thường đang xảy ra trên hệ thống mà còn dự báo và ngăn chặn được các kịch bản tấn công phức tạp nhất thông qua những cơ chế phát hiện đặc biệt.

Phân tích thông minh thông qua đa công nghệ: Hệ thống SIEM tận dụng sự kết hợp của ba công nghệ phân tích tiên tiến để xử lý khối lượng dữ liệu khổng lồ và phát hiện rủi ro. Trước hết, kỹ thuật so sánh đóng vai trò đối chiếu các sự kiện log thu thập được với danh sách những mẫu hành vi tấn công hoặc chữ ký mã độc đã được nhận diện trước đó. Tiếp nối là phân tích hành vi, cơ chế này

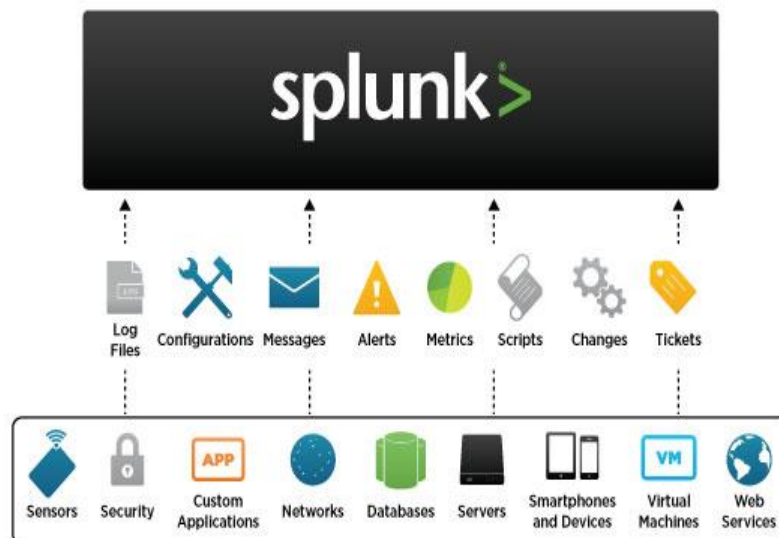
giúp hệ thống thiết lập một trạng thái bình thường cho người dùng và hệ thống dựa trên lịch sử hoạt động, từ đó tự động nhận diện các hành vi lệch lạc đáng nghi. Cuối cùng là Machine Learning cho phép SIEM tự học hỏi từ các xu hướng và dữ liệu lịch sử để dự đoán những rủi ro tiềm ẩn, giúp tổ chức chủ động đối phó với các cuộc tấn công chưa từng được biết đến, hay còn gọi là tấn công Zero-day.

Cơ chế tương quan sự kiện đa nguồn: sức mạnh cốt lõi giúp SIEM được tin dùng hơn các công cụ quản lý log truyền thống chính là khả năng tương quan sự kiện, cho phép liên kết những mảnh ghép dữ liệu rời rạc từ tường lửa, máy chủ, ứng dụng cho đến thiết bị mạng thành một chuỗi tấn công logic. Thông qua các ngôn ngữ truy vấn mạnh mẽ như SPL, người dùng có thể viết các câu lệnh để xác định mối quan hệ giữa các sự kiện. Đặc biệt, tính năng Transaction cho phép nhóm các sự kiện có chung đặc điểm nhận dạng để theo dõi toàn bộ quá trình thực hiện một hành vi đáng ngờ, từ đó có cái nhìn hệ thống về sự cố thay vì chỉ xem xét từng sự kiện đơn lẻ.

Giám sát và cảnh báo thời gian thực: hệ thống vận hành cơ chế giám sát thời gian thực để đảm bảo mọi dữ liệu máy móc phát sinh đều được kiểm tra ngay lập tức nhằm phát hiện sớm các bất thường. Các quy tắc cảnh báo được thiết lập để chạy liên tục dưới nền, tự động quét tìm những dấu hiệu vi phạm chính sách bảo mật hoặc lỗi hệ thống nghiêm trọng. Người quản trị có thể tùy chỉnh các điều kiện kích hoạt linh hoạt, ví dụ như chỉ kích hoạt báo động khi số lượng lỗi đăng nhập vượt quá một ngưỡng nhất định trong một khoảng thời gian ngắn (ví dụ: hơn 100 lỗi trong 1 phút). Khi các điều kiện này được thỏa mãn, hệ thống sẽ thực thi ngay các hành động phản ứng như gửi thông báo qua email, Webhook tới các ứng dụng như Telegram, hoặc tự động chạy các script để ngăn chặn nguồn tấn công.

1.2. Kiến trúc hệ thống Splunk

1.2.1. Tổng quan về Splunk



Hình 1.2. Tổng quan về Splunk

Trong bối cảnh các mối đe dọa an ninh mạng ngày càng tinh vi và khối lượng dữ liệu phát sinh từ hạ tầng công nghệ thông tin ngày càng khổng lồ, các phương pháp quản lý log truyền thống đã dần trở nên lỗi thời, không còn đủ khả năng đáp ứng nhu cầu giám sát toàn diện hay phân tích bảo mật chuyên sâu. Trước thực trạng đó, các hệ thống SIEM đã ra đời và đóng vai trò tập trung hóa dữ liệu và cung cấp khả năng giám sát, phát hiện sự cố theo thời gian thực để bảo vệ tổ chức. Trong số các giải pháp SIEM tiên tiến hiện nay, Splunk Enterprise nổi lên như một nền tảng hàng đầu, không chỉ đơn thuần thực hiện các chức năng của một công cụ quản lý sự kiện mà còn vận hành như một hệ sinh thái dữ liệu. Với khả năng thu thập mọi loại dữ liệu từ mọi nguồn, lập chỉ mục mạnh mẽ và sử dụng ngôn ngữ truy vấn SPL linh hoạt, Splunk đã hiện thực hóa các lý tưởng của SIEM bằng cách biến những khối dữ liệu máy móc thô, phi cấu trúc thành những thông tin chi tiết có giá trị thực tiễn và trực quan. Sự chuyển dịch từ tư duy quản lý log thông thường sang việc ứng dụng sức mạnh phân tích của Splunk giúp doanh nghiệp nâng cao tư thế bảo mật và tối ưu hóa hiệu suất vận hành trong kỷ nguyên số.

Ngoài ra, Splunk Enterprise là một hệ sinh thái linh hoạt có thể kết nối nhiều ứng dụng thông qua kỹ thuật Add-on. Điều này giúp hệ thống mở rộng khả năng tích hợp như tự động hóa công việc và lấy thông tin của các thiết bị. Thông qua các mô hình dữ liệu và chuẩn CIM được cấu hình sẵn, dữ liệu từ nhiều nguồn khác nhau được chuẩn hóa về một ngôn ngữ chung giúp việc phân tích trở nên dễ dàng hơn. Việc ứng dụng hệ sinh thái đa dạng các công cụ và ứng dụng không chỉ giúp giảm thiểu công sức quản trị mà còn giúp người sử dụng tự do tùy biến hệ thống theo nhu cầu đặc thù, biến Splunk thành một nền tảng toàn diện.

Kiến trúc của Splunk được thiết kế như một hệ sinh thái toàn diện và linh hoạt, có khả năng mở rộng từ một máy chủ đơn lẻ cho đến các cụm máy chủ khổng lồ để xử lý khối lượng dữ liệu máy móc ở mọi quy mô. Mục tiêu cốt lõi của kiến trúc này là chuyển đổi dữ liệu thô, phi cấu trúc từ đa dạng nguồn thành thông tin chi tiết dễ tiếp cận hơn để có thể hành động. Tùy theo nhu cầu về hiệu suất và tính sẵn sàng, doanh nghiệp có thể triển khai theo các mô hình sau:

- **Mô hình Standalone:** Tất cả các chức năng (thu thập, lập chỉ mục, tìm kiếm) tích hợp trên một máy chủ duy nhất.
- **Mô hình Distributed:** Các chức năng được tách biệt lên nhiều máy chủ riêng biệt, Forwarders thu thập, Indexers lưu trữ và Search Heads thực hiện tìm kiếm. Mô hình này giúp tăng hiệu suất xử lý, hỗ trợ nhiều người dùng hơn và cho phép mở rộng linh hoạt từng thành phần theo nhu cầu.
- **Mô hình Cluster:** Là kiến trúc cao cấp nhất, kết hợp mô hình phân tán với cơ chế tự động sao chép dự phòng. Dữ liệu sẽ được phân thành từng Bucket, mỗi khi 1 Indexer nhận dữ liệu từ nguồn, thì một bản dữ liệu nén sẽ được gửi và lưu vào các indexer còn lại. Việc này giúp đảm bảo dữ liệu không bị mất và máy tính vẫn hoạt động bình thường.

1.2.2. Các thành phần trong Splunk

Hệ thống Splunk Enterprise được cấu thành từ ba thành phần xử lý dữ liệu chính đóng vai trò nền tảng cho toàn bộ hoạt động của hệ thống.



Indexer



Search Head



Forwarder

Hình 1.3. Các thành phần chính trong Splunk

a) Search Head:

Search Head đóng vai trò giao diện người dùng và bộ điều phối truy vấn. Đây là nơi người dùng thực hiện tìm kiếm bằng ngôn ngữ SPL, xây dựng dashboard và thiết lập cảnh báo. Search Head không trực tiếp lưu trữ dữ liệu mà thực hiện nhiệm vụ gửi yêu cầu đến các Indexer, sau đó tập hợp và hiển thị kết quả cuối cùng cho người dùng. Ngoài ra, các Knowledge Objects trên Search Head còn cho phép làm giàu và biến đổi dữ liệu mà không làm thay đổi dữ liệu gốc trong Index. Các thành phần có trong Knowledge Objects có thể bao gồm:

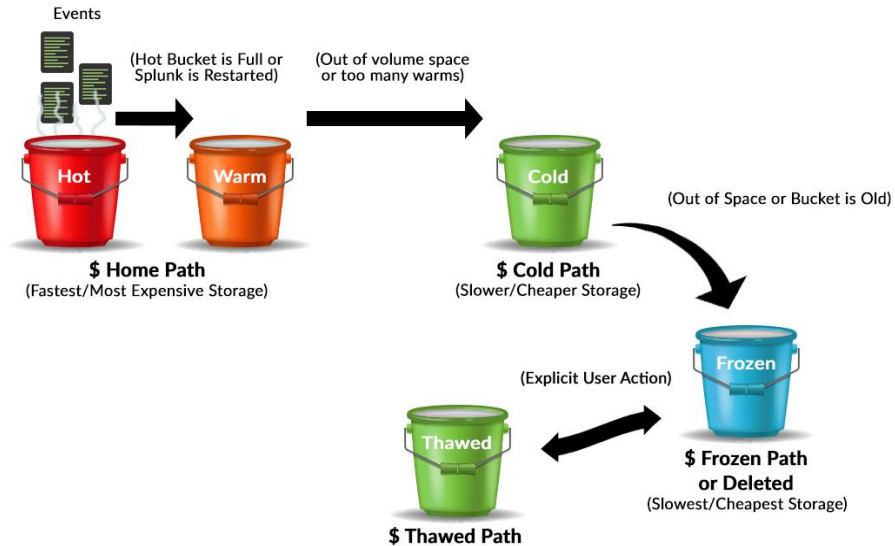
- **Field extraction and fields:** cắt các mảng dữ liệu thành các mảng event nhỏ hơn.

- **Alias:** thay đổi tên các mảng event nhỏ, ví dụ src_id sẽ được đổi thành Source_id.
- **Lookup:** ánh xạ một giá trị trong event tới một nguồn dữ liệu khác và kết nối với event ban đầu, tạo 1 trường mới. Ví dụ: Source_id = 1.2.3.4, sau khi ánh xạ với nguồn dữ liệu của địa chỉ IP thì thấy địa chỉ này thuộc phòng ban nhân sự.
- **Event types:** phân loại các dòng log có đặc điểm chung thì sẽ cho vào cùng 1 thư mục. Ví dụ: status=404 thì cho vào thư mục lỗi.
- **Workflow Action:** tương tác trực tiếp với dữ liệu, bao gồm:
 - **Tạo HTML post:** mở website/tool bên ngoài giá trị field để tra cứu theo thời gian thực.
 - **Gửi http post:** gửi dữ liệu hệ thống ra bên ngoài. Ví dụ: khách hàng gặp vấn đề về tài khoản, người giám sát sẽ tạo event hỗ trợ cho tài khoản này, ticket sẽ có các trường thông tin tự động điền và sẽ gửi cho bên chăm sóc khách hàng một cách tự động.
- **Tags:** gán 1 nhãn cho 1 hoặc nhiều trường dữ liệu khác nhau, không gán nhãn cho hẳn trường mà gán nhãn cho giá trị trường trả về. Ví dụ: tag nguy_hiem sẽ bao gồm status (404), IP (check TI), địa chỉ vật lý (nga). Vậy khi search lại tag nguy_hiem thì nó sẽ hiện đồng thời cả 3 thay vì phải thỏa mãn đủ cả 3 điều kiện.
- **Saved searches:** lưu lại các câu lệnh phức tạp dưới 1 label.
- **Alert:** là 1 dạng của saved search, alert có thể chạy tự động (liên tục hoặc định kỳ) nếu gặp điều kiện thỏa mãn mà người dùng đã đặt ra từ trước thì sẽ tự động kích hoạt cảnh báo.
- **Report:** báo cáo lại một cách định kỳ dựa trên các thông tin đã thu thập được (chart, bảng, giấy report,...).

b) Indexer:

Indexer được coi là nơi lưu trữ và xử lý của Splunk. Indexer chịu trách nhiệm tiếp nhận dữ liệu từ các Forwarder, thực hiện phân tách, lập chỉ mục để tối ưu hóa hiệu năng truy vấn. Dữ liệu tại đây được tổ chức theo cấu trúc Bucket phản ánh vòng đời từ mới đến cũ (Hot, Warm, Cold, Frozen) để tối ưu hóa theo vòng đời từ khi mới nạp cho đến khi hết hạn. Dữ liệu sau khi được Indexer lập chỉ mục sẽ được ghi trực tiếp vào Hot Bucket, đây là tầng dữ liệu mới nhất và là loại bucket duy nhất trong hệ thống có trạng thái cho phép ghi. Khi một Hot Bucket đạt đến giới hạn về dung lượng hoặc thời gian theo cấu hình, nó sẽ được luân chuyển thành Warm Bucket ở trạng thái chỉ đọc, và theo thời gian sẽ tiếp tục chuyển xuống Cold Bucket – tầng dữ liệu cũ thường được lưu trữ trên các thiết bị có chi phí thấp hơn để tối ưu kinh phí cho doanh nghiệp. Khi dữ liệu vượt quá thời hạn lưu giữ theo chính sách (ví dụ: sau 1 năm), nó sẽ chuyển sang trạng thái Frozen và bị xóa bỏ hoàn toàn hoặc chuyển vào kho lưu trữ ngoài nếu được thiết lập cài đặt từ trước theo ý người sử dụng, khiến người dùng không thể tìm kiếm trực tiếp trên giao diện trừ khi được khôi phục thủ công vào Thawed Path. Trong

suốt các giai đoạn lưu trữ này (ngoại trừ Frozen), mỗi bucket luôn duy trì các thành phần cốt lõi bao gồm dữ liệu thô đã nén, các tệp chỉ mục .tsidx để đảm bảo tốc độ truy vấn cực nhanh và các thông tin metadata để theo dõi thuộc tính sự kiện như Host, source hay sourcetype.



Hình 1.4. Cơ chế Bucket trong Indexer

c) Forwarder:

Forwarder là một agent nhỏ được cài trên máy chủ/thiết bị nhằm thu thập dữ liệu. Trong đó, có 2 kiểu forwarder: Universal Forwarder và Heavy Forwarder.

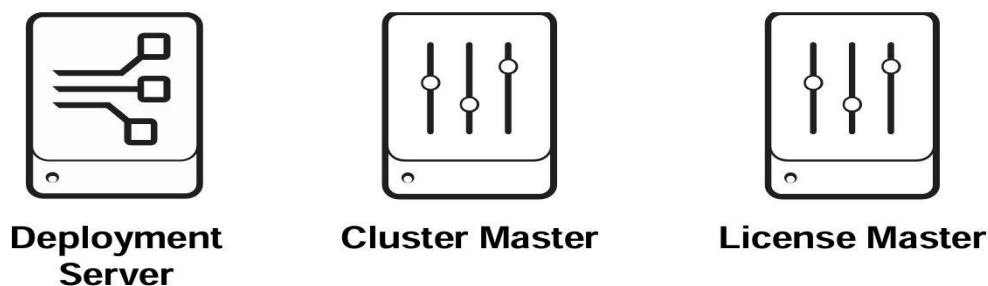
Bảng 1.1. So sánh Universal Forwarder và Heavy Forwarder

Đặc điểm	Universal Forwarder	Heavy Forwarder
Kích thước	Nhẹ	Rất nặng
Tài nguyên	Tiêu tốn ít	Tiêu tốn nhiều
Xử lý dữ liệu	Không	Có
Lọc dữ liệu	Không	Có
Gửi nhiều đích	Không	Có
Vị trí cài đặt	Máy chủ	Máy trung gian
Chi phí	Thấp	Cao

- Dùng Universal Forwarder khi chỉ cần thu thập dữ liệu đơn giản, tiết kiệm tài nguyên trên máy chủ, hệ thống ít máy chủ và dữ liệu về Indexer không quá lớn, chỉ gửi tới 1 địa chỉ duy nhất.
- Dùng heavy Forwarder khi cần lọc dữ liệu trước khi về indexer, hệ thống nhiều máy chủ và dữ liệu về lớn, cần gửi tới nhiều Indexer khác nhau (Indexer về bảo mật, Indexer về vận hành,...) nhằm phân loại và tránh làm hệ thống quá tải.

Sự khác biệt giữa dùng syslog và UF/HF: Syslog dùng giao thức UDP, gửi và mặc kệ, data không mã hóa, nó ở dạng bản rõ. Trong khi đó, UF/HF là Splunk to Splunk, mã hóa và đảm bảo đến nơi, trong trường hợp không thể đến thì dữ liệu vẫn còn được lưu ở bộ nhớ đệm. Trước khi gửi đã dán nhãn cho dữ liệu, gửi thêm thông tin về host, source và sourcetype (win hay linux). Bởi vì firewall không thể nào cài UF nên chúng ta mới cần syslog đẩy về UF (hay còn gọi là UF syslog) để đảm bảo cho dữ liệu cần xử lý, định dạng lại dữ liệu.

Bên cạnh đó, để quản lý các hệ thống quy mô lớn, Splunk còn bao gồm các thành phần quản trị chuyên biệt:



Hình 1.5. Các thành phần phụ trong Splunk

- **Cluster Master:** thực hiện giám sát Indexer (còn sống hay chết, tình trạng hoạt động), đảm bảo đủ bản sao (khi dữ liệu đến indexer sẽ tự động lưu bản sao của nó cho các Indexer khác), điều phối khi gặp sự cố (khi có 1 Indexer lỗi thì Cluster Master sẽ chuyển hướng sang các Indexer khác, dữ liệu có bản sao nên không sợ bị mất dữ liệu, sau khi sửa xong thì sẽ copy lại phần dữ liệu bị mất và hoạt động như bình thường).
- **Search Head Cluster:** quản lý các gói ứng dụng, đẩy các gói ứng dụng và bộ lọc xuống Search head và đồng bộ cấu hình cho cụm Search Head. Trong cụm search head, sẽ có một search head được bầu ngẫu nhiên làm captain để điều phối hoạt động tìm kiếm và lập lịch. Và khi captain hỏng, SHC sẽ phát hiện ra và tự động bầu 1 search head khác làm captain.
- **Deployment Server (DS):** đóng vai trò là máy chủ quản lý tập trung, cho phép đẩy các gói ứng dụng và cấu hình xuống Forwarder trong toàn mạng.
- **License Master:** đảm nhiệm vai trò kiểm soát lưu lượng index mỗi ngày. Lưu lượng này được tính tại indexing phase, tính theo dữ liệu gốc trước khi nén. Luồng đo lường diễn ra như sau: UF thu thập dữ liệu, sau đó gửi về Indexer, Indexer thực hiện Parsing (xử lý thành events), tại thời điểm này License Meter tiến hành đo lường (trước khi nén và ghi vào disk), sau đó dữ liệu mới được ghi vào Disk. Splunk cung cấp các loại license chính với đặc điểm khác nhau:
 - Splunk cung cấp các loại license chính với đặc điểm khác nhau. Cụ thể, license Enterprise dùng cho tổ chức lớn, giới hạn GB theo ngày; license Free giới hạn 500mb/ngày và không có tính năng xác nhận người dùng; license

Trial có thời hạn 60 ngày, giới hạn 500mb/ngày nhưng được cung cấp đầy đủ tính năng; còn license Forwarder là miễn phí và dùng cho UF.

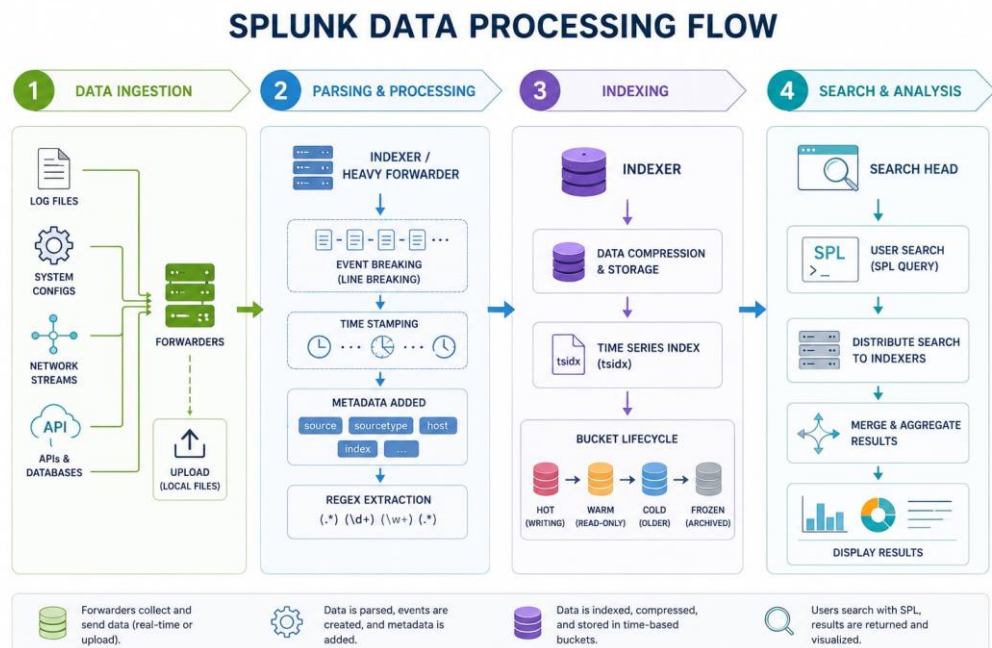
- Trong trường hợp vượt quá giới hạn GB/ngày, Splunk xử lý theo cơ chế phân tầng. Khi vượt quá giới hạn, từ lần 1 đến lần 4, hệ thống chỉ đưa ra cảnh báo. Sang lần thứ 5 trở đi, hệ thống chuyển sang trạng thái vi phạm, khi đó Splunk ngừng cho phép tìm kiếm (vẫn index nhưng không search được). Sau 24 giờ, hệ thống sẽ tự động reset lại.
- Đối với một hệ thống lớn, cơ chế quản lý license được thực hiện thông qua License Server đóng vai trò trung tâm. License Server quản lý license cho toàn bộ hệ thống. Các Indexer (Indexer 1, Indexer 2, Indexer 3,...) sẽ báo cáo lượng data về License Server, sau đó License Server tổng hợp và kiểm tra giới hạn.

Bảng 1.2. Đặc điểm của các loại License

Loại	Đặc điểm
Enterprise	Dùng cho tổ chức lớn, giới hạn GB theo ngày
Free	500mb/ngày, không có tính năng xác nhận người dùng
Trial	60 ngày, 500mb/ngày, đầy đủ tính năng
Forwarder	Miễn phí, dùng cho UF

1.2.3. Luồng xử lý dữ liệu trong Splunk

Luồng dữ liệu trong Splunk tuân theo một chu trình nghiêm ngặt nhằm đảm bảo tính toàn vẹn và tốc độ truy vấn, chia làm bốn giai đoạn chính.



Hình 1.6. Luồng xử lý dữ liệu trong Splunk

Giai đoạn đầu tiên là Nạp dữ liệu, dữ liệu khởi đầu từ các nguồn máy móc thô như tệp nhật ký, cấu hình hệ thống, luồng dữ liệu mạng, hoặc thông tin từ các API và cơ sở dữ liệu. Để thu thập khối lượng dữ liệu này, Splunk sử dụng các Forwarder, là những agent được cài đặt trực tiếp trên các nguồn phát sinh dữ liệu để chuyển tiếp log về hệ thống trung tâm mà không gây ảnh hưởng lớn đến hiệu suất máy chủ. Ngoài gửi dữ liệu liên tục qua Forwarder, người quản trị có thể sử dụng tùy chọn Upload để tải lên các tệp tin cục bộ phục vụ cho mục đích thử nghiệm hoặc phân tích dữ liệu nạp một lần.

Tiếp theo là giai đoạn Phân tích – phân tách dữ liệu: Sau khi được nạp vào, dữ liệu thô sẽ được chuyển đến Indexer (hoặc Heavy Forwarder) để thực hiện quá trình Parsing và chuẩn hóa cơ bản. Tại đây, Splunk bóc tách luồng dữ liệu không lồ thành các sự kiện riêng lẻ dựa trên các quy tắc ngắt dòng để xác định ranh giới sự kiện và cấu hình mốc thời gian chính xác cho từng sự kiện. Trong giai đoạn này, mỗi sự kiện sẽ được gắn kèm các trường metadata mặc định bao gồm máy chủ nguồn, đường dẫn nguồn dữ liệu và định dạng dữ liệu. Việc gán tags chính xác giúp hệ thống có thể phân loại và truy vấn dữ liệu một cách khoa học ở các bước sau, đồng thời các biểu thức chính quy có thể được áp dụng để bóc tách các mẫu dữ liệu phức tạp.

Giai đoạn thứ ba là Lập chỉ mục, đây là giai đoạn Indexer nén dữ liệu thô và ghi xuống đĩa cứng để lưu trữ lâu dài, đồng thời tạo ra các tệp chỉ mục chuỗi thời gian gọi là tsidx. Các tệp này chứa danh sách từ khóa và các con trỏ đến vị trí dữ liệu thực tế, cho phép hệ thống tìm kiếm cực nhanh mà không cần quét qua toàn bộ dữ liệu thô khổng lồ. Dữ liệu được tổ chức vật lý trong các Bucket phản ánh vòng đời của chúng: bắt đầu từ Hot Bucket (đang ghi), sau đó luân chuyển thành Warm Bucket (chỉ đọc) khi đầy. Theo thời gian, dữ liệu cũ sẽ chuyển xuống Cold Bucket để lưu trữ trên các thiết bị rẻ hơn và cuối cùng là Frozen - trạng thái mà dữ liệu bị xóa hoặc lưu trữ ngoài nếu được thiết lập bởi người quản lý trước đó, không thể tìm kiếm trực tiếp trừ khi được khôi phục vào Thawed Path

Giai đoạn cuối cùng cho phép người dùng khai thác giá trị từ dữ liệu đã lưu trữ thông qua giao diện Search Head. Khi người dùng nhập câu lệnh bằng ngôn ngữ SPL, Search Head đóng vai trò bộ điều phối, gửi yêu cầu truy vấn đến các Indexer, sau đó tập hợp và hợp nhất kết quả trả về để hiển thị cho người dùng.

1.2.4. Xây dựng Alert và Dashboard giám sát

a) Xây dựng Alert

Alert trong Splunk không chỉ là một thông báo đơn thuần, mà được định nghĩa là một lệnh tìm kiếm được lưu trữ và tự động thực thi, đi kèm với các điều kiện đánh giá kết quả cụ thể. Cấu trúc cốt lõi của một Alert bao gồm 3 thành phần kỹ thuật:

- **Search Schedule:** Alert là một tiến trình chạy ngầm, liên tục hoặc định kỳ quét các tập dữ liệu dựa trên một khoảng thời gian và chu kỳ được định nghĩa trước.
- **Trigger Condition:** Đây là bộ quy tắc logic dùng để đánh giá dữ liệu đầu ra của lệnh tìm kiếm. Alert chỉ được kích hoạt khi dữ liệu thỏa mãn một ngưỡng cụ thể đã được thiết lập.
- **Trigger Actions:** Khi điều kiện kích hoạt được thỏa mãn, Alert đóng vai trò là một cơ chế phản ứng chủ động. Nó tự động thực thi các hành động đã được lập trình sẵn nhằm thông báo cho quản trị viên, chẳng hạn như: ghi log vào Triggered Alerts, gửi thông báo qua email, hoặc truyền tải dữ liệu cảnh báo sang các hệ thống bên thứ ba thông qua Telegram.

b) Xây dựng Dashboard

Về mặt kỹ thuật, Dashboard trong Splunk được định nghĩa là một giao diện người dùng tổng hợp, chứa một hoặc nhiều khung hiển thị nhằm biểu diễn dữ liệu trực quan dưới dạng biểu đồ, bảng biểu, bản đồ hoặc danh sách sự kiện.

- **Search Engine:** Bản chất của mỗi Panel trên Dashboard là một lớp đại diện cho một hoặc nhiều câu lệnh tìm kiếm Splunk hoặc các truy vấn Data Model chạy ngầm.
- **Vai trò tập trung hóa:** Dashboard đóng vai trò hội tụ, nó ánh xạ và tập hợp các kết quả từ quá trình phân tích, trích xuất trường và tương quan dữ liệu, giúp tổ chức các luồng thông tin rời rạc thành một bảng toàn cảnh về trạng thái hệ thống, phục vụ trực tiếp cho hoạt động giám sát liên tục.

1.3. Tìm hiểu về MITRE ATT&CK và nguồn dữ liệu

1.3.1. Cấu trúc *Tactic, Technique, Sub-technique*

Ma trận MITRE ATT&CK (A Adversarial Tactics, Techniques, and Common Knowledge) được xây dựng dựa trên góc nhìn của kẻ tấn công, phân cấp theo 3 tầng cốt lõi:

- **Tactic (Chiến thuật - "Why"):** Là mục tiêu chiến thuật mà kẻ tấn công muốn đạt được trong một giai đoạn của chuỗi tấn công (Kill Chain). Ví dụ: Initial Access (Xâm nhập ban đầu), Privilege Escalation (Leo thang đặc quyền), Impact (Gây hậu quả).
- **Technique (Kỹ thuật - "How"):** Là phương thức cụ thể mà kẻ tấn công sử dụng để đạt được mục tiêu chiến thuật. Ví dụ, để đạt được Tactic Command and Control, kẻ tấn công có thể dùng Technique T1219 - Remote Access Software (Sử dụng phần mềm truy cập từ xa).

- **Sub-technique (Kỹ thuật phụ - "Detailed How"):** Là mô tả chi tiết và sâu hơn của một kỹ thuật. Ví dụ, kỹ thuật OS Credential Dumping (T1003) có các kỹ thuật phụ như LSASS Memory (T1003.001).

1.3.2. Ứng dụng MITRE ATT&CK trong phát hiện tấn công

Trong quá trình vận hành hệ thống SIEM, MITRE ATT&CK đóng vai trò là ngôn ngữ chung để chuẩn hóa các cảnh báo.

- **Rule Mapping:** Mỗi câu lệnh SPL khi được thiết lập thành Alert cần được gắn thẻ (tag) với một hoặc nhiều Technique ID tương ứng. Điều này giúp các chuyên gia phân tích nhanh chóng hiểu được bản chất của cảnh báo.
- **Detection Coverage:** Bằng cách đối chiếu các rule đang có trên Splunk với ma trận MITRE, đội ngũ bảo mật có thể đánh giá được hệ thống đang "mù" ở những chiến thuật nào, từ đó ưu tiên viết thêm Sigma Rule để lấp đầy các khoảng trống giám sát.

1.3.3. Windows Event Log và Sysmon

a) Windows Event Log:

Windows Event Log là hệ thống nhật ký được tích hợp sẵn trong các hệ điều hành Windows, có nhiệm vụ ghi lại các sự kiện phát sinh trong quá trình vận hành của hệ điều hành, ứng dụng và các dịch vụ liên quan. Các nhật ký này bao gồm thông tin về trạng thái hoạt động, cảnh báo, lỗi cũng như các sự kiện liên quan đến bảo mật. Nhờ tính chất chi tiết và có cấu trúc, Windows Event Log đóng vai trò là nguồn dữ liệu đầu vào quan trọng cho các hệ thống giám sát và phân tích an ninh mạng như Splunk.

Bên cạnh việc lưu trữ các sự kiện hệ thống và bảo mật, Windows còn tận dụng trình giám sát hiệu suất để thu thập các số liệu hiệu năng chi tiết theo thời gian thực. Thông qua cơ chế xử lý, Splunk có thể bóc tách hiệu quả các bộ đếm từ nhiều thành phần phần cứng khác nhau nhằm tối ưu hóa hiệu suất truy vấn. Cụ thể, dữ liệu về CPU được theo dõi qua các thông số như Processor Time hay User Time, Logical Disk tập trung vào dung lượng trống và tốc độ đọc ghi, trong khi đó, dữ liệu về bộ nhớ và mạng giúp nhận diện các tình trạng quá tải hoặc lỗi truyền tải gói tin. Những thông tin này không chỉ hỗ trợ vận hành hạ tầng mà còn là dữ liệu đầu vào quan trọng cho việc phân tích các hành vi bất thường trong hệ thống. Trong lĩnh vực an ninh mạng, Windows Event Logs là nguồn dữ liệu vô giá, cung cấp các bằng chứng số giúp nhận diện và ngăn chặn những mối đe dọa tinh vi bao gồm các thành phần cốt lõi.

- **System:** Ghi lại các sự kiện từ các thành phần hệ thống của Windows, chẳng hạn như lỗi trình điều khiển, quá trình khởi động, tắt máy, và các thay đổi về cấu hình phần cứng.
- **Setup:** Tập trung vào quá trình cài đặt, cập nhật các gói phần mềm và các bản cập nhật hệ điều hành. Việc giám sát log này giúp đảm bảo không có các phần mềm trái phép được đưa vào hệ thống.
- **Application:** Chứa các sự kiện được ghi lại bởi các ứng dụng hoặc chương trình đang chạy trên Windows. Các lỗi ứng dụng hoặc các hành vi bất thường của phần mềm thường được phản ánh tại đây.
- **Security:** lưu trữ các sự kiện hành vi liên quan tới bảo mật như đăng nhập (thành công hoặc thất bại), quyền truy cập tệp tin, và các thay đổi trong chính sách bảo mật.
- **Forwarded Events:** Chứa các sự kiện được chuyển tiếp từ các máy chủ hoặc thiết bị khác thông qua cơ chế WEF. Nhật ký này cho phép tập trung thu thập và quản lý log từ nhiều hệ thống về một máy chủ trung tâm, hỗ trợ giám sát tập trung, phát hiện sự cố và điều tra an ninh hiệu quả hơn trong môi trường doanh nghiệp.

b) Sysmon:

Sysmon là một công cụ của Microsoft thuộc bộ Sysinternals, được thiết kế để giám sát và ghi lại các hoạt động của hệ thống Windows ở mức độ sâu. Khác với các công cụ ghi log bề mặt, Sysmon hoạt động ở tầng sâu của hệ điều hành thông qua sự kết hợp giữa một trình điều khiển thiết bị và một dịch vụ nền. Khi được cài đặt, Sysmon duy trì trạng thái trên hệ thống qua các lần khởi động lại máy, liên tục giám sát và ghi nhận các hoạt động cốt lõi.

Giá trị cốt lõi của Sysmon đối với hệ thống SIEM nằm ở khả năng cung cấp dữ liệu với độ chi tiết cao, đặc biệt tập trung vào các sự kiện thiết yếu cho việc điều tra và săn lùng mối đe dọa điển hình vài ví dụ như:

- **Giám sát tạo tiến trình (Event ID 1):** Ghi lại toàn bộ thông tin khi một tiến trình mới được tạo ra, bao gồm cả chuỗi lệnh thực thi đầy đủ (Command Line), mã băm (Hash - SHA256/MD5) của tệp tin và thông tin về tiến trình cha. Đây là nền tảng để vẽ lại cây tiến trình và phát hiện các công cụ độc hại như phần mềm điều khiển từ xa trái phép.
- **Giám sát tạo tệp tin:** theo dõi quá trình tạo tệp tin (Event ID 11 - File Create), giúp phát hiện các hành vi thả mã độc.

Bảng 1.3. So sánh Windows Security Event và Sysmon Event

Hành động	Windows Security Event	Sysmon Event	Điểm vượt trội của Sysmon
Process Creation	Event ID 4688 Cần bật Audit Process Creation. Có tên tiến trình và Command Line.	Event ID 1 Ghi nhận mặc định theo cấu hình XML.	1. Hash của file: Có ngay MD5, SHA256 của file thực thi. 2. ProcessGUID: Mã định danh duy nhất giúp theo dõi vòng đời tiến trình xuyên suốt hệ thống. 3. Parent Command Line: Cho biết chính xác câu lệnh của tiến trình cha.
Network Connection	Event ID 5156 Cực kỳ nhiều, ghi nhận cả các kết nối rác bên trong hệ thống.	Event ID 3 Lọc được ngay từ nguồn (ví dụ: chỉ bắt port 3389, 4444 hoặc từ các file exe đáng ngờ).	1. Initiated: Cho biết kết nối này là Outbound/Inbound. 2. Gắn với Process GUID: Biết chính xác file thực thi nào đã mở kết nối mạng này, thay vì chỉ biết IP và Port như 5156.
File Creation	Event ID 4663 Phải cấu hình SACL thủ công trên từng thư mục.	Event ID 11 Gắn rule giám sát bằng XML rất linh hoạt (chỉ giám sát đuôi .exe, .ps1, .dll...).	1. CreationUtcTime: Thời gian tạo file thực tế. 2. RuleName: Tag được tên của loại mã độc hoặc kịch bản tấn công ngay trong log để đẩy thẳng lên Alert.
Registry Modification	Event ID 4657 Phải cấu hình SACL trên từng nhánh Registry.	Event ID 12, 13, 14 Ghi nhận tạo khóa, sửa giá trị, đổi tên khóa bằng regex hoặc wildcard.	1. Details: Hiện thị luôn dữ liệu mới vừa bị ghi vào Registry. 2. Gắn với tiến trình: Biết ngay phần mềm nào vừa lên sửa Registry.
DLL/Image Load	Không có log mặc định tương đương.	Event ID 7 Giám sát file .dll được nạp vào bộ nhớ.	SignatureStatus: Báo cáo xem DLL này có chữ ký điện tử hợp lệ không, hay là DLL giả mạo.

1.3.4. Linux Syslog và Audit Log

a) Linux Syslog

Syslog là một giao thức tiêu chuẩn được sử dụng để gửi nhật ký hệ thống hoặc thông báo sự kiện đến một máy chủ cụ thể, được gọi là máy chủ nhật ký hệ thống hay syslog server. Nó chủ yếu được sử dụng để thu thập các bản ghi thiết bị khác nhau từ một số máy khác nhau ở một vị trí trung tâm để theo dõi và xem xét.

Syslog có định dạng là văn bản rõ. Lưu trữ phân tán vào nhiều file khác nhau trong /var/log (ví dụ: auth.log, syslog, kern.log). Syslog sử dụng các công cụ xử lý văn bản cơ bản của Linux: cat, grep, awk, tail, ... Syslog hoạt động dựa trên sự chủ động cấu hình log của ứng dụng. Nếu phần mềm sshd hay sudo được lập trình để báo cáo log, syslog sẽ ghi lại. Nhưng syslog có nhược điểm là nếu kẻ tấn công dùng một mã độc tự viết không báo cáo log, hoặc tắt/vô hiệu hóa các ứng dụng hệ thống, Syslog sẽ hoàn toàn vô dụng. Đối với phân hệ Syslog, Splunk tập trung khai thác tệp nhật ký tổng hợp là /var/log/messages trên các hệ điều hành dòng Red Hat (RHEL/CentOS) hoặc tệp /var/log/syslog trên dòng Debian (Ubuntu). Đây là kho lưu trữ ghi lại các sự kiện hoạt động chung của toàn hệ thống, các cảnh báo lỗi thuộc ngoài hệ điều hành và thông báo vận hành từ các dịch vụ nền không cấu hình tệp lưu vết riêng. Khi dữ liệu được đẩy về Splunk dưới định danh sourcetype=syslog, các quản trị viên có thể theo dõi tổng thể của hạ tầng và phát hiện sớm các xung đột phần mềm hoặc dịch vụ bị dừng đột ngột.

Một thành phần cốt lõi khác của Syslog là tệp /var/log/secure hoặc /var/log/auth.log. Tệp nhật ký này chuyên dụng cho các tiến trình xác thực người dùng và phân quyền truy cập, chịu trách nhiệm ghi nhận mọi hành vi đăng nhập hệ thống qua giao thức SSH, việc sử dụng lệnh chuyển đổi quyền hạn sudo và các quy trình kiểm soát của Pluggable Authentication Modules. Trong môi trường Splunk, nguồn dữ liệu này được chuẩn hóa với định danh sourcetype=linux_secure, cho phép thực hiện các truy vấn chuyên sâu để nhận diện ngay lập tức các chuỗi hành vi đăng nhập thất bại liên tục, từ đó thiết lập cơ chế phát hiện sớm các cuộc tấn công Brute-force hoặc các truy cập trái phép từ các địa chỉ IP độc hại.

Bên cạnh hai nguồn chính kể trên, Syslog còn bao gồm các tệp nhật ký dịch vụ phụ trợ quan trọng khác như /var/log/cron và /var/log/boot.log. Nhật ký cron chịu trách nhiệm lưu vết lịch sử thực thi của tất cả các tác vụ được lập lịch tự động trong hệ thống, giúp rà soát xem có mã độc nào đang lợi dụng tiến trình nền để duy trì sự hiện diện trái phép hay không. Trong khi đó, tệp boot.log kết hợp cùng dữ liệu từ bộ đệm của nhân hệ điều hành sẽ ghi lại chi

tiết quá trình khởi động hệ thống, hỗ trợ chẩn đoán các lỗi liên quan đến trình điều khiển thiết bị và sự cố phần cứng cấp thấp.

b) Linux Audit Log

Audit là một framework kiểm toán an ninh được tích hợp sâu vào nhân kernel của Linux. Khác với các ứng dụng user-space, `auditd` có vị trí trực tiếp tại cửa giao tiếp giữa phần mềm và phần cứng, cho phép nó giám sát mọi hành vi ở mức độ thấp nhất. Audit có khả năng bắt trực tiếp các System Calls: ngôn ngữ giao tiếp giữa mọi phần mềm và hệ điều hành. Khi muốn mở một file, chạy một lệnh, hay kết nối mạng (`syscall connect`), các thao tác bắt buộc phải đi qua Kernel và bị `auditd` ghi lại chính xác từng tham số. Tập nhật ký kiểm toán cốt lõi đặt tại đường dẫn `/var/log/audit/audit.log` và được quản lý độc lập bởi tiến trình hệ thống `auditd`. Khi tích hợp vào cấu hình của Splunk, luồng dữ liệu này được định danh bằng `sourcetype=auditd`, giúp phân tích và truy tìm nguyên nhân gốc rễ của các sự cố an ninh nghiêm trọng.

Chức năng quan trọng của Audit Log là ghi lại hoạt động một cách chi tiết và không thể đảo ngược các lời gọi hệ thống, mọi hành vi thay đổi thuộc tính hoặc quyền hạn tệp tin, các thay đổi trong cấu hình mạng lõi và việc thực thi lệnh của người dùng đặc quyền. Đặc biệt, tập nhật ký này có khả năng theo dõi sát sao hành vi của tài khoản root hoặc các tài khoản được leo thang quyền lực, ghi lại chính xác định danh thực tế của người dùng trước khi họ chuyển đổi sang quyền cao hơn.

1.3.5. Syslog từ thiết bị mạng và tường lửa

Dữ liệu Syslog từ các thiết bị mạng và tường lửa đóng vai trò quan trọng trong hệ thống SIEM Splunk, giúp cung cấp khả năng giám sát lưu lượng, ngăn chặn tấn công và quản lý kết nối toàn diện cho hạ tầng doanh nghiệp. Khác biệt hoàn toàn với các máy chủ Windows hay Linux vốn có thể cài đặt trực tiếp Universal Forwarder, các thiết bị mạng như Router, Switch và Firewall (Cisco, pfsense) thường là những hệ thống đóng, không cho phép cài đặt phần mềm bên thứ ba. Do đó, quy trình thu thập dữ liệu từ các nguồn này đòi hỏi những cơ chế đặc thù và các thành phần trung gian chuyên biệt để đảm bảo luồng thông tin không bị gián đoạn.

Trong môi trường sản xuất, việc thu thập log từ tường lửa và thiết bị mạng sử dụng một thành phần là Heavy Forwarder đóng vai trò trung gian. HF sẽ được cấu hình để lắng nghe dữ liệu đẩy về thông qua giao thức Syslog, thường là qua cổng 514/UDP (hoặc 1514/UDP đối với các dịch vụ không chạy quyền root). Dữ liệu thô từ các thiết bị như tường lửa pfsense sẽ được đẩy trực tiếp đến địa chỉ IP của HF. Tại đây, HF thực hiện nhiệm vụ tiếp nhận, phân tách sơ bộ và chuyển tiếp về Indexer để lập chỉ mục. Đối với các thiết bị của Cisco, Splunk cung cấp

sẵn các sourcetype chuyên dụng như cisco_firewall, cisco_esa hay cisco_wsa_squid để chuẩn hóa dữ liệu ngay từ giai đoạn nạp.

Bên cạnh cơ chế đẩy log truyền thống, Splunk còn hỗ trợ phương thức kéo dữ liệu thông qua giao thức SNMP (đặc biệt là SNMPv3 để đảm bảo bảo mật). Phương thức này thường được áp dụng để thu thập các thông số hiệu năng và trạng thái của thiết bị mà Syslog có thể bỏ sót. Quản trị viên sẽ thiết lập các script chạy định kỳ trên Heavy Forwarder để truy vấn các OID cụ thể từ thiết bị, chẳng hạn như phần trăm CPU, dung lượng bộ nhớ đã sử dụng, hay băng thông cổng mạng, sau đó ghi các kết quả này vào một tệp nhật ký cục bộ để Splunk giám sát và nạp vào hệ thống. Sự kết hợp giữa Syslog thời gian thực và truy vấn SNMP giúp tổ chức có cái nhìn đa chiều về cả an ninh lẫn sức khỏe của hạ tầng mạng.

1.3.6. Các phương thức thu thập log

Quy trình thu thập nhật ký trong hệ thống Splunk được thiết kế với tính linh hoạt cực cao, cho phép tiếp nhận dữ liệu từ mọi nơi của hạ tầng công nghệ thông tin thông qua nhiều phương thức khác nhau. Các phương thức thu thập log và những lưu ý quan trọng để đảm bảo tính toàn vẹn và hiệu quả của dữ liệu:

Sử dụng các bộ chuyển tiếp dữ liệu (Forwarders): Đây là phương thức chủ đạo và hiệu quả nhất trong các môi trường sản xuất quy mô lớn. Splunk cung cấp hai loại Forwarder chính để đáp ứng các nhu cầu khác nhau. Universal Forwarder là một agent siêu nhẹ, tiêu tốn rất ít tài nguyên hệ thống, được cài đặt trực tiếp trên các máy chủ nguồn (Windows, Linux) để thu thập và gửi log nguyên bản về Indexer. Ngược lại, Heavy Forwarder là một phiên bản đầy đủ của Splunk Enterprise nhưng được cấu hình để chuyển tiếp dữ liệu; HF có khả năng phân tách, lọc bỏ các sự kiện không cần thiết và thậm chí là chạy các script xử lý dữ liệu phức tạp trước khi gửi đi. Điểm đặc biệt cần lưu ý là đối với các thiết bị không thể cài agent như Firewall hoặc thiết bị mạng, HF đóng vai trò bắt buộc như một Syslog listener để tiếp nhận dữ liệu đẩy về.

Phương thức giám sát trực tiếp (Monitor): Cơ chế Monitor cho phép Splunk theo dõi các nguồn dữ liệu động ngay trên các thực thể Splunk Enterprise hoặc thông qua Forwarder. Phương thức này thường áp dụng cho việc giám sát các tệp tin log cục bộ, các thư mục chứa dữ liệu thay đổi liên tục, hoặc các cổng mạng (TCP/UDP) đang lắng nghe dữ liệu từ thiết bị ngoại vi. Ngoài ra, cơ chế này còn hỗ trợ các Modular Inputs cho các nguồn dữ liệu bên thứ ba và các script lấy dữ liệu tùy chỉnh chạy định kỳ. Quản trị viên có thể sử dụng các bộ lọc Allowlist hoặc Denylist dựa trên biểu thức chính quy giúp dễ dàng tìm kiếm để chỉ định chính xác tệp tin nào trong một thư mục được phép nạp vào hệ thống, giúp tối ưu hóa dung lượng bản quyền.

Cơ chế Push và Pull: Sự khác biệt giữa hai cơ chế này nằm ở thực thể chủ động khởi tạo kết nối. Cơ chế Push log vận hành theo nguyên lý nguồn phát sinh dữ liệu chủ động gửi các bản tin log trực tiếp về phía Splunk Heavy Forwarder luôn túc trực trên các cổng dịch vụ được cấu hình sẵn để tiếp nhận luồng thông tin đổ về. Ngược lại, cơ chế Pull đòi hỏi Splunk phải chủ động thiết lập kết nối và gửi yêu cầu truy vấn tới nguồn dữ liệu để lấy thông tin về theo một lịch trình định kỳ. Phương thức này thường áp dụng cho các nguồn tin không có tính năng tự động đẩy log hoặc yêu cầu các giao thức kết nối, xác thực phức tạp.

Bảng 1.4. So sánh phương thức Push và Pull

Tiêu chí	Push	Pull
Cách truyền	Truyền khi có dữ liệu mới	Truyền theo định kỳ
Kiểu dữ liệu	Thường áp dụng cho Firewall	Ưu tiên dữ liệu đặc thù (Office 365, cloud)
Phương thức lấy dữ liệu	Luôn mở 1 cổng để dữ liệu có thể tự động đẩy khi có dữ liệu mới về (cấu hình lại đích đến, giao thức sử dụng, phân loại mức độ nguy hiểm và nguồn từ đâu và phải cho đường truyền này thông suốt qua các thiết bị khác như switch hay router)	Sẽ kéo dữ liệu về dựa trên thời gian đã định sẵn, sử dụng API key/Token
Độ tin cậy	Có thể mất dữ liệu nếu đường truyền gặp sự cố và không có cơ chế lưu đệm	Sẽ kéo dữ liệu về dựa trên thời gian kéo gần nhất
Tốc độ	Nhanh	Chậm

1.4. Tìm hiểu về ngôn ngữ truy vấn Sigma Rule

1.4.1. Giới thiệu và vai trò của Sigma

Nếu YARA là tiêu chuẩn để mô tả chữ ký mã độc, Snort là tiêu chuẩn cho luồng mạng, thì Sigma là định dạng chữ ký chung (Generic Signature Format) dành riêng cho log.

- **Giải quyết bài toán "Vendor Lock-in":** Sigma độc lập với mọi nền tảng SIEM. Người viết rule không cần quan tâm hệ thống đang dùng Splunk, ElasticSearch hay QRadar. Một file Sigma có thể được bộ biên dịch (Sigma Compiler - sigmac) dịch tự động ra ngôn ngữ SPL của Splunk.
- **Chia sẻ tri thức:** Giúp cộng đồng an toàn thông tin dễ dàng chia sẻ các tập luật phát hiện mối đe dọa mới nhất dưới định dạng chung.

1.4.2. Cú pháp và cấu trúc rule Sigma

Một file Sigma được viết bằng ngôn ngữ YAML, cực kỳ thân thiện với con người. Cấu trúc bắt buộc bao gồm các khối:

- **title & id:** Tên của rule và mã định danh duy nhất (UUID).
- **logsource:** Xác định nguồn log cần phân tích (ví dụ: category: process_creation, product: windows).
- **detection:** Phần cốt lõi chứa các logic tìm kiếm (selection) và điều kiện lọc (condition).

Ví dụ minh họa phát hiện AnyDesk:

```
logsource:
  product: windows
  category: process_creation
detection:
  selection:
    Image|endswith: '\AnyDesk.exe'
    CommandLine|contains: '--set-password'
  condition: selection
level: high
```

1.5. Ngôn ngữ SPL

1.5.1. Tổng quan về SPL

a) Vai trò và mục đích

Search Processing Language hay SPL là ngôn ngữ truy vấn và xử lý dữ liệu được sử dụng trong hệ thống Splunk. Đây là thành phần cốt lõi trong quá trình khai thác dữ liệu vì hầu hết các thao tác phân tích trong Splunk đều được thực hiện thông qua các câu lệnh tìm kiếm (Search String). Thông qua các câu lệnh SPL, người dùng có thể thực hiện nhiều thao tác như tìm kiếm sự kiện, lọc dữ liệu theo điều kiện, trích xuất các trường thông tin, tính toán thống kê, sắp xếp kết quả và tạo các biểu đồ trực quan.

b) Cấu trúc Search Pipeline

Mô hình pipeline (Đường ống) là một trong những cơ chế ưu việt của hệ thống SPL. Cơ chế này cho phép kết quả của một lệnh trước được chuyển tiếp làm đầu vào cho lệnh tiếp theo thông qua ký tự dấu gạch đứng “|”. Trong các câu lệnh SPL, quá trình xử lý bắt đầu từ việc truy xuất các sự kiện phù hợp từ dữ liệu lưu trữ trên đĩa. Sau đó dữ liệu sẽ lần lượt đi qua các bước lọc, biến đổi và xử lý theo

từng lệnh trong pipeline, trước khi được tổng hợp và hiển thị dưới dạng kết quả cuối cùng.

Để hỗ trợ người dùng khi viết các câu lệnh dài, Splunk cung cấp tính năng cho phép tách từng đoạn sau dấu “|” xuống dòng riêng biệt bằng phím tắt, giúp câu lệnh trở nên dễ đọc và dễ quản lý hơn.

c) Thành phần cơ bản

Một câu lệnh SPL hoàn chỉnh thường bao gồm năm thành phần chính, mỗi thành phần đảm nhiệm một vai trò riêng trong quá trình xử lý dữ liệu:

- **Search term:** Là thành phần đầu tiên, dùng để xác định dữ liệu cần truy vấn bao gồm từ khóa, cụm từ. Ví dụ như `index=os_win`
- **Commands (câu lệnh):** Là các thao tác được áp dụng lên kết quả tìm kiếm, ví dụ như `table` để hiển thị dạng bảng, `sort` để sắp xếp dữ liệu hoặc `fields` để lựa chọn trường dữ liệu cần hiển thị.
- **Functions:** Là các hàm được sử dụng để thực hiện các phép tính hoặc đánh giá dữ liệu, chẳng hạn như `sum` để tính tổng hoặc `avg` để tính giá trị trung bình.
- **Arguments (đối số):** Là các giá trị đầu vào cụ thể cho hàm hoặc lệnh, thường là tên các trường dữ liệu.
- **Clauses:** Là các mệnh đề dùng để nhóm dữ liệu hoặc thay đổi cách hiển thị kết quả, trong đó `as` được sử dụng để đổi tên trường và `by` được dùng để gom nhóm dữ liệu theo tiêu chí nhất định.

1.5.2. Tương quan sự kiện với SPL

Trong hệ thống Splunk, dữ liệu log được thu thập từ nhiều nguồn khác nhau và được lưu trữ dưới dạng các sự kiện (events). Mỗi sự kiện thường chỉ phản ánh một hành động đơn lẻ như đăng nhập, thực thi tiến trình hoặc truy cập mạng. Tuy nhiên, trong thực tế phân tích an ninh và vận hành hệ thống, một sự kiện đơn lẻ thường không đủ để đưa ra kết luận chính xác. Do đó kỹ thuật tương quan sự kiện (event correlation) trong SPL được sử dụng nhằm kết nối và phân tích mối quan hệ giữa nhiều sự kiện khác nhau để hình thành một bức tranh tổng thể.

Tương quan sự kiện là quá trình liên kết và xâu chuỗi các log đơn lẻ dựa trên những đặc điểm chung như địa chỉ IP, tên người dùng, máy chủ hoặc thời gian xảy ra sự kiện. Thay vì phân tích từng dòng log riêng biệt, kỹ thuật này giúp người quản trị có thể quan sát toàn bộ chuỗi hành vi của attacker dưới dạng một bức tranh tổng thể.

Trong ngôn ngữ SPL của Splunk Enterprise, tương quan sự kiện không phải là một lệnh riêng biệt mà là cách liên kết nhiều sự kiện với nhau để xác định một hành vi cụ thể. Ví dụ, Splunk có thể tương quan sự kiện tạo tiến trình để phát

hiện trường hợp phần mềm điều khiển từ xa anydesk.exe được kích hoạt ngầm và tự động thiết lập mật khẩu cố định thông qua tham số --set-password:

```
index=os_win source="WinEventLog:Microsoft-  
Windows-Sysmon/Operational" EventCode=1  
CommandLine="*anydesk*" CommandLine="*--set-  
password*"  
| table _time, host, User, Image, CommandLine
```

Hành vi này được xem là bất thường vì thông thường người dùng chỉ sử dụng AnyDesk qua giao diện đồ họa với mật khẩu ngẫu nhiên. Do đó, khi xuất hiện chuỗi tham số thực thi ngầm kèm theo lệnh gán mật khẩu cố định, hệ thống có thể nghi ngờ đây là dấu hiệu của việc hacker đang cố tình thiết lập một backdoor nhằm duy trì quyền truy cập bền vững và kiểm soát máy tính nạn nhân từ xa một cách trái phép.

Tương quan sự kiện đóng vai trò đặc biệt quan trọng trong việc phát hiện các hành vi bất thường bằng cách kết nối những sự kiện tưởng chừng không liên quan thành một dấu hiệu tấn công rõ ràng. Mặc dù mỗi dấu hiệu riêng lẻ chưa đủ để kết luận tấn công, việc kết hợp mối quan hệ giữa tiến trình cha (ParentImage) và nội dung dòng lệnh (CommandLine) giúp hệ thống nhận diện hành vi bất thường có khả năng liên quan đến macro độc hại hoặc thực thi mã độc. Ngoài ra, kỹ thuật này còn hỗ trợ hiệu quả cho quá trình điều tra số thông qua việc xác định nguyên nhân và cho biết các mốc thời gian của cuộc tấn công. Tương quan sự kiện còn giúp giảm số lượng các báo động giả bằng cách yêu cầu nhiều điều kiện log phải xảy ra đồng thời hoặc theo một trình tự nhất định trước khi sinh cảnh báo. Sáu câu lệnh thường dùng nhất là:

- **stats:** Gom nhóm và tính toán tổng hợp (chỉ giữ lại kết quả tổng kết).
- **eventstats:** Tính toán tổng hợp, giữ nguyên từng dòng log ban đầu để so sánh.
- **streamstats:** Tính toán dựa trên các sự kiện đang chảy qua theo thời gian (Rất mạnh để tìm hành vi lặp đi lặp lại liên tục).
- **transaction:** Gộp các log có chung ID/IP thành một “giao dịch” duy nhất từ lúc bắt đầu đến khi kết thúc.
- **join:** Gộp dữ liệu từ 2 bảng/kết quả tìm kiếm khác nhau lại làm một dựa trên trường chung.
- **lookup:** Đối chiếu log với một bảng dữ liệu bên ngoài (Ví dụ: Đối chiếu IP trong log với danh sách IP Blacklist).

1.5.3. Lookup và Enrichment

Trong Splunk Enterprise, Lookup là một dạng knowledge object được sử dụng để làm giàu dữ liệu (data enrichment) bằng cách bổ sung thêm thông tin từ

bên ngoài vào kết quả tìm kiếm. Cơ chế này giúp mở rộng ý nghĩa của dữ liệu log mà không làm thay đổi dữ liệu gốc đã được index trong hệ thống.

Lookup hoạt động giống như một bảng tra cứu giúp chuyển các giá trị kỹ thuật thành thông tin dễ hiểu hơn. Ví dụ, mã lỗi HTTP 503 có thể được ánh xạ thành “Service Unavailable”, hoặc một mã nhân viên nội bộ có thể được chuyển thành tên người dùng tương ứng. Ngoài ra, Lookup còn cho phép bổ sung các thông tin không tồn tại trong log gốc như phòng ban, vị trí địa lý hoặc mức độ ưu tiên của thiết bị.

Một ưu điểm quan trọng khác của Lookup là giúp tách biệt dữ liệu log và dữ liệu tham chiếu. Khi thông tin trong bảng tra cứu thay đổi, quản trị viên chỉ cần cập nhật file lookup mà không cần chỉnh sửa dữ liệu đã lưu trữ trong index. Điều này giúp hệ thống linh hoạt và dễ bảo trì hơn, đặc biệt trong các môi trường có dữ liệu thay đổi thường xuyên. Splunk hỗ trợ nhiều loại Lookup khác nhau nhằm đáp ứng các nhu cầu phân tích đa dạng:

- **File-based Lookup:** Sử dụng các tệp như .csv hoặc .gz để lưu trữ dữ liệu tra cứu bên trong máy. Đây là loại phổ biến nhất.
- **Key-Value Store Lookup:** Phù hợp với dữ liệu có quy mô lớn và hỗ trợ truy cập đồng thời thông qua cơ chế key-value.
- **External Lookup:** Cho phép Splunk gọi các script bên ngoài, chẳng hạn Python, để thực hiện các truy vấn động như DNS lookup hoặc kiểm tra threat intelligence.
- **Geospatial Lookup:** Hỗ trợ dữ liệu không gian thông qua các định dạng như .kml hoặc .kmz, phục vụ hiển thị dữ liệu trên bản đồ.

Quá trình triển khai Lookup thường gồm các bước: tải dữ liệu tra cứu lên hệ thống, định nghĩa cấu trúc lookup và thiết lập cơ chế ánh xạ dữ liệu trong SPL. Trong ngôn ngữ SPL, lệnh `lookup` được sử dụng để đối chiếu giữa trường dữ liệu đầu vào và bảng tra cứu nhằm bổ sung thêm thông tin cho sự kiện. Tùy chọn `OUTPUTNEW` thường được sử dụng để tránh ghi đè lên các field đã tồn tại. Ngoài ra, lệnh `inputlookup` cho phép truy xuất trực tiếp nội dung của bảng lookup mà không cần tìm kiếm trên dữ liệu index.

Splunk cũng hỗ trợ cơ chế time-based lookup, cho phép đối chiếu dữ liệu dựa trên thời điểm xảy ra sự kiện. Cơ chế này hữu ích trong các trường hợp dữ liệu tham chiếu thay đổi theo thời gian, chẳng hạn như giá sản phẩm, trạng thái hệ thống hoặc thông tin người dùng ở từng giai đoạn khác nhau. Ta có thể xem Lookup như một lớp trung gian giúp kết nối dữ liệu log thô với thông tin nghiệp vụ, từ đó nâng cao khả năng phân tích, correlation và xây dựng báo cáo trong hệ thống giám sát và quản trị dữ liệu.

1.6. Chuẩn hoá dữ liệu trong Splunk

1.6.1. Tổng quan về chuẩn hoá dữ liệu logs

Trong một hệ thống mạng doanh nghiệp, dữ liệu log được thu thập từ hàng trăm thiết bị khác nhau như tường lửa Cisco, máy chủ Linux, hay phần mềm diệt virus... Mỗi nhà sản xuất lại có cách đặt tên trường thông tin khác nhau; ví dụ, địa chỉ IP nguồn có thể được ghi là `client_ip`, `src`, hoặc `ip_addr`. Nếu không có chuẩn hóa, người quản trị sẽ phải viết hàng chục câu lệnh khác nhau cho cùng một mục tiêu tìm kiếm, gây lãng phí thời gian và dễ sai sót.

Chuẩn hóa dữ liệu chính là phương pháp luận để đưa tất cả các định dạng khác biệt này về một ngôn ngữ chung duy nhất. Quá trình này diễn ra chủ yếu tại thời điểm tìm kiếm khi hệ thống khởi chạy các yêu cầu truy vấn. Điều này nghĩa là Splunk không thay đổi dữ liệu gốc ban đầu nhằm đảm bảo tính toàn vẹn. Thay vào đó, hệ thống sử dụng Knowledge Objects như Field Aliases trường để gán các tên trường khác nhau vào một tên chuẩn, và Tags để phân loại các sự kiện có đặc điểm chung, từ đó cho phép so sánh tương quan dữ liệu từ nhiều nguồn một cách nhanh chóng và chính xác.

1.6.2. Ứng dụng Data Model trong Splunk

Data Model là một bản ánh xạ có cấu trúc phân cấp, được thực hiện tại thời gian tìm kiếm, về kiến trúc ngữ nghĩa của một hoặc nhiều tập dữ liệu. Nó mã hóa các kiến thức miền cần thiết để xây dựng một loạt các tìm kiếm chuyên biệt cho các tập dữ liệu đó. Các tìm kiếm chuyên biệt này được phần mềm Splunk sử dụng để tạo ra các báo cáo phục vụ cho người dùng Pivot.

Ứng dụng Data Model trong Splunk đóng vai trò là một Knowledge objects quan trọng, được cấu trúc theo dạng phân cấp để biến đổi dữ liệu máy móc thô thành các trường có ý nghĩa quản trị. Về bản chất, đây là tập hợp các tập dữ liệu giúp tạo ra các tìm kiếm tự động và là nền tảng để vận hành công cụ Pivot. Thay vì buộc người dùng phải thành thạo ngôn ngữ SPL phức tạp, mô hình này cho phép người dùng khai thác dữ liệu dựa trên các khái niệm nghiệp vụ gần gũi thông qua giao diện kéo thả trực quan để thiết kế Reports và Dashboards.

Cấu trúc cốt lõi của một Data Model bao gồm ba loại tập dữ liệu chính là Event, Search và Transaction, trong đó các tập dữ liệu con thường kế thừa thuộc tính và constraints từ tập dữ liệu cha. Tập dữ liệu Event chứa các bản ghi thô được lọc theo điều kiện xác định, trong khi tập dữ liệu Search được định nghĩa bằng các câu lệnh tìm kiếm có sử dụng các lệnh biến đổi, và tập dữ liệu Transaction dùng để nhóm các sự kiện liên quan lại với nhau dựa trên các trường chung hoặc khoảng thời gian. Việc tổ chức theo phân cấp này giúp quản trị viên dễ dàng quản lý và tái sử dụng các định nghĩa dữ liệu một cách khoa học trên quy mô lớn.

Một trong những tính năng mạnh mẽ nhất của Data Model chính là cơ chế tăng tốc nhằm xử lý khối lượng dữ liệu khổng lồ với tốc độ vượt trội. Splunk cung cấp hai hình thức tăng tốc là Ad hoc và Persistent, phục vụ cho các nhu cầu sử dụng khác nhau của doanh nghiệp. Tăng tốc Ad-hoc được tạo ra tạm thời trên Search Head ngay khi người dùng mở giao diện Pivot và chỉ tồn tại trong suốt phiên làm việc đó, điều này có thể gây tăng tải cho máy chủ tìm kiếm nếu có quá nhiều người dùng truy cập đồng thời. Ngược lại, tăng tốc bền vững là phương thức hiệu quả nhất khi các bản tóm tắt dữ liệu dưới dạng tệp .tsidx được lưu trữ lâu dài ngay trên các Indexer, nằm song song với các bucket dữ liệu gốc. Hệ thống sẽ tự động cập nhật các tệp chỉ mục này sau mỗi 5 phút và loại bỏ dữ liệu cũ sau 30 phút (tùy thuộc vào cấu hình), giúp các báo cáo chạy nhanh hơn đáng kể vì hệ thống chỉ cần quét qua các bản tóm tắt nhỏ gọn thay vì toàn bộ dữ liệu thô ban đầu. Để kích hoạt tính năng này, người dùng bắt buộc phải có quyền admin hoặc quyền `accelerate_datamodel` đặc biệt.

Trong quá trình vận hành và kiểm tra, người dùng có thể sử dụng các câu lệnh chuyên dụng như `datamodel` để hiển thị cấu trúc mô hình dưới định dạng JSON hoặc câu lệnh `tstats` để truy vấn cực nhanh vào các mô hình đã được tăng tốc. Lệnh `tstats` được coi là công cụ siêu tốc vì nó chỉ tập trung quét các tệp `tsidx`, mang lại hiệu suất cao hơn nhiều lần so với lệnh `stats` thông thường khi làm việc với hàng tỷ bản ghi. Ngoài ra, lệnh `from` cũng là một cách tiếp cận đơn giản để bắt đầu tìm kiếm trực tiếp từ một tập dữ liệu cụ thể đã được định nghĩa sẵn trong mô hình.

Tuy nhiên, có một số điểm đặc biệt quan trọng cần lưu ý để tránh các lỗi vận hành trong thực tế. Một khi Data Model đã được bật tính năng tăng tốc bền vững thì sẽ không thể chỉnh sửa cấu trúc các trường hay tập dữ liệu của nó trừ khi tắt tính năng tăng tốc đi. Bên cạnh đó, các tập dữ liệu dạng Transaction hoàn toàn không thể sử dụng cơ chế tăng tốc bền vững, chỉ có các tập Root Event và Root Search (sử dụng streaming commands) mới được hưởng lợi từ cơ chế này. Cuối cùng, các mô hình dữ liệu đi kèm với Add-on CIM thường mặc định tắt tính năng tăng tốc để tiết kiệm tài nguyên, do đó quản trị viên cần chủ động bật cho các mục tiêu giám sát an ninh quan trọng.

1.6.3. Chuẩn hoá dữ liệu với Common Information Model

Common Information Model, là mô hình chuẩn hóa dữ liệu được Splunk phát triển nhằm thống nhất cách biểu diễn dữ liệu logs từ nhiều nguồn khác nhau trong hệ thống. Trong môi trường thực tế, các thiết bị và ứng dụng thường sử dụng tên trường dữ liệu khác nhau dù mang cùng ý nghĩa. Việc chuẩn hóa dữ liệu theo CIM giúp Splunk tăng khả năng tương quan dữ liệu giữa nhiều nguồn logs khác nhau mà không phụ thuộc vào định dạng gốc của từng thiết bị. Nhờ sử dụng chung một cấu trúc dữ liệu, quản trị viên có thể xây dựng các SPL queries,

correlation rules hoặc dashboards dùng chung cho nhiều loại logs mà không cần viết lại truy vấn cho từng nguồn dữ liệu riêng biệt.

CIM còn hỗ trợ nhiều ứng dụng Splunk hoạt động đồng thời trên cùng một hệ thống thông qua việc sử dụng chung các data models và knowledge object. Điều này giúp tối ưu hóa quá trình triển khai và mở rộng hệ thống phân tích dữ liệu.

Nếu Data Model là cấu trúc, thì CIM chính là bộ quy tắc tiêu chuẩn toàn cầu của Splunk. CIM bao gồm hơn 26 mô hình dữ liệu đã được định nghĩa sẵn cho các lĩnh vực phổ biến như lưu lượng mạng, xác thực, hay mã độc. Việc tuân thủ CIM đảm bảo rằng dù doanh nghiệp có thay đổi thiết bị phần cứng của nhà cung cấp này sang nhà cung cấp khác, các báo cáo và bảng điều khiển an ninh vẫn hoạt động bình thường vì dữ liệu đã được ánh xạ về cùng một chuẩn.

Quy trình áp dụng CIM được thực hiện qua các bước logic để khớp nội dung thô với mô hình chuẩn:

- **Model:** Đầu tiên, quản trị viên kiểm tra dữ liệu thô để chọn mô hình CIM tương ứng (ví dụ: log đăng nhập sẽ khớp với mô hình Authentication).
- **Event Types và Tags:** Đánh dấu các sự kiện giúp Splunk nhận diện chúng thuộc về một nhóm chức năng cụ thể trong CIM.
- **Field Alias:** Các trường dữ liệu gốc (như clientip) tới tên trường yêu cầu của CIM (như src).
- **Làm giàu dữ liệu:** Nếu dữ liệu thô thiếu thông tin, ta sử dụng Lookups hoặc Field Extractions để bổ sung các trường cần thiết nhằm đáp ứng đầy đủ yêu cầu của mô hình.
- **Kiểm chứng:** Cuối cùng, sử dụng lệnh “|” datamodel hoặc công cụ Pivot để xác nhận dữ liệu đã hiển thị chính xác trong cấu trúc chuẩn hóa.

Các mô hình dữ liệu CIM mặc định thường tắt tính năng tăng tốc để tránh tốn tài nguyên hệ thống, vì vậy quản trị viên cần chủ động bật tính năng này cho các mô hình quan trọng. Ngoài ra, việc sử dụng các Add-on kỹ thuật từ Splunkbase là cách tốt nhất để chuẩn hóa dữ liệu, vì các ứng dụng này thường đã được nhà sản xuất cấu hình sẵn theo đúng chuẩn CIM, giúp giảm bớt gánh nặng cấu hình thủ công cho đội ngũ vận hành.

1.6.4. Chuẩn hóa dữ liệu với TA trên forwarder

Technology Add-ons không chỉ là các trình kết nối đơn thuần, mà là các domain-specific knowledge objects do Splunk hoặc các đối tác phát triển. Một TA (ví dụ: *Splunk Add-on for Windows*, *Splunk Add-on for Cisco ASA*) chứa đựng các tệp cấu hình cốt lõi (`inputs.conf`, `props.conf`, `transforms.conf`) được thiết kế riêng để đọc định dạng log thô của từng loại thiết bị.

Khi được triển khai trên tầng Forwarder, TA đóng vai trò như một màng lọc thông minh, thực hiện các tác vụ phân tích cú pháp ngay tại điểm thu thập trước khi luồng dữ liệu được mã hóa và chuyển qua mạng.

1.7. Ứng dụng AI trong Splunk

1.7.1. Tổng quan về mô hình ngôn ngữ lớn

a) Khái niệm mô hình ngôn ngữ lớn:

Mô hình ngôn ngữ lớn (LLM) là một dạng mô hình học sâu được huấn luyện trên tập dữ liệu văn bản có quy mô rất lớn nhằm học được các đặc trưng của ngôn ngữ tự nhiên. Thông qua quá trình huấn luyện, mô hình có khả năng nhận biết mối quan hệ giữa các từ, cụm từ và ngữ cảnh xuất hiện trong văn bản để tạo ra phản hồi phù hợp với yêu cầu đầu vào.

Khác với các hệ thống xử lý ngôn ngữ truyền thống thường được xây dựng cho một nhiệm vụ cụ thể, LLM có khả năng thực hiện nhiều nhiệm vụ khác nhau thông qua cùng một cơ chế tương tác bằng ngôn ngữ tự nhiên. Người dùng không cần thay đổi cấu trúc mô hình mà chỉ cần thay đổi nội dung yêu cầu để thực hiện các tác vụ như trả lời câu hỏi, tóm tắt văn bản, phân loại nội dung hoặc sinh mã nguồn.

Sự phát triển của LLM đã tạo ra bước tiến quan trọng trong lĩnh vực xử lý ngôn ngữ tự nhiên. Một số mô hình tiêu biểu hiện nay gồm GPT, Gemini, Claude... Các mô hình này được ứng dụng rộng rãi trong nhiều lĩnh vực khác nhau như giáo dục, tài chính, y tế và an toàn thông tin.

b) Cơ chế hoạt động của LLM

Mô hình ngôn ngữ lớn (LLM) hoạt động dựa trên kiến trúc Transformer và được huấn luyện trên tập dữ liệu văn bản có quy mô rất lớn nhằm học các quy luật ngôn ngữ, mối quan hệ ngữ nghĩa và ngữ cảnh giữa các từ trong văn bản. Về bản chất, LLM không lưu trữ tri thức dưới dạng cơ sở dữ liệu truyền thống mà học thông qua việc điều chỉnh hàng tỷ tham số trong quá trình huấn luyện. Nhiệm vụ cốt lõi của mô hình là dự đoán token tiếp theo dựa trên chuỗi token đã xuất hiện trước đó. Thông qua việc lặp đi lặp lại quá trình dự đoán này trên lượng dữ liệu khổng lồ, mô hình dần hình thành khả năng hiểu ngữ cảnh, tổng hợp thông tin, suy luận và sinh văn bản tự nhiên.

Khi người dùng cung cấp một câu hỏi hoặc yêu cầu dưới dạng ngôn ngữ tự nhiên, dữ liệu đầu vào trước tiên được đưa qua giai đoạn Tokenization để phân tách thành các đơn vị nhỏ gọi là token. Tùy thuộc vào phương pháp mã hóa được sử dụng, một token có thể tương ứng với một từ hoàn chỉnh, một phần của từ

hoặc một ký tự riêng lẻ. Việc phân tách này giúp mô hình chuyển đổi dữ liệu ngôn ngữ tự nhiên thành định dạng có thể xử lý được bằng các phép tính toán học. Sau đó, các token được ánh xạ thành các vector số nhiều chiều thông qua lớp Embedding Layer. Mỗi vector embedding mang thông tin biểu diễn ngữ nghĩa của token tương ứng, đồng thời phản ánh mối quan hệ giữa token đó với các token khác trong không gian biểu diễn.

Sau khi được biểu diễn dưới dạng vector, dữ liệu được đưa vào các khối Transformer, đây là thành phần quan trọng nhất của LLM. Mỗi khối Transformer bao gồm hai cơ chế xử lý chính:

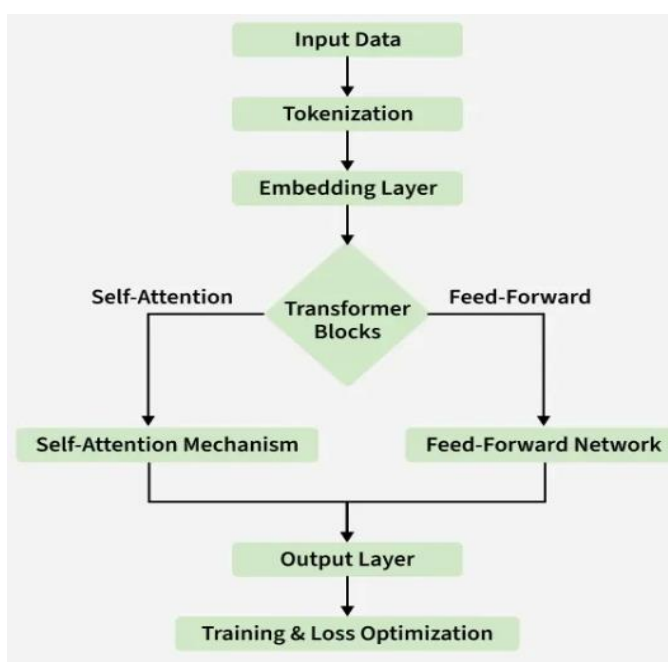
- **Self-Attention Mechanism:** cho phép mô hình xác định mức độ liên quan giữa một token với tất cả các token còn lại trong cùng chuỗi dữ liệu. Thay vì chỉ xem xét các từ lân cận như các mô hình ngôn ngữ truyền thống, Self-Attention giúp mô hình đánh giá toàn bộ ngữ cảnh của câu hoặc đoạn văn. Nhờ đó, mô hình có thể nhận biết các mối quan hệ ngữ nghĩa phức tạp, xử lý hiệu quả các câu dài và duy trì sự nhất quán của ngữ cảnh trong quá trình sinh văn bản.
- **Feed-Forward Network:** tiếp nhận kết quả từ cơ chế Self-Attention và thực hiện các phép biến đổi phi tuyến nhằm trích xuất các đặc trưng ở mức trừu tượng cao hơn. Thành phần này giúp mô hình học được các mẫu ngôn ngữ phức tạp và tăng cường khả năng biểu diễn tri thức đã học.

Các khối Transformer thường được xếp chồng thành nhiều tầng liên tiếp. Tùy theo kiến trúc và quy mô của mô hình, số lượng tầng có thể lên tới hàng chục hoặc hàng trăm lớp. Qua mỗi lớp xử lý, mô hình dần xây dựng được biểu diễn ngữ nghĩa ngày càng đầy đủ hơn về dữ liệu đầu vào, từ đó nâng cao khả năng hiểu ngữ cảnh và thực hiện suy luận.

Sau khi hoàn thành quá trình xử lý trong các khối Transformer, kết quả được chuyển tới lớp đầu ra. Tại đây, mô hình tính toán phân bố xác suất cho toàn bộ tập token có thể xuất hiện tiếp theo. Token có xác suất phù hợp nhất sẽ được lựa chọn và bổ sung vào chuỗi đầu ra. Quá trình này được lặp lại nhiều lần cho đến khi mô hình hoàn thành câu trả lời hoặc đạt đến điều kiện dừng đã được xác định trước. Cơ chế sinh từng token liên tiếp này chính là nền tảng tạo nên khả năng đối thoại và tạo sinh văn bản của các hệ thống LLM hiện nay.

Trong giai đoạn huấn luyện, kết quả dự đoán của mô hình được so sánh với dữ liệu mục tiêu thông qua hàm mất mát. Sai số thu được sẽ được lan truyền ngược qua toàn bộ mạng bằng thuật toán Backpropagation để điều chỉnh các trọng số. Quá trình tối ưu hóa này được thực hiện liên tục trên khối lượng dữ liệu rất lớn với mục tiêu giảm thiểu sai số dự đoán. Sau khi hoàn thành giai đoạn tiền huấn luyện, mô hình có thể tiếp tục được tinh chỉnh hoặc huấn luyện theo chỉ dẫn trên các tập dữ liệu chuyên biệt nhằm nâng cao khả năng thực hiện những nhiệm vụ cụ thể.

Trong các hệ thống AI hiện đại, bao gồm các ứng dụng hỗ trợ vận hành an toàn thông tin, LLM thường hoạt động ở chế độ suy luận. Khi nhận được yêu cầu từ người dùng, mô hình sử dụng các tham số đã học để phân tích ngữ cảnh, xác định ý định của câu hỏi và sinh ra phản hồi phù hợp. Nhờ khả năng xử lý ngôn ngữ tự nhiên và khai thác ngữ cảnh hiệu quả, LLM có thể được ứng dụng trong nhiều bài toán như tóm tắt dữ liệu, phân tích nhật ký hệ thống, hỗ trợ điều tra sự cố và chuyển đổi yêu cầu ngôn ngữ tự nhiên thành các truy vấn kỹ thuật.



Hình 1.7. Sơ đồ cơ chế hoạt động của LLM

Về bản chất, LLM không lưu trữ sẵn câu trả lời cho từng câu hỏi cụ thể. Thay vào đó, phản hồi được hình thành thông qua việc khai thác các mẫu ngôn ngữ đã học được trong quá trình huấn luyện kết hợp với ngữ cảnh của yêu cầu hiện tại. Chính cơ chế này cho phép mô hình thích ứng với nhiều nhiệm vụ khác nhau mà không cần thiết kế riêng một thuật toán cho từng bài toán cụ thể.

c) Khả năng và hạn chế của LLM

Khả năng của LLM xuất phát từ việc mô hình đã được huấn luyện trên lượng dữ liệu văn bản rất lớn. Nhờ đó, mô hình có khả năng nhận diện các mẫu ngôn ngữ, mối quan hệ ngữ nghĩa và tri thức xuất hiện trong dữ liệu. Trong thực tế, các khả năng của LLM có thể được chia thành ba nhóm chính:

- **Khả năng hiểu và xử lý ngôn ngữ tự nhiên:** LLM có thể xác định ý định của người dùng ngay cả khi cùng một yêu cầu được diễn đạt theo nhiều cách khác nhau. Ví dụ, các câu “liệt kê các lần đăng nhập thất bại”, “tìm các sự kiện xác thực không thành công” hoặc “hiển thị các phiên login thất bại” đều có thể được

hiểu là cùng một nhu cầu tìm kiếm thông tin. Đây là đặc điểm quan trọng giúp LLM trở thành giao diện tương tác tự nhiên giữa người dùng và hệ thống SIEM.

- **Khả năng tổng hợp và sinh nội dung:** Mô hình có thể phân tích một lượng lớn dữ liệu văn bản, xác định các thông tin quan trọng và trình bày kết quả dưới dạng dễ hiểu hơn. Trong lĩnh vực an toàn thông tin, khả năng này đặc biệt hữu ích đối với các tác vụ như tóm tắt cảnh báo bảo mật, giải thích nội dung log hoặc xây dựng báo cáo điều tra sự cố.
- **Khả năng suy luận dựa trên ngữ cảnh:** Mặc dù LLM không thực hiện suy luận logic tuyệt đối như các hệ thống chuyên biệt, mô hình vẫn có khả năng nhận diện các mối liên hệ giữa các sự kiện xuất hiện trong cùng một ngữ cảnh. Điều này cho phép LLM hỗ trợ phân tích chuỗi sự kiện, xác định mối liên hệ giữa các hành động và đề xuất hướng điều tra ban đầu

Bên cạnh những ưu điểm trên, LLM vẫn tồn tại nhiều hạn chế cần được xem xét khi triển khai trong các hệ thống hỗ trợ vận hành bảo mật.

- **Hiện tượng Hallucination:** Đây là trường hợp mô hình tạo ra thông tin không tồn tại trong dữ liệu đầu vào nhưng vẫn trình bày dưới dạng một câu trả lời hợp lý. Nguyên nhân của hiện tượng này xuất phát từ mục tiêu tối ưu của mô hình là dự đoán token có xác suất xuất hiện cao nhất chứ không phải xác minh tính đúng đắn của thông tin. Vì vậy, mô hình có thể tạo ra các địa chỉ IP, tên tiến trình hoặc kỹ thuật MITRE ATT&CK không thực sự xuất hiện trong dữ liệu.
- **Phụ thuộc vào ngữ cảnh đầu vào:** Chất lượng phản hồi của LLM chịu ảnh hưởng lớn bởi thông tin được cung cấp trong Prompt. Nếu dữ liệu đầu vào không đầy đủ hoặc không chính xác, mô hình rất khó tạo ra kết quả đáng tin cậy. Đây cũng là lý do Prompt Engineering và RAG trở thành những kỹ thuật quan trọng trong các hệ thống ứng dụng LLM.
- **Giới hạn tri thức miền:** Mặc dù được huấn luyện trên lượng dữ liệu rất lớn, LLM không tự động hiểu được các đặc điểm riêng của từng tổ chức. Ví dụ, mô hình không biết trước cấu trúc dữ liệu, quy tắc đặt tên index hoặc các quy trình vận hành nội bộ của hệ thống Splunk nếu những thông tin này không được cung cấp trong ngữ cảnh đầu vào.
- **Giới hạn bởi kích thước ngữ cảnh xử lý:** Khi lượng dữ liệu đầu vào vượt quá giới hạn của mô hình, một phần thông tin sẽ không được xem xét trong quá trình suy luận. Điều này có thể ảnh hưởng đến chất lượng phân tích khi xử lý các tập log hoặc báo cáo sự cố có kích thước lớn.

d) Vai trò của LLM trong trung tâm điều hành an ninh

Trung tâm điều hành an ninh mạng (SOC) là nơi thực hiện các hoạt động giám sát, phát hiện, phân tích và xử lý sự cố an toàn thông tin. Trong quá trình vận

hành, SOC phải xử lý khối lượng lớn dữ liệu được thu thập từ nhiều nguồn khác nhau như hệ điều hành, thiết bị mạng, tường lửa, hệ thống phát hiện xâm nhập và các giải pháp bảo mật khác.

Phần lớn dữ liệu trong môi trường SOC tồn tại dưới dạng văn bản hoặc dữ liệu bán cấu trúc. Điều này khiến việc phân tích phụ thuộc nhiều vào khả năng đọc hiểu và tổng hợp thông tin của chuyên gia vận hành. Đây là đặc điểm khiến LLM trở thành công nghệ phù hợp để hỗ trợ các hoạt động của SOC:

- LLM có khả năng xử lý ngôn ngữ tự nhiên nên có thể giúp người dùng tương tác với hệ thống SIEM thông qua các câu lệnh gần với ngôn ngữ giao tiếp thông thường thay vì phải ghi nhớ cú pháp truy vấn phức tạp. Điều này giúp giảm đáng kể rào cản kỹ thuật đối với người sử dụng.
- LLM có khả năng phân tích và diễn giải nội dung log dưới dạng ngôn ngữ dễ hiểu. Thay vì phải tự phân tích từng trường dữ liệu trong bản ghi, người vận hành có thể nhận được phần mô tả tóm tắt về ý nghĩa của sự kiện và các dấu hiệu bất thường cần chú ý.
- LLM có khả năng tổng hợp thông tin từ nhiều nguồn dữ liệu khác nhau để hỗ trợ quá trình điều tra sự cố. Mô hình có thể liên kết các sự kiện liên quan, xây dựng dòng thời gian của cuộc tấn công và hỗ trợ tạo báo cáo sơ bộ phục vụ hoạt động phân tích.

Với các đặc điểm trên, LLM đang trở thành một trong những công nghệ được quan tâm trong việc nâng cao hiệu quả vận hành SOC, đặc biệt đối với các hệ thống SIEM thế hệ mới tích hợp khả năng tương tác bằng ngôn ngữ tự nhiên.

1.7.2. Kỹ thuật hỗ trợ độ chính xác của LLM

a) Prompt Engineering

Prompt Engineering là kỹ thuật thiết kế và tối ưu nội dung đầu vào nhằm định hướng mô hình tạo ra kết quả phù hợp với yêu cầu của bài toán. Chất lượng phản hồi của LLM phụ thuộc rất lớn vào cách xây dựng Prompt. Hai người dùng có thể sử dụng cùng một mô hình nhưng thu được kết quả hoàn toàn khác nhau nếu cách diễn đạt yêu cầu khác nhau. Trong các ứng dụng thông thường, Prompt có thể chỉ là một câu hỏi ngắn. Tuy nhiên, trong môi trường an toàn thông tin, điều này thường không đủ để đảm bảo tính chính xác. Các tác vụ như sinh câu truy vấn SPL, phân tích log hoặc điều tra sự cố đều yêu cầu mô hình phải tuân thủ những quy tắc nghiệp vụ cụ thể. Vì vậy, Prompt thường được xây dựng theo nhiều lớp thông tin khác nhau.

Lớp đầu tiên là System Prompt, đóng vai trò xác định vai trò và phạm vi hoạt động của mô hình. Ví dụ, mô hình có thể được yêu cầu hoạt động như một chuyên gia SOC, chỉ được sử dụng dữ liệu xuất hiện trong log và không được tự suy diễn

thêm thông tin. Lớp thứ hai là Context, cung cấp dữ liệu nền liên quan đến hệ thống đang vận hành. Lớp cuối cùng là User Prompt chứa yêu cầu thực tế của người dùng. Việc sử dụng Prompt Engineering giúp giảm đáng kể các phản hồi không liên quan, đồng thời tăng tính nhất quán giữa các lần thực thi. Đối với các hệ thống vận hành bảo mật, đây là kỹ thuật cơ bản nhưng có ảnh hưởng trực tiếp đến chất lượng của toàn bộ hệ thống AI.

b) Retrieval-Augmented Generation

Một trong những hạn chế lớn của LLM là kiến thức của mô hình bị giới hạn bởi dữ liệu huấn luyện. Khi làm việc với các hệ thống chuyên biệt như Splunk, mô hình thường không có sẵn thông tin về cấu trúc dữ liệu nội bộ của tổ chức. Nếu chỉ dựa vào kiến thức đã học, mô hình có thể sinh ra các câu truy vấn hoặc kết luận không phù hợp với môi trường thực tế.

Để khắc phục vấn đề này, Lewis và cộng sự đã đề xuất mô hình Retrieval-Augmented Generation vào năm 2020. Ý tưởng cốt lõi của RAG là bổ sung thêm dữ liệu liên quan từ một nguồn tri thức bên ngoài trước khi gửi yêu cầu đến mô hình ngôn ngữ. Thay vì chỉ dựa vào tri thức có sẵn trong tham số của mô hình, LLM sẽ được cung cấp thêm ngữ cảnh phù hợp với từng yêu cầu cụ thể. Quy trình hoạt động của RAG gồm ba thành phần chính: nguồn tri thức, cơ chế truy xuất dữ liệu và mô hình sinh ngôn ngữ. Khi người dùng gửi yêu cầu, hệ thống sẽ tìm kiếm thông tin liên quan trong kho dữ liệu trước khi ghép chúng vào Prompt và chuyển đến LLM để sinh phản hồi.

Trong các hệ thống RAG hiện đại, nguồn tri thức thường được lưu trữ trong Vector Database kết hợp với cơ chế Embedding và Similarity Search. Tuy nhiên, việc triển khai đầy đủ các thành phần này không phải là yêu cầu bắt buộc của mọi hệ thống. Trong đề tài này, nguyên lý của RAG được áp dụng thông qua việc bổ sung thông tin về index, sourcetype và các quy tắc nghiệp vụ vào ngữ cảnh trước khi gửi yêu cầu đến mô hình. Cách tiếp cận này giúp mô hình hiểu rõ hơn môi trường Splunk đang vận hành, từ đó tăng độ chính xác của các câu truy vấn được sinh ra.

c) Hiện tượng Hallucination và biện pháp giảm thiểu

Hallucination là hiện tượng mô hình tạo ra thông tin không tồn tại trong dữ liệu đầu vào nhưng vẫn trình bày dưới dạng một câu trả lời có vẻ hợp lý. Đây là vấn đề được nhắc đến trong hầu hết các nghiên cứu và tài liệu chính thức liên quan đến LLM.

Trong lĩnh vực an toàn thông tin, Hallucination có thể dẫn đến những hậu quả nghiêm trọng. Mô hình có thể tự sinh ra địa chỉ IP, tài khoản người dùng, mã sự kiện hoặc kỹ thuật MITRE ATT&CK không xuất hiện trong log. Nếu người vận hành sử dụng trực tiếp các thông tin này để điều tra, kết quả phân tích có thể bị sai lệch.

Không tồn tại phương pháp loại bỏ hoàn toàn Hallucination. Tuy nhiên, nhiều nghiên cứu cho thấy có thể giảm đáng kể hiện tượng này bằng cách kết hợp nhiều biện pháp khác nhau. Prompt Engineering giúp giới hạn phạm vi phản hồi của mô hình. RAG giúp cung cấp thêm dữ liệu thực tế để mô hình dựa vào khi suy luận. Ngoài ra, việc yêu cầu mô hình chỉ sử dụng dữ liệu xuất hiện trong log và không tự bổ sung thông tin bên ngoài cũng là một biện pháp thường được áp dụng trong các hệ thống SOC.

d) Điều chỉnh tham số suy luận của mô hình

Ngoài dữ liệu đầu vào, kết quả sinh ra của LLM còn phụ thuộc vào các tham số suy luận được cấu hình trong quá trình thực thi. Trong số đó, Temperature là tham số được sử dụng phổ biến nhất. Temperature kiểm soát mức độ ngẫu nhiên trong quá trình sinh phản hồi. Khi Temperature cao, mô hình có xu hướng tạo ra nhiều phương án khác nhau và phản hồi mang tính sáng tạo hơn. Ngược lại, khi Temperature thấp, mô hình ưu tiên lựa chọn các phương án có xác suất cao nhất, giúp kết quả ổn định và nhất quán hơn.

Đối với các tác vụ sáng tạo nội dung, Temperature cao có thể mang lại kết quả tốt hơn. Tuy nhiên, trong môi trường SOC, yêu cầu quan trọng nhất là tính chính xác và khả năng tái lập kết quả. Vì vậy, các hệ thống hỗ trợ phân tích log hoặc sinh truy vấn thường sử dụng Temperature ở mức thấp nhằm hạn chế các phản hồi mang tính suy đoán.

Bên cạnh Temperature, Context Window cũng ảnh hưởng trực tiếp đến chất lượng phản hồi. Nếu lượng dữ liệu log vượt quá giới hạn ngưỡng của mô hình, một phần thông tin có thể bị loại bỏ trước khi xử lý. Điều này làm giảm khả năng phân tích toàn diện của hệ thống. Do đó, việc lựa chọn mô hình phù hợp và xây dựng cơ chế quản lý ngưỡng hiệu quả là yêu cầu quan trọng trong quá trình triển khai thực tế.

1.7.3. Định hướng ứng dụng LLM hỗ trợ vận hành bảo mật

Sự phát triển của các mô hình ngôn ngữ lớn đã mở ra nhiều hướng tiếp cận mới trong lĩnh vực vận hành an toàn thông tin. Khác với các công cụ phân tích truyền thống chủ yếu dựa trên quy tắc cố định hoặc các mô hình học máy chuyên biệt, LLM có khả năng xử lý và tổng hợp thông tin từ dữ liệu ngôn ngữ tự nhiên,

dữ liệu bán cấu trúc và dữ liệu phi cấu trúc. Điều này giúp mở rộng phạm vi ứng dụng của trí tuệ nhân tạo trong các hoạt động của Trung tâm điều hành an ninh mạng.

a) Threat Intelligence.

Threat Intelligence là quá trình thu thập, phân tích và chuyển đổi các thông tin liên quan đến mối đe dọa thành tri thức phục vụ hoạt động phòng thủ. Nguồn dữ liệu của Threat Intelligence thường đến từ nhiều kênh khác nhau như báo cáo nghiên cứu bảo mật, cơ sở dữ liệu lỗ hổng, nguồn tin tình báo mã nguồn mở (OSINT), diễn đàn kỹ thuật hoặc các bản tin cảnh báo của nhà cung cấp.

Một trong những khó khăn lớn của hoạt động Threat Intelligence là lượng thông tin cần xử lý rất lớn và được trình bày dưới nhiều định dạng khác nhau. LLM có khả năng hỗ trợ trích xuất các thông tin quan trọng như IOC, kỹ thuật tấn công, nhóm tác nhân đe dọa hoặc lỗ hổng liên quan từ các tài liệu có dung lượng lớn. Ngoài ra, mô hình còn có thể tóm tắt nội dung báo cáo và chuẩn hóa thông tin theo các khung tri thức phổ biến như MITRE ATT&CK hoặc STIX/TAXII.

Khả năng xử lý ngôn ngữ tự nhiên giúp giảm đáng kể thời gian phân tích tài liệu tình báo, đồng thời hỗ trợ chuyên gia bảo mật nhanh chóng nắm bắt các xu hướng tấn công mới xuất hiện trong môi trường thực tế.

b) Malware Analysis

Phân tích mã độc là hoạt động nhằm xác định cơ chế hoạt động, mục tiêu và tác động của một chương trình độc hại. Quá trình này thường yêu cầu chuyên gia bảo mật phải đọc và phân tích lượng lớn thông tin kỹ thuật như mã nguồn, nhật ký thực thi, kết quả sandbox hoặc báo cáo điều tra.

LLM có thể hỗ trợ diễn giải các kết quả phân tích kỹ thuật thành ngôn ngữ dễ hiểu hơn, giúp rút ngắn thời gian nghiên cứu của chuyên gia. Trong nhiều trường hợp, mô hình có khả năng nhận diện các hành vi đặc trưng của mã độc thông qua mô tả hành vi hoặc chuỗi lệnh thực thi và hỗ trợ ánh xạ chúng với các kỹ thuật trong khung MITRE ATT&CK.

Tuy nhiên, cần lưu ý rằng LLM không thay thế các công cụ phân tích mã độc chuyên dụng. Vai trò chính của mô hình là hỗ trợ tổng hợp và diễn giải thông tin, trong khi việc xác minh hành vi thực tế của mã độc vẫn cần được thực hiện thông qua các công cụ phân tích tĩnh và phân tích động.

c) Alert Triage

Alert Triage là quá trình phân loại, đánh giá và ưu tiên xử lý các cảnh báo an ninh được sinh ra từ các hệ thống giám sát. Đây là một trong những hoạt động tiêu tốn nhiều nguồn lực nhất của SOC do số lượng cảnh báo thường rất lớn và tỷ lệ cảnh báo giả có thể ở mức cao.

LLM có thể hỗ trợ phân tích nội dung cảnh báo, tổng hợp các dữ liệu liên quan và cung cấp đánh giá sơ bộ về mức độ nghiêm trọng của sự kiện. Thay vì yêu cầu chuyên gia đọc từng bản ghi riêng lẻ, mô hình có thể tóm tắt những thông tin quan trọng như tài sản bị ảnh hưởng, hành vi bất thường được ghi nhận hoặc các dấu hiệu liên quan đến kỹ thuật tấn công. Việc ứng dụng LLM trong Alert Triage giúp giảm khối lượng công việc thủ công và hỗ trợ đội ngũ vận hành tập trung vào các cảnh báo có nguy cơ cao hơn. Đây cũng là một trong những hướng nghiên cứu được nhiều nhà cung cấp giải pháp bảo mật hiện nay quan tâm.

d) Incident Response

Incident Response là quá trình phát hiện, phân tích, khoanh vùng, xử lý và khôi phục sau sự cố an toàn thông tin. Trong quá trình điều tra, chuyên gia bảo mật thường phải tổng hợp dữ liệu từ nhiều nguồn khác nhau như log hệ thống, cảnh báo SIEM, dữ liệu EDR hoặc các kết quả điều tra trước đó.

Nhờ khả năng xử lý lượng lớn dữ liệu văn bản, LLM có thể hỗ trợ xây dựng dòng thời gian của sự cố, tổng hợp các sự kiện liên quan và mô tả diễn biến của cuộc tấn công dưới dạng dễ hiểu hơn. Ngoài ra mô hình còn có thể hỗ trợ ánh xạ các hành vi quan sát được với các chiến thuật và kỹ thuật trong khung MITRE ATT&CK nhằm hỗ trợ chuẩn hóa quá trình điều tra. Mặc dù vậy, các quyết định xử lý cuối cùng vẫn cần được xác nhận bởi chuyên gia bảo mật. LLM nên được xem là công cụ hỗ trợ điều tra thay vì thay thế hoàn toàn vai trò của người phân tích.

e) Report Generation

Báo cáo kỹ thuật là đầu ra quan trọng của nhiều hoạt động vận hành bảo mật như điều tra sự cố, đánh giá rủi ro hoặc phân tích mối đe dọa. Việc xây dựng báo cáo thường yêu cầu tổng hợp thông tin từ nhiều nguồn dữ liệu khác nhau và trình bày lại theo cấu trúc phù hợp với từng đối tượng sử dụng.

LLM có khả năng tự động tóm tắt dữ liệu, tổ chức nội dung và xây dựng báo cáo theo các khuôn mẫu được xác định trước. Điều này giúp giảm thời gian thực hiện các công việc mang tính lặp lại, đồng thời cải thiện tính nhất quán giữa các báo cáo được tạo ra bởi nhiều chuyên gia khác nhau.

Ngoài các báo cáo kỹ thuật dành cho chuyên gia SOC, mô hình còn có thể hỗ trợ chuyển đổi các nội dung chuyên sâu thành ngôn ngữ dễ hiểu hơn phục vụ các nhà quản lý hoặc lãnh đạo doanh nghiệp.

f) Natural Language Query

Một trong những hướng ứng dụng nổi bật nhất của LLM trong các hệ thống bảo mật hiện đại là khả năng tương tác bằng ngôn ngữ tự nhiên. Thay vì yêu cầu người dùng phải ghi nhớ cú pháp truy vấn hoặc ngôn ngữ tìm kiếm chuyên biệt, mô hình cho phép diễn đạt yêu cầu dưới dạng ngôn ngữ thông thường.

Khi nhận được yêu cầu từ người dùng, LLM sẽ thực hiện phân tích ý định, nhận diện các thực thể liên quan và chuyển đổi thành câu truy vấn phù hợp với hệ thống phía sau. Cách tiếp cận này giúp giảm đáng kể rào cản kỹ thuật đối với người sử dụng, đồng thời mở rộng khả năng khai thác dữ liệu cho các nhóm không chuyên sâu về an toàn thông tin.

Sự phát triển của các trợ lý bảo mật dựa trên LLM trong những năm gần đây cho thấy Natural Language Query đang trở thành một trong những hướng ứng dụng quan trọng nhất của trí tuệ nhân tạo trong lĩnh vực vận hành bảo mật.

Từ các định hướng trên có thể thấy LLM có tiềm năng ứng dụng trong nhiều hoạt động khác nhau của SOC. Tuy nhiên, do giới hạn về thời gian và phạm vi nghiên cứu, đề tài tập trung vào các bài toán liên quan đến truy vấn dữ liệu bằng ngôn ngữ tự nhiên, hỗ trợ phân tích log và hỗ trợ điều tra sự cố an toàn thông tin.

1.8. Tổng kết chương 1

Chương 1 đã trình bày một cách hệ thống và toàn diện các nền tảng lý thuyết cốt lõi, tạo cơ sở vững chắc cho toàn bộ quá trình triển khai hệ thống.

Trước hết, chương làm rõ vai trò của hệ thống SIEM như một trung tâm chỉ huy trong chiến lược an ninh mạng, từ việc tập trung hóa dữ liệu, phát hiện mối đe dọa qua các cơ chế so sánh mẫu và phân tích hành vi, đến việc tự động hóa phản ứng sự cố. Điểm nhấn của chương là phân tích sâu về kiến trúc của Splunk Enterprise, từ các thành phần chính (Search Head, Indexer, Forwarder) và cơ chế lập chỉ mục Bucket cho đến các mô hình triển khai phân tán và cluster nhằm đáp ứng yêu cầu về hiệu suất và tính sẵn sàng cao.

Về mặt kỹ thuật phát hiện, chương đã giới thiệu bộ khung MITRE ATT&CK như một ngôn ngữ chung để chuẩn hóa các cảnh báo và đánh giá độ phủ của hệ thống, đi kèm với các nguồn dữ liệu đầu vào quan trọng như Windows Event Log, Sysmon, Linux Audit Log và Syslog từ thiết bị mạng. Điểm đặc biệt cần lưu ý là việc giới thiệu Sigma Rule - một định dạng chữ ký chung cho log, giải quyết được bài toán

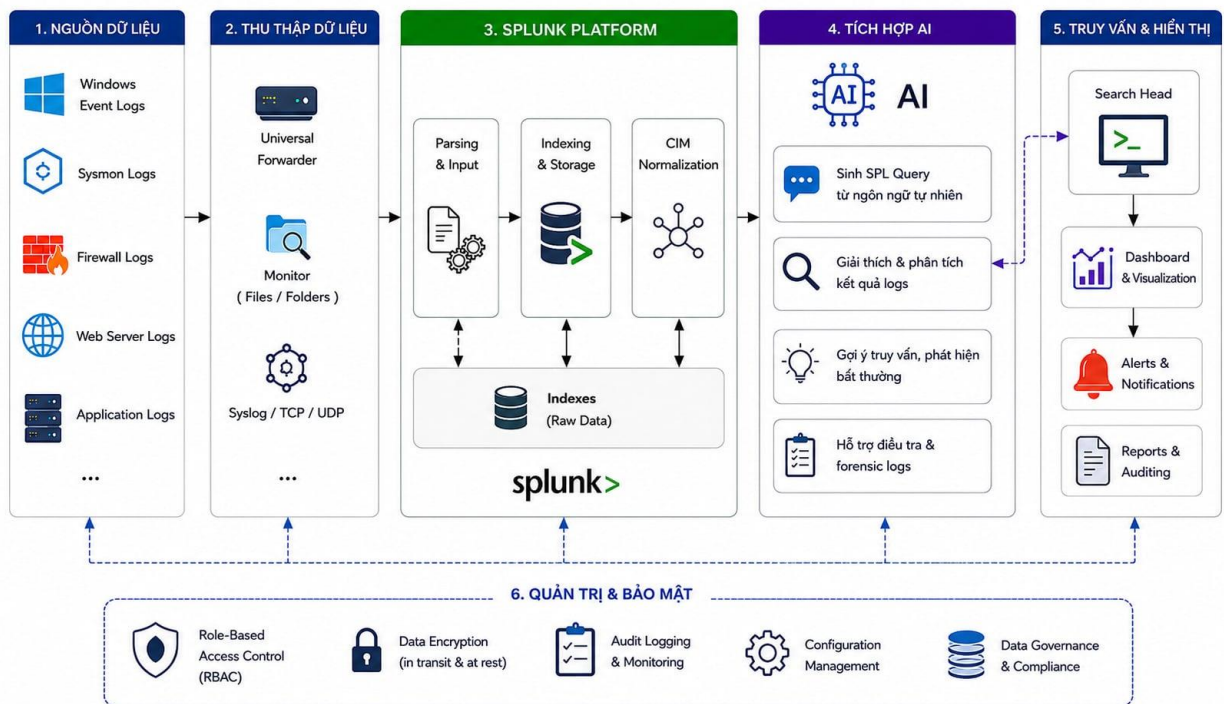
phụ thuộc vào nhà cung cấp SIEM và tạo điều kiện chia sẻ tri thức trong cộng đồng bảo mật.

Bên cạnh đó, chương cũng giới thiệu ngôn ngữ truy vấn SPL và các kỹ thuật tương quan sự kiện, làm giàu dữ liệu qua Lookup. Cuối cùng, chương đã đề cập đến hướng ứng dụng AI vào vận hành bảo mật, tập trung vào các mô hình ngôn ngữ lớn (LLM) và các kỹ thuật như Prompt Engineering, RAG, để nâng cao hiệu quả phân tích và hỗ trợ điều tra sự cố.

CHƯƠNG 2. TRIỂN KHAI TÍCH HỢP AI TRONG SPLUNK

2.1. Mô hình triển khai

Mô hình triển khai tích hợp AI trong Splunk được xây dựng nhằm hỗ trợ quá trình giám sát, phân tích và phát hiện các hành vi bất thường trong hệ thống thông tin. Việc kết hợp giữa nền tảng Splunk và các công nghệ trí tuệ nhân tạo giúp nâng cao khả năng xử lý dữ liệu logs, tự động hóa quá trình phân tích và hỗ trợ phát hiện tấn công theo thời gian thực. Trong mô hình này, Splunk đóng vai trò là nền tảng trung tâm thực hiện thu thập, lưu trữ và tìm kiếm dữ liệu, trong khi các mô hình AI đảm nhận nhiệm vụ phân tích hành vi và hỗ trợ đưa ra dự đoán hoặc cảnh báo.



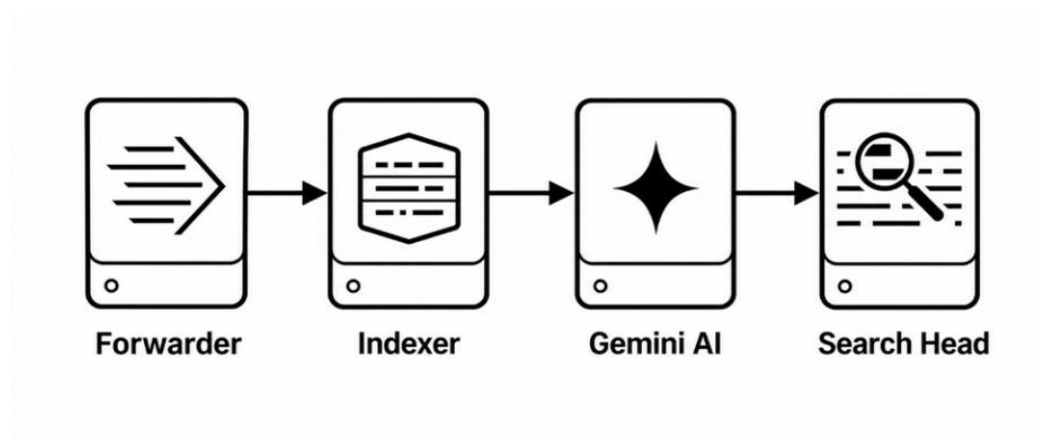
Hình 2.1. Mô hình triển khai hệ thống Splunk tích hợp AI

2.2. Thu thập và chuẩn hoá dữ liệu

2.2.1. Thu thập logs

a) Mô hình thu thập logs trong Splunk

Splunk sử dụng mô hình thu thập dữ liệu phân tán nhằm hỗ trợ việc tiếp nhận, lưu trữ và phân tích logs từ nhiều nguồn khác nhau trong hệ thống. Mô hình này bao gồm các thành phần chính như Forwarder, Indexer, Search Head và hệ thống AI giúp phân tích dữ liệu. Trong mô hình triển khai, dữ liệu logs từ các thiết bị hoặc máy chủ sẽ được gửi thông qua Forwarder đến Indexer để thực hiện quá trình lưu trữ và lập chỉ mục trước khi được truy vấn trên Search Head hoặc phân tích bởi AI.



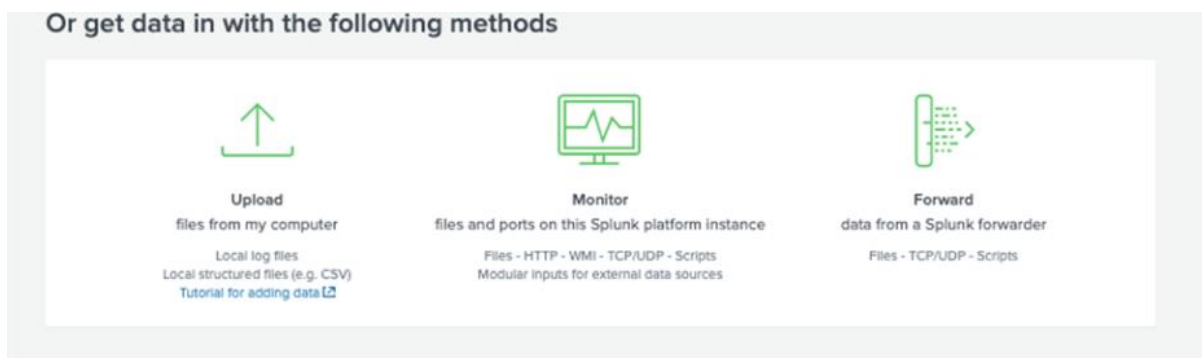
Hình 2.2. Mô hình thu thập logs trong Splunk

Forwarder là thành phần chịu trách nhiệm thu thập dữ liệu từ các máy nguồn và gửi về hệ thống Splunk. Trong thực tế, Splunk hỗ trợ hai loại Forwarder gồm Universal Forwarder và Heavy Forwarder. Universal Forwarder là một agent nhẹ, được sử dụng phổ biến để thu thập logs từ endpoint hoặc máy chủ với mức tiêu thụ tài nguyên thấp. Thành phần này hỗ trợ thu thập nhiều loại dữ liệu như Windows Event Logs, Sysmon, PowerShell logs hoặc Linux logs. Trong khi đó, Heavy Forwarder hỗ trợ thêm các chức năng xử lý dữ liệu như filtering, parsing hoặc routing trước khi chuyển tiếp dữ liệu đến Indexer. Sau khi dữ liệu được gửi đến hệ thống, Indexer sẽ thực hiện nhiệm vụ lưu trữ và lập chỉ mục dữ liệu. Thành phần này giúp chuyển đổi dữ liệu thô thành các sự kiện có thể tìm kiếm được thông qua cơ chế indexing của Splunk. Nhờ đó, hệ thống có thể truy vấn dữ liệu với hiệu năng cao ngay cả khi khối lượng logs lớn.

Cuối cùng, Search Head là thành phần cho phép người dùng thực hiện tìm kiếm, trực quan hóa và phân tích dữ liệu. Trong mô hình tích hợp AI, Search Head có thể kết hợp với các nền tảng AI như Gemini AI Assistant Assistant nhằm hỗ trợ sinh SPL queries từ ngôn ngữ tự nhiên, phân tích logs và đề xuất các correlation rules phục vụ cho hoạt động giám sát và điều tra an ninh mạng.

b) Cơ chế thu thập logs trong Splunk

Splunk hỗ trợ nhiều cơ chế thu thập dữ liệu khác nhau nhằm đáp ứng các nhu cầu triển khai và phân tích log trong thực tế. Các phương thức phổ biến bao gồm Monitor, Upload và Forwarder. Mỗi cơ chế có đặc điểm riêng và được sử dụng trong các trường hợp khác nhau.



Hình 2.3. Cơ chế thu thập logs trong Splunk

Cơ chế Monitor cho phép Splunk theo dõi trực tiếp các file hoặc thư mục chứa log trên hệ thống. Khi dữ liệu mới được ghi vào file, Splunk sẽ tự động đọc và index dữ liệu theo thời gian thực. Phương pháp này thường được sử dụng để thu thập Linux logs, application logs hoặc web server logs như Apache và Nginx. Việc monitor file giúp hệ thống có thể giám sát liên tục mà không cần thao tác thủ công từ quản trị viên.

Splunk còn hỗ trợ cơ chế Upload, cho phép người dùng tải trực tiếp các tập tin log lên giao diện hệ thống để phân tích. Phương pháp này thường được sử dụng trong môi trường thử nghiệm, forensic hoặc các bài lab khi cần kiểm tra nhanh một tập tin log độc lập. Tuy nhiên, Upload không hỗ trợ cập nhật dữ liệu theo thời gian thực và chủ yếu mang tính thủ công.

Ngoài Monitor và Upload, Forwarder là phương thức thu thập logs quan trọng nhất trong Splunk. Universal Forwarder được cài đặt trên các máy nguồn để thu thập và gửi dữ liệu về Splunk Indexer. Phương pháp này hỗ trợ thu thập tập trung từ nhiều hệ thống khác nhau như Windows Event Logs, Sysmon, PowerShell logs hoặc Linux logs. Trong các môi trường lớn, Heavy Forwarder có thể được sử dụng để thực hiện thêm các tác vụ xử lý dữ liệu như filtering hoặc routing trước khi chuyển tiếp logs đến hệ thống lưu trữ trung tâm. Dữ liệu sau khi được thu thập sẽ đóng vai trò là nguồn đầu vào cho các hệ thống AI hỗ trợ phân tích trong Splunk. Các mô hình AI có thể sử dụng dữ liệu logs để hỗ trợ phát hiện bất thường, sinh SPL queries hoặc phân tích hành vi tấn công trong môi trường SOC.

c) Quá trình thu thập logs trong Splunk

Sau khi dữ liệu được thu thập vào hệ thống, Splunk sẽ thực hiện quá trình xử lý nhằm chuyển đổi dữ liệu thô thành các sự kiện có thể tìm kiếm và phân tích. Quá trình này bao gồm các giai đoạn chính như Data Ingestion, Indexing và Search Processing.

Giai đoạn đầu tiên là Data Ingestion, trong đó dữ liệu logs được tiếp nhận từ nhiều nguồn khác nhau như hệ điều hành, ứng dụng, firewall hoặc thiết bị mạng. Trong quá trình này, Splunk sẽ xác định sourcetype để nhận diện định dạng dữ

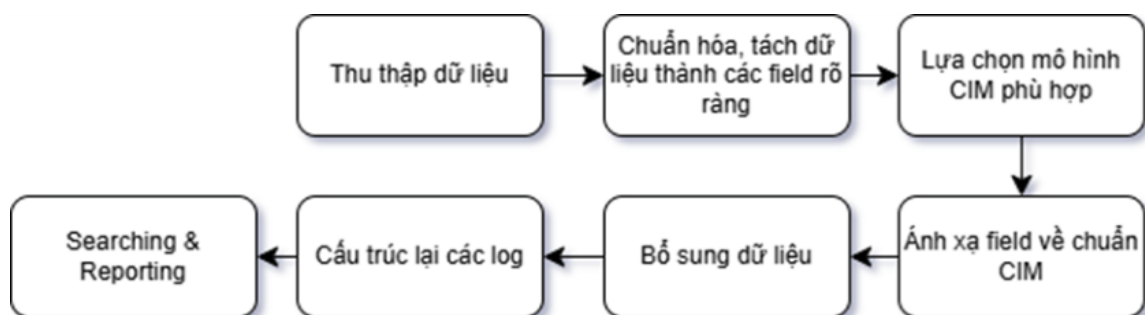
liệu và hỗ trợ việc phân tách logs thành các sự kiện riêng biệt dựa trên timestamp hoặc các quy tắc parsing phù hợp. Sau khi tiếp nhận dữ liệu, hệ thống chuyển sang giai đoạn Indexing. Tại đây, Indexer thực hiện xử lý dữ liệu thô, nén dữ liệu và lưu trữ vào các index nhằm tối ưu cho việc truy vấn. Đồng thời, Splunk cũng gán các trường dữ liệu mặc định như host, source, sourcetype, _time và _raw vào từng sự kiện.

Khi người dùng thực hiện truy vấn trên Search Head, dữ liệu sẽ được xử lý thông qua Search Pipeline. Các lệnh SPL được thực hiện tuần tự từ trái sang phải, trong đó kết quả của lệnh trước sẽ trở thành đầu vào cho lệnh kế tiếp. Trong mô hình tích hợp AI, Gemini AI Assistant có thể hỗ trợ người dùng xây dựng SPL queries từ mô tả bằng ngôn ngữ tự nhiên, giải thích logs hoặc đề xuất các truy vấn phục vụ cho quá trình threat hunting và forensic.

Ngoài khả năng hỗ trợ truy vấn, AI còn có thể tham gia phân tích hành vi bất thường dựa trên dữ liệu logs đã được index. Điều này giúp giảm thời gian xử lý thủ công và nâng cao hiệu quả phát hiện các hoạt động đáng ngờ trong hệ thống. Sau khi hoàn tất xử lý, kết quả tìm kiếm sẽ được hiển thị dưới dạng bảng biểu, biểu đồ hoặc dashboard trực quan trên Search Head, hỗ trợ quản trị viên trong quá trình giám sát và điều tra.

2.2.2. Chuẩn hoá dữ liệu theo CIM

Trong Splunk Enterprise, để một nguồn dữ liệu phù hợp với cấu trúc của Splunk, quản trị viên cần thực hiện quá trình chuẩn hóa nhằm đảm bảo dữ liệu phù hợp với cấu trúc chuẩn của Splunk CIM. Mục tiêu của quá trình này là giúp các nguồn logs khác nhau có thể sử dụng chung một cấu trúc field, từ đó hỗ trợ correlation, tìm kiếm và xây dựng detection rules hiệu quả hơn.



Hình 2.4. Quy trình chuẩn hóa dữ liệu theo tiêu chuẩn CIM

Quy trình bắt đầu từ việc thu thập dữ liệu thô từ nhiều nguồn khác nhau như hệ thống, ứng dụng, firewall hoặc endpoint. Các dữ liệu này thường ở dạng logs thô và chưa có cấu trúc thống nhất. Tiếp theo, dữ liệu được đưa vào bước chuẩn hóa và tách field, trong đó Splunk sẽ phân tích log để trích xuất các trường thông tin quan trọng như IP, user, action, status... Sau đó, hệ thống tiến hành lựa chọn

mô hình CIM phù hợp, tùy theo loại dữ liệu (ví dụ: Authentication, Network Traffic, Web, Endpoint...).

Dữ liệu tiếp tục được xử lý thông qua bước ánh xạ field về chuẩn CIM, tức là chuyển các field không đồng nhất từ nhiều nguồn về một bộ field chuẩn chung của CIM. Kế tiếp là bước bổ sung dữ liệu, trong đó dữ liệu được làm giàu thêm từ lookup hoặc các nguồn thông tin bổ trợ như asset database, user identity hoặc threat intelligence. Sau khi hoàn tất, dữ liệu được cấu trúc lại thành log chuẩn hóa theo CIM, đảm bảo tính thống nhất và sẵn sàng cho phân tích. Cuối cùng, dữ liệu đã chuẩn CIM sẽ được sử dụng cho tìm kiếm, phân tích và báo cáo, hỗ trợ correlation và xây dựng các rules phát hiện an ninh hiệu quả.

Sau khi dữ liệu được chuẩn hóa theo CIM, Splunk cung cấp nhiều công cụ và lệnh SPL nhằm hỗ trợ việc tìm kiếm và phân tích dữ liệu hiệu quả hơn. Một trong những lệnh phổ biến là Datamodel, cho phép người dùng kiểm tra cấu trúc của các mô hình dữ liệu hoặc truy vấn trực tiếp các sự kiện bên trong Data Model cụ thể. Splunk còn hỗ trợ lệnh `from`, cho phép truy xuất dữ liệu trực tiếp từ các Data Models đã được định nghĩa. Việc truy vấn thông qua Data Models giúp giảm sự phụ thuộc vào cấu trúc logs gốc và tạo tính nhất quán trong quá trình phân tích dữ liệu.

Trong các hệ thống có khối lượng dữ liệu lớn, lệnh `tstats` thường được sử dụng để tăng tốc độ tìm kiếm trên các Data Models đã được accelerated. So với các câu lệnh tìm kiếm SPL thông thường, lệnh này cho hiệu năng cao hơn đáng kể do dữ liệu đã được tối ưu và lập chỉ mục trước đó. Điều này đặc biệt hữu ích trong các hệ thống SOC khi cần thực hiện correlation hoặc phát hiện tấn công theo thời gian thực. Việc áp dụng CIM trong Splunk giúp chuẩn hóa dữ liệu trên toàn hệ thống, nâng cao khả năng tương quan giữa các nguồn logs và tối ưu hiệu quả xây dựng các cơ chế giám sát an ninh mạng.

2.3. Thiết kế Rule bằng SPL

2.3.1. Kỹ thuật phân tích ngữ nghĩa dòng lệnh

Phân tích ngữ nghĩa dòng lệnh là kỹ thuật khai thác thông tin từ trường CommandLine được ghi nhận trong nhật ký hệ thống nhằm xác định mục đích thực thi thực tế của một tiến trình. Khác với phương pháp chỉ dựa trên tên tiến trình, kỹ thuật này tập trung phân tích các tham số được truyền khi chương trình khởi chạy để nhận diện hành vi, đánh giá mức độ rủi ro và phát hiện các dấu hiệu tấn công tiềm ẩn.

Trong các cuộc tấn công hiện đại, đặc biệt là các chiến dịch sử dụng mã độc tàng hình hay các kỹ thuật nguy trang, tin tặc thường xuyên sử dụng các tham số dòng lệnh phức tạp để vượt qua các cơ chế rà quét tĩnh. Điển hình như việc sử

dùng powershell.exe với các cờ như -ExecutionPolicy Bypass, -WindowStyle Hidden hoặc mã hóa toàn bộ payload bằng Base64 (-Enc). Do đó, cần phải phân tích ý nghĩa của các tham số dòng lệnh giúp phát hiện các hành vi bất thường ngay cả khi tiến trình sử dụng là hợp pháp.

Kỹ thuật trích xuất trong SPL sử dụng lệnh rex kết hợp với biểu thức chính quy để phân tích linh hoạt các thành phần ẩn sâu bên trong trường CommandLine của nhật ký Sysmon (Event ID 1). Sau khi bóc tách, lệnh eval được sử dụng để định nghĩa lại ngữ nghĩa của dòng lệnh, giúp hệ thống phân loại mức độ rủi ro ngay lập tức.

Ví dụ lệnh SPL trích xuất và phân tích mã lệnh PowerShell độc hại:

```
index=os_win sourcetype="XmlWinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=1 Image="*powershell.exe"
| rex field=CommandLine "(?i)(?<Evasion_Flags>-w\s+hidden|-ep\s+bypass|-enc\s+|-nop\s+)"
| search Evasion_Flags="*"
| eval Risk_Level = if(match(CommandLine, "(?i)-enc"), "Critical", "High")
| eval Action_Semantics = "Phát hiện nỗ lực thực thi mã lệnh ẩn danh hoặc vượt quyền"

| table _time, host, user, ParentImage, CommandLine, Evasion_Flags, Risk_Level, Action_Semantics
```

Giải thích: Câu lệnh trên tự động rà quét toàn bộ tiến trình PowerShell, sử dụng rex để trích xuất các tham số lẫn tránh vào trường mới là Evasion_Flags. Dựa trên kết quả này, hàm eval sẽ đánh giá: nếu dòng lệnh chứa cờ -enc, mức độ rủi ro sẽ lập tức bị gán là critical đi kèm với giải nghĩa hành vi, hỗ trợ nhận diện mũi tấn công chỉ trong vài giây.

2.3.2. Kỹ thuật tương quan cây tiến trình

Tương quan cây tiến trình là kỹ thuật phân tích mối quan hệ giữa các tiến trình dựa trên cấu trúc cha - con được hình thành trong quá trình thực thi trên hệ điều hành. Mỗi tiến trình khi được tạo ra đều có một tiến trình cha chịu trách nhiệm khởi tạo, từ đó hình thành nên chuỗi thực thi phản ánh toàn bộ hoạt động của hệ thống.

Trong hoạt động điều tra số và giám sát an toàn thông tin, một sự kiện riêng lẻ thường không cung cấp đủ thông tin để xác định bản chất của một cuộc tấn công. Việc phân tích mối quan hệ giữa các tiến trình cho phép nhà phân tích tái

hiện toàn bộ chuỗi hành vi, từ điểm khởi nguồn cho đến hành động cuối cùng mà đối tượng tấn công thực hiện trên hệ thống. Đây là kỹ thuật được sử dụng rộng rãi trong các giải pháp EDR, XDR và Threat Hunting hiện đại.

Trong Splunk, thay vì sử dụng lệnh transaction tiêu tốn nhiều tài nguyên bộ nhớ cho các tập dữ liệu lớn, kỹ thuật tối ưu là sử dụng lệnh stats để gom nhóm dữ liệu theo các thuộc tính gốc, từ đó phác họa lại toàn bộ chuỗi thực thi, ví dụ như từ lúc người dùng mở một tệp Word đính kèm macro độc hại cho đến khi máy tính tự động mở cổng kết nối ra máy chủ C&C.

Ví dụ lệnh SPL tương quan phát hiện mã độc từ bộ ứng dụng Office:

```
index=os_win sourcetype="XmlWinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=1
| search (ParentImage="*\\winword.exe" OR ParentImage="*\\excel.exe")
AND (Image="*\\cmd.exe" OR Image="*\\powershell.exe")
| stats count, min(_time) as First_Execution, max(_time) as Last_Execution,
values(CommandLine) as Malicious_Payload by host, user, ParentImage,
Image
| convert ctime(First_Execution) ctime(Last_Execution)
| rename ParentImage as "Tiền trình khởi nguồn", Image as "Tiền trình phái
sinh", Malicious_Payload as "Mã lệnh thực thi"
```

Giải thích: Câu lệnh tập trung tìm kiếm các hành vi bất thường mang tính kiến trúc: phần mềm văn phòng (Word/Excel) không có chức năng mặc định gọi ra hệ vỏ lệnh (cmd/powershell). Bằng cách dùng stats để tương quan ParentImage và Image, hệ thống không chỉ phát hiện ra chuỗi tấn công Spear-phishing mà còn gom được toàn bộ mã lệnh CommandLine (Malicious_Payload) mà macro độc hại đã cố gắng thực thi trên máy nạn nhân.

2.3.3. Kỹ thuật phân tích ngưỡng và phát hiện dị thường

Kỹ thuật phân tích ngưỡng và phát hiện dị thường là phương pháp xác định các hành vi bất thường dựa trên việc so sánh dữ liệu hiện tại với một ngưỡng hoặc đường cơ sở đã được thiết lập trước. Khi giá trị quan sát vượt quá giới hạn cho phép, hệ thống sẽ đánh dấu đây là một sự kiện bất thường và tiến hành cảnh báo.

Tấn công mạng không phải lúc nào cũng sử dụng mã độc. Các chiến dịch như Brute-force, DDoS hay Data Exfiltration thường lạm dụng các giao thức hợp lệ, chỉ khác biệt ở tần suất và lưu lượng. Kỹ thuật phân tích ngưỡng thiết lập các

đường cơ sở về hoạt động bình thường, từ đó sử dụng các lệnh thống kê chuỗi thời gian để phát hiện các dị thường vượt quá ngưỡng cho phép.

Trong Splunk, kỹ thuật này thường được hiện thực bằng cách kết hợp các lệnh bin, stats, timechart và where. Dữ liệu được chia thành các khoảng thời gian cố định, sau đó tiến hành thống kê số lượng sự kiện trong từng khoảng thời gian và so sánh với ngưỡng được xác định trước. Khi điều kiện vượt ngưỡng xảy ra, hệ thống sẽ sinh cảnh báo hoặc chuyển tiếp kết quả cho các quy trình phân tích tiếp theo.

Ví dụ lệnh SPL phát hiện tấn công dò mật khẩu (Brute-force) qua RDP/SSH:

```
index=os_win sourcetype="WinEventLog:Security" EventCode=4625
| bin _time span=5m
| stats count as Failed_Attempts, dc(user) as Unique_Accounts_Targeted,
values(user) as Target_Users by _time, host, src_ip
| where Failed_Attempts >= 15
| eval Threat_Type = if(Unique_Accounts_Targeted > 3, "Password Spraying
Attack", "Targeted Brute-force")
| table _time, host, src_ip, Target_Users, Failed_Attempts, Threat_Type
| sort - Failed_Attempts
```

Giải thích: Câu lệnh chia dòng thời gian thành các cửa sổ 5 phút (span=5m) và đếm số lượng sự kiện đăng nhập thất bại (Event ID 4625). Lệnh where Failed_Attempts >= 15 đóng vai trò là ngưỡng kích hoạt cảnh báo. Hàm eval kết hợp dc() (distinct count) giúp hệ thống tự động phân tích sâu hơn: Nếu một IP dò nhiều mật khẩu trên một tài khoản, đó là Brute-force truyền thống, nhưng nếu IP đó dò một vài mật khẩu trên rất nhiều tài khoản khác nhau (Unique_Accounts_Targeted > 3), hệ thống sẽ phân loại đây là cuộc tấn công diện rộng Password Spraying, giúp đưa ra chiến lược ngăn chặn tường lửa (Firewall Blocking) tương ứng.

2.3.4. Tích hợp AI hỗ trợ truy vấn log

a) Mô tả chức năng

Chức năng hỗ trợ truy vấn log là một mô-đun chuyển đổi ngôn ngữ tự nhiên thành câu lệnh SPL, hoạt động trong kiến trúc mở rộng của hệ thống SIEM Splunk Enterprise. Chức năng này chịu trách nhiệm tiếp nhận yêu cầu phân tích bằng tiếng Việt từ người dùng, tự động tra cứu từ điển dữ liệu của hệ thống để hiểu cấu trúc index, sourcetype và trường dữ liệu, sau đó sử dụng mô hình ngôn

ngữ lớn để sinh ra câu lệnh SPL chính xác, giúp người dùng không cần am hiểu sâu về cú pháp SPL vẫn có thể khai thác dữ liệu.

Phương thức vận hành của chức năng được thực hiện thông qua một chuỗi quy trình xử lý dữ liệu khép kín, bao gồm năm giai đoạn kỹ thuật cốt lõi từ lúc tiếp nhận câu hỏi cho đến khi trả về câu lệnh SPL:

- **Tiếp nhận câu hỏi từ giao diện Form:** Người dùng nhập câu hỏi bằng tiếng Việt vào ô nhập liệu trên giao diện Form. Khi nhấn submit, câu hỏi được truyền vào token \$user_prompt\$ và gửi đến custom search command.
- **Gọi Custom Search Command:** Giao diện Form sử dụng lệnh | geminispl prompt="\$user_prompt\$" để gọi custom search command đã được đăng ký. Splunk spawn một tiến trình con chạy file gemini_spl.py và truyền câu hỏi qua tham số prompt.
- **Đọc Data Dictionary:** Python đọc file data_dictionary.json chứa cấu trúc index, sourcetype, EventCode và các trường dữ liệu của hệ thống. Dữ liệu này được nhúng vào System Prompt để mô hình AI hiểu được ngữ cảnh dữ liệu thực tế của tổ chức.
- **Xây dựng System Prompt và gọi LLM:** Python xây dựng System Prompt với các quy tắc bắt buộc về cấu trúc SPL, cách xử lý tương quan sự kiện, lọc false positive và định dạng đầu ra. Toàn bộ prompt được gửi đến API Router (hỗ trợ giao thức OpenAI) để mô hình sinh câu lệnh SPL.
- **Parse kết quả và trả về:** Python nhận phản hồi từ AI, loại bỏ các thẻ markdown (spl) và trả về câu lệnh SPL dưới dạng bảng kết quả hiển thị trên giao diện Form. Người dùng có thể sao chép và sử dụng câu lệnh để tìm kiếm trên Splunk.

b) Triển khai chức năng

❖ Đăng ký custom search command

Đăng ký lệnh geminispl để Splunk có thể gọi đến file Python tương ứng.

```
[geminispl]
chunked = true
filename = gemini_spl.py
python.version = python3
```

- **chunked = true:** Cho phép dữ liệu trả về được chia thành nhiều chunk.
- **filename = gemini_spl.py:** Tên file Python xử lý lệnh này.
- **python.version = python3:** Sử dụng Python 3 để thực thi.

❖ Đọc Data Dictionary từ file JSON

Python đọc file `data_dictionary.json` chứa cấu trúc index, sourcetype, EventCode và các trường dữ liệu của hệ thống, phục vụ cho kỹ thuật RAG.

```
base_dir = os.path.dirname(os.path.abspath(__file__))
dict_path = os.path.join(base_dir, "data_dictionary.json")
with open(dict_path, 'r', encoding='utf-8') as f:
    data_dict = json.load(f)
dict_str = json.dumps(data_dict, ensure_ascii=False, indent=2)
```

- **data_dictionary.json:** Tập chứa metadata về index, sourcetype, EventCode.
- **json.dumps(...):** Chuyển thành chuỗi JSON có định dạng đẹp, giữ nguyên tiếng Việt.

❖ Xây dựng System Prompt với ràng buộc SPL

Python nhúng Data Dictionary vào System Prompt cùng các quy tắc bắt buộc về cấu trúc SPL, tương quan sự kiện, lọc false positive và định dạng đầu ra.

```
system_instruction = f"""Ban là SOC Analyst và chuyên gia Splunk SPL.
Tu dien Du lieu cua he thong:
{dict_str}
NHIEM VU VA QUY TRINH TAO SPL (BAT BUOC):
1. BASE SEARCH: Chon dung index, sourcetype, EventCode tu Tu dien.
2. USER CONDITION: Phan dinh ro ParentImage va Image.
3. LOC FALSE POSITIVE: Dung `| search NOT (...)` o cuoi.
4. FORMAT: KET THUC bang `| table _time host` va cac fields quan trong.
5. TUYET DOI chi tra ve 1 dong lenh SPL. KHONG dung markdown.
"""
```

- **dict_str:** Dữ liệu từ điển giúp AI biết tên index, sourcetype thực tế (kỹ thuật RAG).
- **BASE SEARCH:** Yêu cầu AI chọn đúng index, sourcetype, EventCode.
- **USER CONDITION:** Phân biệt ParentImage và Image.
- **LOC FALSE POSITIVE:** Đưa điều kiện loại trừ vào `| search NOT (...)` thay vì `!=`.
- **KHÔNG dung markdown:** Tránh AI bọc SPL trong spl ...

❖ Gửi request đến LLM Gateway qua API

Python gửi System Prompt và câu hỏi người dùng đến API Router để sinh câu lệnh SPL.

```
payload = {
    "model": "Splunk-AI",
    "messages": [
        {"role": "system", "content": system_instruction},
        {"role": "user", "content": self.prompt}
    ],
    "temperature": 0.2
}
req = urllib.request.Request(url, data=json.dumps(payload).encode(),
headers=headers)
with urllib.request.urlopen(req, timeout=60) as resp:
    ai_spl = json.loads(resp.read())['choices'][0]['message']['content']
```

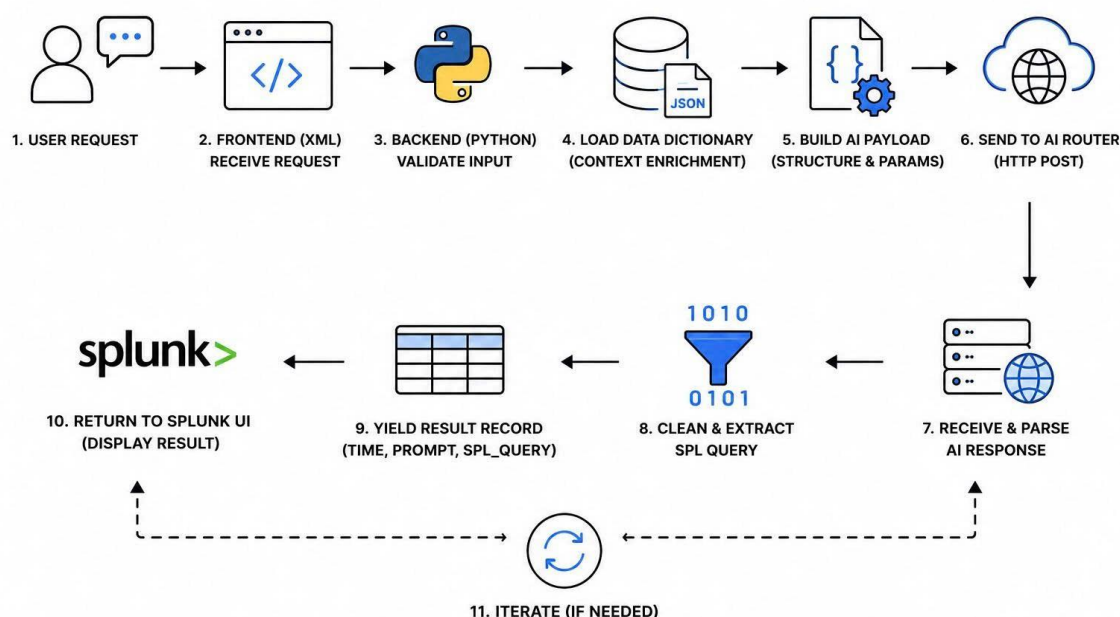
- **temperature: 0.2:** Kiểm soát độ ngẫu nhiên, giữ ở mức thấp để kết quả nhất quán.
- **timeout=60:** Giới hạn thời gian chờ phản hồi từ API.
- Các lỗi (HTTP, JSON, timeout) được bắt gọn ở ngoài hoặc lược bỏ để làm nổi bật luồng chính.

❖ Yield kết quả trả về pipeline

Python đóng gói câu hỏi gốc và câu lệnh SPL đã làm sạch thành dictionary và dùng yield để bơm vào pipeline Splunk.

```
yield {'_time': time.time(), 'Yeu_Cau': self.prompt, 'SPL_Query': ai_spl}
```

- **Yeu_Cau:** Câu hỏi ban đầu của người dùng (tiếng Việt).
- **SPL_Query:** Câu lệnh SPL do AI sinh ra.
- **yield:** Bơm dòng kết quả vào pipeline, hiển thị dưới dạng bảng.



Hình 2.5. Luồng hoạt động chức năng hỗ trợ truy vấn log

2.4. Cảnh báo tự động

2.4.1. Tích hợp AI hỗ trợ phân tích logs

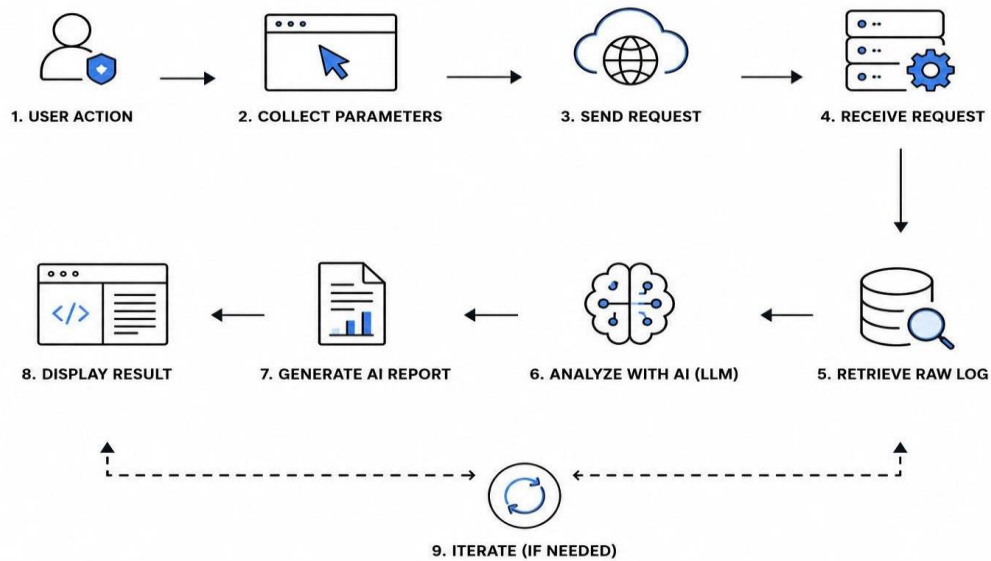
a) Mô tả chức năng

Chức năng phân tích logs là một mô-đun phân tích tương tác cấp sự kiện hoạt động độc lập trong kiến trúc mở rộng của hệ thống SIEM Splunk Enterprise. Chức năng này chịu trách nhiệm tiếp nhận, cô lập và giải nghĩa tự động một bản ghi nhật ký phi cấu trúc, phức tạp hoặc chứa mã độc thành một báo cáo kỹ thuật bằng ngôn ngữ tự nhiên. Việc xây dựng chức năng này nhằm mục đích thiết lập một đường ống xử lý dữ liệu an toàn, kết nối trực tiếp lõi lưu trữ của SIEM với động cơ suy luận ngôn ngữ lớn chạy cục bộ để giải mã ngữ cảnh hành vi của các sự kiện an ninh mạng, giúp giảm thiểu tối đa thời gian điều tra sự cố.

Phương thức vận hành của chức năng được thực hiện thông qua một chuỗi quy trình xử lý dữ liệu khép kín, bao gồm năm giai đoạn kỹ thuật cốt lõi từ lúc tiếp nhận thông tin cho đến khi kết xuất kết quả:

- **Trích xuất dữ liệu gốc:** Khi một sự kiện nhật ký được chỉ định phân tích, hệ thống sử dụng các thông tin định danh hệ thống để thực hiện thuật toán truy vấn tối ưu, đi xuyên qua tầng lọc giao diện và tiếp cận thẳng vào phân vùng lưu trữ trên đĩa cứng của SIEM. Cơ chế này đảm bảo thu về chính xác nội dung chuỗi nhật ký thô nguyên thủy phục vụ cho công tác điều tra số, loại bỏ hoàn toàn nguy cơ mất mát ký tự hoặc sai lệch thông tin do các bộ lọc hiển thị của ứng dụng web.

- **Tiền xử lý và chuẩn hóa dữ liệu đầu vào:** Chuỗi nhật ký thô sau khi trích xuất sẽ được hệ thống làm sạch, xử lý các ký tự ẩn bẫy dòng và cấu trúc hóa lại. Dữ liệu chuẩn hóa này được lồng ghép vào System Prompt đóng vai trò làm hành lang kiểm soát tư duy của mô hình AI. Kịch bản này thiết lập các quy tắc ràng buộc khẳng định nghiêm ngặt, bắt buộc động cơ AI chỉ được phép lập luận dựa trên các Token thực tế có trong log, triệt tiêu hoàn toàn nguy cơ mô hình tự ý biên diễn hoặc sinh thông tin giả mạo.
- **Biên dịch kỹ thuật và giải nghĩa ngữ cảnh:** Gói dữ liệu hoàn chỉnh (Payload) được truyền tải bất đồng bộ thông qua kết nối HTTP nội bộ (Local Loopback) sang hệ thống mô hình ngôn ngữ lớn hoạt động trong môi trường cô lập mạng (Air-gapped). Tại đây, mô hình thực hiện tiến trình suy luận để bóc tách các thực thể an ninh, giải mã các đoạn mã thực thi ẩn (như chuỗi mã hóa Base64 hoặc lệnh PowerShell ẩn danh) và chuyển dịch toàn bộ các tham số kỹ thuật cấp thấp phức tạp sang văn bản ngữ cảnh bằng ngôn ngữ tự nhiên.
- **Ánh xạ an ninh và phân cấp độ rủi ro nghiệp vụ:** Dựa trên kết quả dịch giải ngữ cảnh, logic hệ thống yêu cầu mô hình thực hiện phân loại mức độ đe dọa của sự kiện theo các thang đo an toàn thông tin chuẩn hóa. Đồng thời, mô hình tiến hành đối chiếu hành vi của dòng log với các chiến thuật, kỹ thuật tấn công chuyên biệt để ánh xạ trực tiếp vào khung kỹ nghệ tấn công MITRE ATT&CK. Quá trình này áp đặt điều kiện kiểm soát nhiều chặt chẽ: nếu sự kiện được xác định là hành vi hệ thống hợp lệ, các trường thông tin an ninh sẽ được gán nhãn đóng để tránh tạo ra cảnh báo giả.
- **Kiểm soát an toàn dữ liệu đầu ra và chịu lỗi hệ thống:** Để bảo vệ nền tảng giám sát an ninh mạng, chuỗi dữ liệu báo cáo kết quả bắt buộc phải đi qua một bộ lọc mã hóa thực thể HTML (HTML Entity Encoding) nhằm triệt tiêu các Payload mã độc có thể được kẻ tấn công cài cắm trong log để tấn công trình duyệt của chuyên gia (XSS Sanitization). Song song với đó, hệ thống cấu hình các thuộc tính ngắt dòng bắt buộc đối với các chuỗi ký tự dài (như mã băm hoặc mã độc thực thi) để chống tràn bộ đệm hiển thị. Toàn bộ tiến trình được bao bọc bởi cơ chế Try-Catch ở tầng Backend, trong trường hợp động cơ AI bị quá tải tài nguyên phần cứng hoặc gặp sự cố gián đoạn, hệ thống sẽ tự động ngắt luồng chờ và trả về thông báo lỗi chuyên nghiệp, đảm bảo tính sẵn sàng cao cho toàn bộ hệ thống SIEM.



Hình 2.6. Luồng hoạt động chức năng hỗ trợ phân tích log

b) Triển khai chức năng

❖ Đóng gói thông số ghi vào URL ghi định vị

Khi người dùng click vào Event Action tại một dòng log trên màn hình Search, Splunk đọc workflow_actions.conf và tự động đóng gói các tham số định vị vật lý của bản ghi vào URL rồi mở tab Dashboard mới.

```
# workflow_actions.conf
link.uri =
/app/gemini_spl/log_analyzer_view?index=$index$&time=$_time$&host=$host$&st=$sourcetype$&ec=$EventCode$&bkt=$_bkt$&cd=$_cd$
```

- **\$_bkt\$**: Định danh vật lý của bucket trên đĩa cứng chứa bản ghi (ví dụ: main~12~34), được Splunk tự động điền từ metadata của dòng log đang được chọn.
- **\$_cd\$**: Byte offset của bản ghi trong bucket, kết hợp với _bkt xác định duy nhất một dòng log.
- **\$_time\$**: Epoch timestamp của sự kiện, dùng làm mốc xây dựng lưới thời gian ở các bước tiếp theo.
- **\$index\$, \$host\$, \$sourcetype\$, \$EventCode\$**: Các tham số hỗ trợ phục vụ cho việc thu hẹp phạm vi truy vấn khi _bkt và _cd không đủ để định vị.

❖ Đọc tham số định vị từ URL

Khi Dashboard load, JavaScript đọc toàn bộ tham số định vị bản ghi từ URL để chuẩn bị cho bước gửi request đến REST Endpoint.

```
// log_analyzer.js
function getUrlParams() {
    var params = {};
    var searchParams = new URLSearchParams(window.location.search);
    for (var key of searchParams.keys()) {
        params[key] = searchParams.get(key);
    }
    return params;
}
var params = getUrlParams();
```

- **window.location.search:** Trỏ đến phần query string của URL hiện tại, nơi chứa toàn bộ tham số định vị bản ghi đã được workflow_actions.conf đóng gói ở bước trước.
- **URLSearchParams:** API trình duyệt parse chuỗi query string thành các cặp key-value, cho phép truy xuất từng tham số theo tên mà không cần xử lý chuỗi thủ công.

❖ **Mã thông báo CSRF được gửi đến điểm cuối REST bằng AJAX GET**

JavaScript lấy mã thông báo CSRF và cookie của phiên bản Splunk hiện tại, đính kèm tệp đính kèm vào tiêu đề và gửi toàn bộ tham số định vị đến điểm cuối REST của Python qua AJAX.

```
// log_analyzer.js
function getCsrftoken() {
    var match = document.cookie.match(new
    RegExp('(^\s)splunkweb_csrf_token_8000=([^\s;]+)'));
    if (match) return match[2];
    return "";
}
$.ajax({
    url: restUrl,
    type: 'GET',
    data: params,
    headers: { 'X-Splunk-Form-Key': getCsrftoken() },
    timeout: 180000,
});
```

- **document.cookie.match(splunkweb_csrf_token_8000)**: Trích xuất CSRF token từ cookie của phiên Splunk, token này được Splunk Web Framework yêu cầu bắt buộc trên mọi request đến REST Endpoint để xác thực nguồn gốc hợp lệ.
- **X-Splunk-Form-Key**: Header chứa CSRF token, đây là tên header chuẩn mà Splunk Web Framework kiểm tra trước khi cho phép request chạm vào REST Endpoint.
- **data: params**: Chuyển tiếp toàn bộ tham số định vị bản ghi sang Python REST Handler.
- **timeout: 180000**: Giới hạn thời gian chờ 180 giây, bao phủ toàn bộ thời gian xử lý của Python bao gồm cả giai đoạn suy luận của Ollama.

❖ Xây dựng mạng lưới thời gian hạn chế phạm vi quét

Python tiếp nhận tham số time từ request và xây dựng lưới thời gian hẹp quanh timestamp của sự kiện, buộc Splunk chỉ mở các bucket nằm trong khoảng 5 giây này thay vì quét toàn bộ index.

```
# log_analyzer.py
t = float(log_time)
earliest = int(t) - 2
latest = int(t) + 3
search_query = f'search index="{log_index}" earliest={earliest} latest={latest}'
```

- **float(log_time)**: Chuyển đổi timestamp dạng chuỗi từ URL sang số thực epoch, làm cơ sở tính toán hai mốc thời gian biên.
- **earliest = t - 2**: Mốc thời gian dưới lùi 2 giây, đảm bảo không bỏ sót bản ghi do lệch đồng hồ nhỏ giữa các thành phần.
- **latest = t + 3**: Mốc thời gian trên tiến 3 giây, tạo lưới thời gian tổng cộng 5 giây, Splunk chỉ mở các bucket nằm trong khoảng này, bỏ qua toàn bộ phần còn lại của index.

❖ Xây dựng câu hỏi SPL với _bkt, _cd và thực thi Pipeline Bypass

Sau khi thu hẹp phạm vi bằng lưới thời gian, Python bổ sung mệnh đề where với tọa độ vật lý _bkt và _cd lấy chính xác một dòng log duy nhất, sau đó thực thi qua simpleRequest.

```
# log_analyzer.py
search_query += f' | where _bkt="{log_bkt}" AND _cd="{log_cd}"'
search_query += ' | head 1'
response, content = splunk.rest.simpleRequest(
    '/services/search/jobs/export',
```

```

    sessionKey=session_key,
    postargs={'search': search_query, 'exec_mode': 'oneshot', 'output_mode':
'json'},
    method='POST'
)

```

- | **where _bkt AND _cd**: Mệnh đề lọc tọa độ vật lý ở tầng pipeline, trở thẳng vào đúng sector lưu trữ trên đĩa cứng sau khi lưới thời gian đã thu hẹp phạm vi, bảo toàn toàn vẹn nội dung raw log không qua bất kỳ bộ lọc hiển thị nào.
- | **head 1**: Ngắt pipeline ngay sau khi tìm thấy đúng một kết quả, ngăn engine tiếp tục quét các bucket còn lại.
- **exec_mode=oneshot**: Yêu cầu Splunk thực thi đồng bộ và trả kết quả ngay trong cùng một HTTP response, phù hợp với luồng xử lý tuần tự của REST Handler.
- **/services/search/jobs/export**: Endpoint nội bộ của Splunk thực thi SPL và xuất kết quả trực tiếp không qua lớp giao diện web, là điểm mấu chốt của kỹ thuật Pipeline Bypass.

❖ Phản hồi phân tích cú pháp đi tới trích xuất_raw

Python parse từng dòng JSON trong response của Splunk để lấy đúng trường _raw chứa nội dung log nguyên thủy.

```

# log_analyzer.py
lines = content.decode('utf-8').split('\n')
for line in lines:
    data_json = json.loads(line)
    if 'result' in data_json and '_raw' in data_json['result']:
        raw_log = data_json['result']['_raw']
        break

```

- **content.decode('utf-8').split('\n')**: Response từ Splunk trả về dạng JSON Lines (mỗi dòng là một JSON object độc lập), cần tách theo ký tự xuống dòng trước khi parse từng phần tử.
- **data_json['result']['_raw']**: Trở đúng vào trường chứa chuỗi log nguyên thủy chưa qua bất kỳ bước transform nào của Splunk, đảm bảo tính toàn vẹn dữ liệu đầu vào cho AI.
- **break**: Dừng vòng lặp ngay khi tìm thấy _raw, tránh xử lý các dòng JSON metadata không cần thiết phía sau.

❖ Xây dựng System Prompt với các ràng buộc tích cực và đóng gói Payload

Raw log được nhúng vào System Prompt được thiết kế theo kỹ thuật Positive Constraints, sau đó đóng gói thành Payload JSON để truyền đến Ollama.


```
# log_analyzer.py
system_prompt = (
    "RULE 1 - ZERO INVENTIONS: NEVER invent, guess, or append IPs, Hashes, "
    "Domains, or Usernames that are NOT explicitly written in the raw log.\n"
    "RULE 2 - MITRE ATT&CK ACCURACY: DO NOT generate specific MITRE "
    "Technique "
    "IDs unless you are mathematically certain. Use broad Tactic names "
    "instead.\n"
    "***1. Tóm tắt Sự kiện:**\n"
    "***2. Dữ liệu Điều tra Cốt lõi:**\n"
    "***3. Phân tích Ngữ cảnh Chuyên sâu:**\n"
    "***4. Đánh giá Rủi ro & Khung MITRE:**\n"
    "- **Mức độ rủi ro:** [Benign / Low / Medium / High / Critical]\n"
)
payload = {
    "model": "llama3.2:3b",
    "prompt": f"System: {system_prompt}\n\nUser: --- [RAW LOG DATA] --- "
    "\n{raw_log}",
    "stream": False
}
```

- **RULE 1 - ZERO INVENTIONS:** Ràng buộc cứng ngăn mô hình tự sinh thêm các dữ liệu không có trong raw log, ngăn chặn nguy cơ Hallucination ở tầng IOC.
- **RULE 2 - MITRE ATT&CK ACCURACY:** Giới hạn mô hình chỉ sử dụng tên Tactic cấp cao thay vì Technique ID cụ thể, do mô hình không thể xác minh tính chính xác của ID theo thời gian thực.
- **System: {system_prompt}\n\nUser: {raw_log}:** Cấu trúc phân tách System Prompt và dữ liệu người dùng bằng dấu phân cách rõ ràng, ngăn nội dung raw log can thiệp vào Prompt Injection Prevention.
- **stream: False:** Yêu cầu Ollama tích lũy toàn bộ output trước khi trả về một JSON response duy nhất, phù hợp với luồng xử lý đồng bộ của REST Handler.

❖ Gửi Payload đến Ollama qua HTTP loopback

Payload được truyền tải qua kết nối HTTP nội bộ đến tiến trình Ollama, mô hình thực hiện suy luận và trả về kết quả phân tích theo Output Schema đã ràng buộc trong System Prompt.

```
# log_analyzer.py
req = urllib.request.Request(
    "http://127.0.0.1:11434/api/generate",
```

```

data=json.dumps(payload).encode('utf-8'),
method='POST'
)
with urllib.request.urlopen(req, timeout=120) as res:
    ai_reply = json.loads(res.read().decode('utf-8')).get("response", "")

```

- **http://127.0.0.1:11434/api/generate:** Địa chỉ loopback nội bộ của tiến trình Ollama, đảm bảo toàn bộ dữ liệu log nhạy cảm không rời khỏi máy chủ trong suốt quá trình suy luận.
- **json.dumps(payload).encode('utf-8'):** Serialize Payload thành bytes UTF-8 trước khi truyền qua HTTP, đảm bảo các ký tự đặc biệt trong raw log không bị mã hóa sai trong quá trình truyền tải.
- **timeout=120:** Giới hạn thời gian chờ 120 giây cho riêng giai đoạn suy luận của Ollama, ngăn tiến trình Python bị treo khi động cơ AI quá tải.
- **.get("response"):** Trích xuất trường kết quả từ JSON response của Ollama, chứa toàn bộ báo cáo phân tích theo Output Schema đã ràng buộc trong System Prompt.

❖ Nhận AJAX response và thực hiện XSS Sanitization

JavaScript nhận kết quả từ Python qua AJAX callback và thực hiện HTML Entity Encoding để triệt tiêu Payload mã độc có thể được nhúng trong log trước khi đưa lên giao diện.

```

// log_analyzer.js
success: function(response) {
    var rawReply    = data.ollama_reply || "";
    var escapedReply = $('<div>').text(rawReply).html();
    var formattedReply = escapedReply.replace(/\n/g, "<br>");
}

```

- **\$('<div>').text(rawReply).html():** Kỹ thuật HTML Entity Encoding của jQuery, chuyển đổi các ký tự đặc biệt (<, >, ", &) trong AI output thành thực thể HTML an toàn, triệt tiêu vector tấn công XSS có thể được kẻ tấn công nhúng trong log.
- **.replace(/\n/g, "
"):** Chuyển ký tự xuống dòng của Markdown output thành thực thể HTML
, bảo toàn cấu trúc trình bày của báo cáo sau bước Entity Encoding.

❖ Inject DOM và hiển thị Split-screen

Sau khi sanitization hoàn tất, JavaScript inject nội dung báo cáo vào đúng ô phân tích bên phải của bố cục Split-screen 50/50, hoàn tất vòng đời xử lý của chức năng.

```
// log_analyzer.js
$("#ai_status").hide();
$("#ai_result").show();
$("#cell_prompt").text(data.prompt || "");
$("#cell_reply").html(formattedReply);
```

- **`$("#ai_status").hide()`**: Ẩn trạng thái chờ "Calling Ollama..." khi kết quả đã về, chuyển giao diện sang chế độ hiển thị báo cáo.
- **`$("#ai_result").show()`**: Hiển thị bố cục Split-screen 50/50 đã được ẩn trong lúc chờ xử lý.
- **`$("#cell_prompt").text(data.prompt)`**: Inject raw log vào ô bên trái dưới dạng plain text, giữ nguyên định dạng nguyên thủy để điều tra viên đối chiếu chéo với báo cáo phân tích.
- **`$("#cell_reply").html(formattedReply)`**: Inject nội dung báo cáo phân tích đã qua sanitization vào ô bên phải, sử dụng `.html()` thay vì `.text()` để render các thẻ `<br>` đã được chuyển đổi từ ký tự xuống dòng.

2.4.2. Tích hợp AI hỗ trợ điều tra sự cố

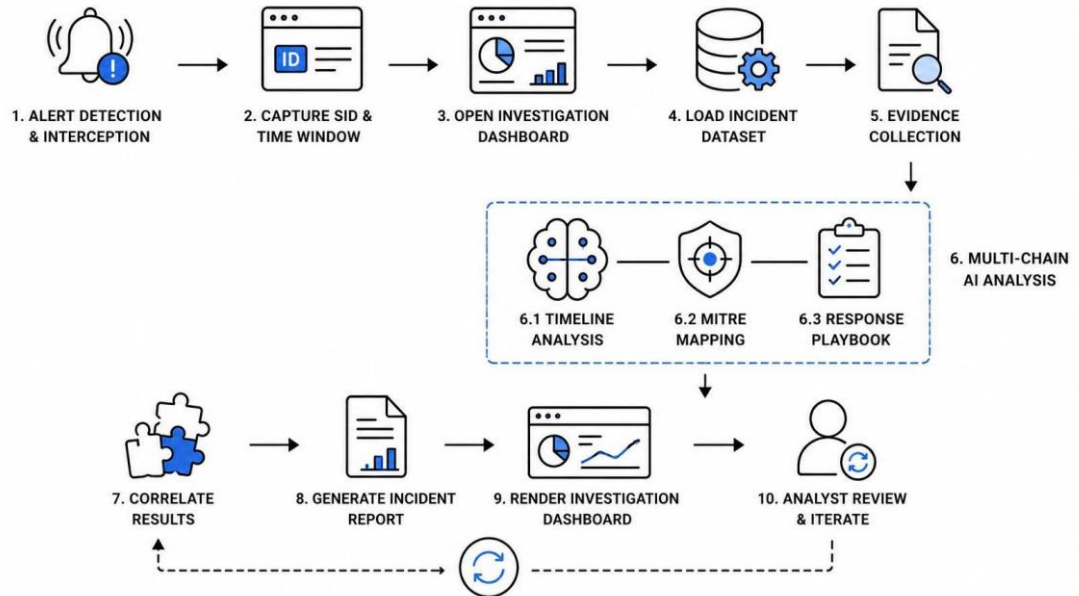
a) Mô tả chức năng

Chức năng điều tra sự cố là một mô-đun phân tích cấp cảnh báo hoạt động trong kiến trúc mở rộng của hệ thống SIEM Splunk Enterprise. Chức năng này chịu trách nhiệm tự động tổng hợp toàn bộ dữ liệu liên quan đến một cảnh báo an ninh, bao gồm các log xung quanh thời điểm phát sinh sự cố, sau đó sử dụng mô hình ngôn ngữ lớn để tái tạo dòng thời gian tấn công, ánh xạ sang ma trận MITRE ATT&CK và đề xuất các bước ứng cứu ban đầu.

Phương thức vận hành của chức năng được thực hiện thông qua một chuỗi quy trình xử lý dữ liệu khép kín, bao gồm bảy giai đoạn kỹ thuật cốt lõi từ lúc tiếp nhận cảnh báo cho đến khi kết xuất kết quả:

- **Đánh chặn cảnh báo và trích xuất tham số**: Khi người dùng bấm nút điều tra trên một cảnh báo, JavaScript đánh chặn hành vi mặc định của Splunk, trích xuất SID của cảnh báo từ cột thứ tư của bảng và khoảng thời gian điều tra do người dùng nhập, sau đó chuyển hướng đến Dashboard để điều tra sự cố và kèm theo các tham số `form.sid` và `form.time`.
- **Hứng tham số và gọi Custom Search Command**: Giao diện Dashboard sử dụng thẻ `<search>` với câu lệnh `| incidentinvestigator sid = "$form.sid$" time = "$form.time$"` để kích hoạt custom search command. Splunk spawn một tiến trình con chạy file `incident_investigator.py` và truyền tham số qua STDIN.

- **Định vị sự cố:** Python nhận SID và khoảng thời gian, kết nối với lõi Splunk thông qua SDK, đọc kết quả của job từ SID, trích xuất chính xác thời điểm Root Time và Victim Host. Xử lý trường hợp job bị hết hạn bằng cách sử dụng thời gian nổ Alert làm tâm chấn dự phòng.
- **Cần quét dữ liệu diện rộng:** Dựa trên tâm chấn đã xác định, hệ thống tự động xây dựng truy vấn SPL để quét toàn bộ log của máy chủ bị ảnh hưởng trong khoảng thời gian T_gốc +/- N phút (N do người dùng nhập). Truy vấn chỉ lấy các EventCode quan trọng (1, 3, 11, 4104, 4688, 4624, 4625, 7045) và loại bỏ các log nhiễu từ hệ thống.
- **Multi-Chain AI xử lý:** Do giới hạn ngữ cảnh của mô hình ngôn ngữ lớn, dữ liệu log không thể gửi toàn bộ một lần. Hệ thống chia quá trình xử lý thành ba Chain nối tiếp: Chain 1 nhận log context, tóm tắt và tái tạo Timeline sự cố; Chain 2 nhận Timeline làm ngữ cảnh, ánh xạ các hành vi sang ma trận MITRE ATT&CK; Chain 3 nhận Timeline và MITRE, đề xuất sổ tay ứng phó khẩn cấp. Mỗi Chain gọi mô hình AI cục bộ qua API <http://localhost:11434/api/generate> với nhiệt độ 0.1 để đảm bảo kết quả nhất quán.
- **Yield dữ liệu trở về pipeline:** Sau khi hoàn thành ba Chain, Python gộp tất cả kết quả (Timeline, MITRE, Checklist, Victim, Earliest, Latest) vào một dictionary và sử dụng lệnh yield để bơm dữ liệu trở lại pipeline của Splunk dưới dạng một dòng log giả lập.
- **Hiển thị Dashboard đa panel:** Các token được gán giá trị từ kết quả yield, kích hoạt 6 panel hiển thị đồng loạt. Panel Timeline, MITRE và Playbook hiển thị trực tiếp nội dung từ AI. Panel Top Talkers và Bảng chứng cứ sử dụng token \$tok_victim\$, \$tok_earliest\$, \$tok_latest\$ để chạy truy vấn độc lập xuống cơ sở dữ liệu, đảm bảo thông tin được cập nhật chính xác theo ngữ cảnh của sự cố.



Hình 2.7. Luồng hoạt động chức năng hỗ trợ điều tra sự cố

b) Triển khai chức năng

❖ Đánh chặn cảnh báo và trích xuất SID

Trích xuất SID từ cột thứ tư của bảng và yêu cầu người dùng nhập khoảng thời gian quét.

```

$(document).on("click", "td:contains('Analysis alert')", function(e) {
    e.preventDefault();
    var sid = $(this).closest("tr").find("td:nth-child(4)").text().trim();
    var timeWindow = prompt("Nhập số phút quét mở rộng:", "5");
    window.location.href = "incident_investigation_dashboard?form.sid=" +
    encodeURIComponent(sid) + "&form.time=" + timeWindow;
});
  
```

- **e.preventDefault():** Ngăn chặn hành vi mặc định của Splunk (mở trang search thông thường).
- **td:nth-child(4):** SID của alert nằm ở cột thứ tư trong bảng Triggered Alerts.
- **prompt():** Hộp thoại yêu cầu người dùng nhập số phút quét mở rộng (mặc định 5).
- **encodeURIComponent(sid):** Mã hóa SID để truyền an toàn qua URL.

❖ Nhận tham số và gọi custom search command

Giao diện Dashboard sử dụng thẻ <search> để nhận tham số từ URL và gọi custom search command incidentinvestigator.

```
<search id="ai_multi_chain_core">
  <query>| incidentinvestigator sid="$form.sid$"
time="$form.time$"</query>
  <done>
    <set token="tok_timeline">$result.Timeline$</set>
    <set token="tok_victim">$result.Victim$</set>
    <set token="tok_earliest">$result.Earliest$</set>
    <set token="tok_latest">$result.Latest$</set>
  </done>
</search>
```

- **\$form.sid\$, \$form.time\$:** Token nhận giá trị SID và khoảng thời gian từ URL.
- **| incidentinvestigator:** Custom search command, gọi đến incident_investigator.py.
- **\$result.Timeline\$:** Token nhận giá trị từ trường Timeline do Python yield ra.

❖ Xử lý SID và lấy sự cố

Python nhận SID, kết nối với lõi Splunk qua SDK, đọc kết quả job để xác định Root Time và Victim Host.

```
try:
    alert_job = service.jobs[self.sid]
except KeyError:
    yield {"Timeline": "Job expired", "Victim": "N/A"}
    return

alert_results = results.ResultsReader(alert_job.results())
for result in alert_results:
    t_goc = float(result.get('_time', time.time()))
    victim_host = result.get('host', '*')
    break
```

- **service.jobs[self.sid]:** Truy xuất job đã lưu trong Splunk thông qua SID.
- **KeyError:** Job bị xóa do quá hạn, trả về thông báo lỗi thay vì crash.

- **result.get('_time', time.time()):** Lấy _time từ log đầu tiên; nếu không có thì dùng thời gian hiện tại.
- **result.get('host', '*'):** Lấy tên máy chủ bị ảnh hưởng; mặc định * (quét toàn bộ).

❖ Quét dữ liệu diện rộng

Dựa trên cảnh báo đã xác định, Python xây dựng truy vấn SPL để quét toàn bộ log liên quan đến máy chủ bị ảnh hưởng trong khoảng thời gian mở rộng.

```
earliest_time = int(t_goc - int(self.time) * 60)
latest_time = int(t_goc + int(self.time) * 60)
search_query = f'search host="{victim_host}" EventCode IN
(1,3,11,4104,4688) earliest={earliest_time} latest={latest_time} | head 400'
oneshot_results = service.jobs.oneshot(search_query, output_mode="json")
log_context = oneshot_results.read().decode('utf-8')[:3000]
```

- **earliest_time / latest_time:** Xác định khoảng thời gian quét: $T_{\text{gốc}} \pm N$ phút (N do người dùng nhập).
- **EventCode IN (1,3,11,4104,4688):** Chỉ lấy các EventCode quan trọng (Process, Network, File, PowerShell).
- **head 400:** Giới hạn 400 dòng để tránh quá tải bộ nhớ và context window của AI.
- **oneshot():** Thực thi truy vấn và trả kết quả dưới dạng JSON.
- **[:3000]:** Cắt dữ liệu xuống 3000 ký tự để vừa context window của mô hình.

❖ Multi-Chain AI xử lý

Do giới hạn ngữ cảnh của mô hình ngôn ngữ lớn, dữ liệu log không thể gửi toàn bộ một lần. Hệ thống chia quá trình xử lý thành ba Chain nối tiếp.

```
def call_ollama(prompt):
    payload = {"model": "llama3.2:3b", "prompt": prompt, "stream": False,
"options": {"temperature": 0.1}}
    req = urllib.request.Request("http://localhost:11434/api/generate",
data=json.dumps(payload).encode())
    return json.loads(urllib.request.urlopen(req,
timeout=120).read()).get("response", "")
# Chain 1: Tạo Timeline
output_timeline = call_ollama(f'Log: {log_context}\nYêu cầu: Tóm tắt
timeline tấn công')
# Chain 2: Ảnh xạ MITRE
```

```

output_mitre = call_ollama(f"Timeline: {output_timeline}\nYêu cầu: Map
sang MITRE ATT&CK")
# Chain 3: Tạo Playbook
output_checklist = call_ollama(f"Timeline: {output_timeline}\nYêu cầu: Đề
xuất bước ứng phó")

```

- **model: llama3.2:3b:** Mô hình ngôn ngữ nhỏ (3B), chạy cục bộ qua Ollama.
- **temperature: 0.1:** Tham số thấp để kết quả nhất quán, ít ngẫu nhiên.
- **stream: False:** Tắt chế độ stream, đợi phản hồi hoàn chỉnh.
- **timeout: 120:** Thời gian chờ tối đa 120 giây cho mỗi chain.
- **Chain nối tiếp:** Chain 2 dùng output của Chain 1, Chain 3 dùng output của Chain 1.

❖ Yield dữ liệu trở về pipeline

Sau khi hoàn thành ba Chain, Python gộp tất cả kết quả vào một dictionary và sử dụng lệnh yield để bơm dữ liệu trở lại pipeline của Splunk.

```

yield {
    "Timeline": output_timeline,
    "MITRE": output_mitre,
    "Checklist": output_checklist,
    "Victim": victim_host,
    "Earliest": earliest_time,
    "Latest": latest_time
}

```

- **yield:** Đẩy một dòng log giả lập vào pipeline của Splunk
- **Các trường Timeline, MITRE, Checklist, Victim, Earliest, Latest:** Sẽ được file XML hứng qua cú pháp \$result.TênTrường\$.

❖ Hiển thị Dashboard đa panel

Các token được gán giá trị từ kết quả yield, kích hoạt các panel hiển thị đồng loạt. Base search dùng chung giúp các panel truy xuất cùng bộ dữ liệu mà không cần chạy lại truy vấn.

```

<panel>
  <html><div class="text-fade-container">$tok_timeline$</div></html>
</panel>

```



```

<search id="base_splunk_logs">
  <query>index=* host="$tok_victim$"
  earliest=$tok_earliest$ latest=$tok_latest$</query>
</search>
<panel id="panel_raw_artifacts">
  <table search="base_splunk_logs">
    <query>| table _time, host, source, _raw</query>
  </table>
</panel>

```

- **\$tok_timeline\$:** Token hiển thị Timeline do AI tạo ra (Chain 1).
- **\$tok_victim\$, \$tok_earliest\$, \$tok_latest\$:** Token đồng bộ toàn bộ panel trong cùng máy chủ và khung thời gian.
- **search id="base_splunk_logs":** Base search dùng chung, tránh truy vấn nhiều lần.
- **<table search="base_splunk_logs">:** Panel tham chiếu đến base search để hiển thị bảng chứng cứ.

2.4.3. Cơ chế Scheduled Search trong Splunk

Cơ chế Scheduled Search trong Splunk Alert trong Splunk không phải là phép màu, bản chất của nó là các câu lệnh tìm kiếm (Saved Searches) được lập lịch chạy ngầm.

- **Cron Expression:** Splunk sử dụng cú pháp Cron để xác định chu kỳ chạy (ví dụ * * * * * nghĩa là chạy mỗi phút 1 lần).
- **Threshold:** Để giảm thiểu tình trạng ngập lụt cảnh báo (Alert Fatigue), hệ thống yêu cầu cấu hình Trigger alert when. Ví dụ: chỉ nổ cảnh báo dò mật khẩu (Brute-force) khi số lượng sự kiện Failed_Attempts >= 15 trong 5 phút.
- **Throttling:** Cơ chế chặn các cảnh báo trùng lặp. Nếu một IP liên tục tấn công, Throttling giúp hệ thống chỉ gửi 1 cảnh báo đầu tiên và "im lặng" trong một khoảng thời gian nhất định (ví dụ 60 giây).

2.4.4. Trigger Alert và Add-on Telegram

a) Cơ chế Trigger Alert

Khi các điều kiện đã thiết lập trong quá trình tìm kiếm (như Scheduled Search) được thỏa mãn—ví dụ như số lượng kết quả trả về vượt qua một ngưỡng nhất định—hệ thống Splunk sẽ tự động chuyển sang trạng thái kích hoạt cảnh báo và tiến hành thực thi các "Trigger Actions" (Hành động phản hồi). Quy trình này

đóng vai trò quyết định trong việc biến các kết quả truy vấn thô thành các thông tin có thể hành động được.

- **Đóng gói dữ liệu sự cố:** Splunk sẽ thu thập các bản ghi vi phạm và bóc tách các trường thông tin quan trọng nhất từ kết quả của câu lệnh tìm kiếm SPL.
- **Cấu hình thông điệp động :** Quản trị viên tùy biến nội dung cảnh báo bằng các biến động (dynamic variables) với cú pháp `$result.tên_trường$`. Ví dụ, `$result.host$`, `$result.SourceUser$`, `$result.Users_Logged_In$` sẽ tự động điền giá trị thực tế của sự cố, giúp chuyên gia phân tích nắm bắt nhanh bởi cảnh mà không cần đăng nhập vào Splunk.

b) Tích hợp Add-on Telegram

Để đáp ứng yêu cầu về thời gian phản ứng (MTTR) trong môi trường SOC, thay vì chỉ lưu cảnh báo thụ động trên hệ thống, Splunk được cấu hình để truyền tải thông tin ra bên ngoài thông qua Add-on Telegram. Thành phần này đóng vai trò như một cầu nối API, giúp Splunk tương tác trực tiếp với nền tảng nhắn tin ngoại vi:

- **Thiết lập tham số kết nối:** Để luồng dữ liệu được định tuyến chính xác và bảo mật, hệ thống cần được khai báo hai tham số bắt buộc. Thứ nhất là `Bot token`—khóa API do Telegram cấp để xác thực quyền truy cập của Splunk; thứ hai là `Chat ID`—mã định danh trò đích xác đến nhóm chat hoặc cá nhân chịu trách nhiệm tiếp nhận sự cố.
- **Kết xuất và truyền tải:** Ngay khi Trigger Alert được kích hoạt ở bước trước, Add-on sẽ đóng gói thông điệp đã được làm giàu thông tin và đẩy qua giao thức HTTPS tới máy chủ API của Telegram. Kết quả thực nghiệm cho thấy, khi cảnh báo nổ ra, một tin nhắn văn bản tức thời sẽ được gửi thành công đến ứng dụng Telegram của người trực. Tin nhắn này chứa đầy đủ các trường dữ liệu quan trọng, giúp đội ngũ bảo mật nhanh chóng nhận diện chính xác các thông tin thiết yếu của sự cố để đưa ra biện pháp ứng phó kịp thời.

2.5. Trực quan hoá và Dashboard

2.5.1. Các loại biểu đồ

Splunk cung cấp nhiều công cụ trực quan hóa nhằm giảm thiểu thời gian phân tích (MTTR):

- **Timechart:** Phân tích tần suất sự kiện theo dòng thời gian (Event Spike). Rất hữu hiệu để nhận diện các đợt tấn công DDoS hoặc Brute-force có tính chu kỳ.

- **Pie Chart/Bar Chart:** Hiển thị phân bố tỉ lệ, thường dùng để bóc tách Top Talkers (Các IP/Process tạo ra nhiều kết nối hoặc log nhất).
- **Data Table:** Bảng hiển thị dữ liệu chứng cứ gốc (Raw Artifacts), cho phép điều tra viên xem chi tiết CommandLine, TargetFilename mà không cần phải tự gõ lại lệnh tìm kiếm.

2.5.2. Thiết kế Dashboard tích hợp AI

Sự khác biệt của hệ thống nằm ở khả năng tương tác linh hoạt trên giao diện:

- **Cơ chế truyền tham số (Token Passing):** Giao diện sử dụng các biến token như `$form.sid$` (Mã định danh cảnh báo) và `$tok_victim$` (Máy chủ nạn nhân). Khi người dùng click vào một cảnh báo, token thay đổi sẽ tự động cập nhật toàn bộ các biểu đồ liên quan mà không cần load lại trang.
- **Base Search Optimization:** Để tránh việc mỗi panel (ô biểu đồ) phải tự chạy lại một câu lệnh SPL nặng nề gây nghẽn máy chủ Search Head, Dashboard sử dụng cơ chế `base search`. Một luồng dữ liệu tổng quát được kéo về bộ nhớ tạm, sau đó các panel khác nhau (Timeline, Table, AI Report) sẽ chia sẻ và tái sử dụng luồng dữ liệu này.

2.6. Tổng kết chương 2

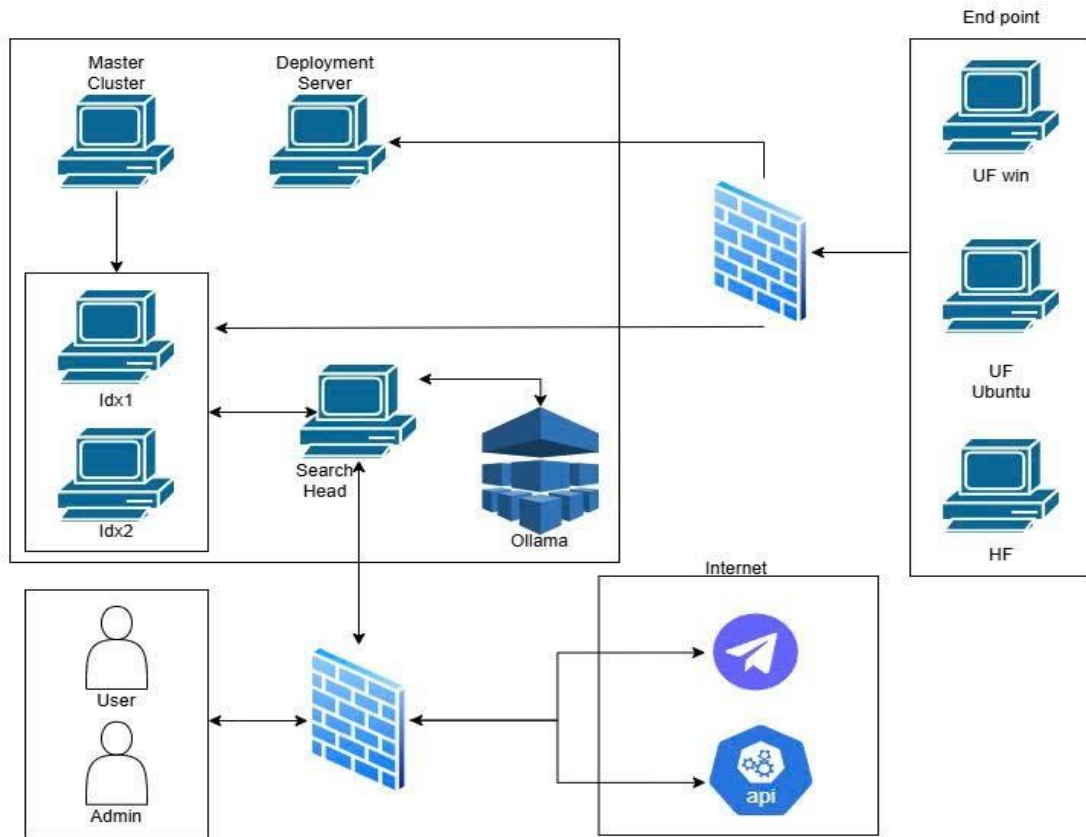
Trên cơ sở lý thuyết đã xây dựng, chương 2 tập trung vào thiết kế và triển khai hệ thống thực tế. Mô hình thu thập dữ liệu được xây dựng dựa trên đặc điểm của từng nguồn log, sử dụng Universal Forwarder cho các máy chủ và Heavy Forwarder làm Syslog listener cho pfSense. Điểm quan trọng là việc áp dụng chuẩn hóa CIM để các log từ nhiều nguồn khác nhau có thể được xử lý thống nhất, từ đó nâng cao hiệu quả tương quan sự kiện và xây dựng rule phát hiện.

Về kỹ thuật phát hiện, các rule SPL được xây dựng dựa trên ba cách tiếp cận chính: phân tích ngữ nghĩa dòng lệnh để nhận diện tham số bất thường, tương quan cây tiến trình để phát hiện chuỗi hành vi đáng ngờ, và phân tích ngưỡng để phát hiện các tấn công dựa trên tần suất. Các kỹ thuật này hướng tới việc phát hiện kịch bản tấn công T1219 sử dụng AnyDesk và các hành vi liên quan trong thực nghiệm.

Điểm nhấn của chương là việc tích hợp AI vào Splunk với ba chức năng hỗ trợ: truy vấn log bằng ngôn ngữ tự nhiên, phân tích chi tiết từng dòng log và điều tra sự cố cấp độ cảnh báo. Các chức năng này cho thấy tiềm năng của việc ứng dụng LLM trong vận hành SOC, giúp người dùng tương tác với hệ thống một cách tự nhiên hơn và giảm khối lượng công việc thủ công. Cuối cùng, cơ chế Scheduled Search và Trigger Alert qua Telegram cùng với Dashboard giám sát được thiết kế để hoàn thiện một quy trình vận hành khép kín từ phát hiện đến cảnh báo và trực quan hóa.

CHƯƠNG 3. THỰC NGHIỆM TÍCH HỢP AI TRONG SPLUNK

3.1. Mô hình thực nghiệm



Hình 3.1. Mô hình thực nghiệm hệ thống Splunk tích hợp AI

- Yêu cầu:

Bảng 3.1. Yêu cầu cấu hình các thành phần hệ thống

Thành phần	Hệ điều hành	CPU (core)	Memory (GB)	Disk (GB)
Search Head	Ubuntu Server	2	12	60
Indexer (idx1)	Ubuntu Server	2	2	60
Indexer (idx2)	Ubuntu Server	2	2	60
Master Cluster (MC)	Ubuntu Server	2	2	60
Heavy Forwarder	Ubuntu Server	2	2	60
Deployment Server	Ubuntu Server	2	2	60
Forwarder (UF nix)	Linux	4	4	60
Forwarder (UF win)	Windows 10	4	4	60
Forwarder (HF)	Ubuntu Server	2	2	60
Pfsense	pfsense	1	1	40

Bảng 3.2. Danh sách địa chỉ IP của các thành phần hệ thống Splunk

Thành phần	Địa chỉ IP
Search Head (sh1)	192.168.214.157
Indexer (idx1)	192.168.214.158
Indexer (idx2)	192.168.214.161
Master Cluster (MC)	192.168.214.160
Heavy Forwarder (HF)	192.168.214.162
Deployment Server (DS)	192.168.214.164
Forwarder (UF nix)	192.168.214.166
Forwarder (UF win)	192.168.214.145
Pfsense	192.168.214.165

Bảng 3.3. Các cổng mạng sử dụng trong mô hình thực nghiệm Splunk

Port	Source	Destination	Mô tả
8000/tcp	Web Browser (user)	Splunk Web	Truy cập giao diện Web của Splunk
8089/tcp	Splunk CLI, SH, Deployer, CM	Indexer, UF, DS	Quản trị nội bộ Splunk (splunkd, REST API), DS
9997/tcp	UF / HF	Indexer	Nhận log từ Universal / Heavy Forwarder
514/udp	Syslog source (router / firewall)	HF hoặc Indexer (Syslog listener)	Nhận log từ thiết bị gửi theo chuẩn syslog
22/tcp	Admin	Splunk server (Linux/SSH)	SSH quản trị hệ thống
443/tcp	Search Head	Telegram	Nhận cảnh báo qua API telegram
20128/tcp	Search Head	API Smart(9Router)	Quản lý và điều phối API
11434/tcp	Search Head	Ollama	Truy cập ollama nội bộ

3.2. Quy trình thực nghiệm

3.2.1. Kịch bản thực nghiệm

Để đánh giá khả năng thu thập, phát hiện và cảnh báo của hệ thống, đề tài xây dựng kịch bản thực nghiệm bao gồm hai giai đoạn chính: giai đoạn tấn công và giai đoạn phát hiện - xử lý.

a) Giai đoạn tấn công

Kịch bản mô phỏng một cuộc tấn công sử dụng kỹ thuật T1219 - Remote Access Software trong khung MITRE ATT&CK, được thực hiện theo các bước tuần tự sau:

- **Bước 1 - xâm nhập hệ thống qua RDP:** Kẻ tấn công sử dụng địa chỉ IP và mật khẩu đã biết trước để đăng nhập từ xa vào máy chủ mục tiêu qua giao thức RDP (cổng 3389), nhằm chiếm quyền truy cập ban đầu vào hệ thống.
- **Bước 2 - Cài đặt AnyDesk:** Sau khi đăng nhập thành công, kẻ tấn công tiến hành tải và cài đặt phần mềm truy cập từ xa hợp pháp AnyDesk lên máy nạn nhân. Việc sử dụng phần mềm hợp pháp giúp kẻ tấn công tránh bị phát hiện bởi các công cụ bảo mật.
- **Bước 3 - Tạo backdoor:** Kẻ tấn công chạy lệnh `anydesk.exe --set-password` để gán mật khẩu cố định và thiết lập chế độ truy cập không giám sát (unattended access), biến phần mềm hợp pháp thành kênh backdoor cho phép kết nối trở lại bất kỳ lúc nào.
- **Bước 4 - Truyền file:** Thông qua phiên kết nối AnyDesk, kẻ tấn công thực hiện truyền tải file giữa máy tấn công và máy nạn nhân, mô phỏng hành vi đánh cắp dữ liệu hoặc đưa công cụ độc hại vào hệ thống.
- **Bước 5 - Thực thi lệnh từ xa:** Kẻ tấn công mở Command Prompt (`cmd.exe`) thông qua AnyDesk và thực thi các lệnh bất thường trên máy mục tiêu, mô phỏng giai đoạn kẻ tấn công đã kiểm soát hoàn toàn hệ thống.

Toàn bộ các hành động trên đều được ghi nhận bởi các nguồn log bao gồm Windows Event Logs, Sysmon và tệp nhật ký nội bộ của AnyDesk (`ad.trace`), sau đó được thu thập về Splunk.

b) Giai đoạn phát hiện và xử lý

Sau khi dữ liệu log đã được thu thập vào Splunk, hệ thống thực hiện các hoạt động phát hiện và xử lý như sau:

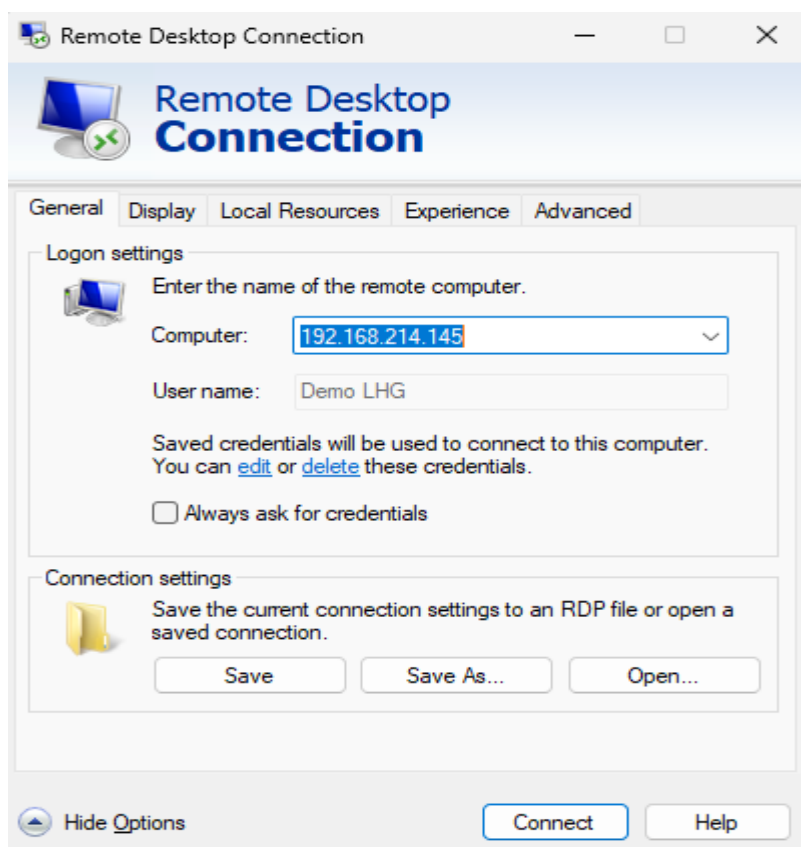
- **Bước 6 - Phát hiện bằng Rule SPL:** Hệ thống sử dụng các rule SPL đã được xây dựng để phát hiện các dấu hiệu tấn công đặc trưng như tiến trình AnyDesk với tham số `--set-password` hoặc Command Prompt được khởi chạy từ tiến trình AnyDesk. Khi phát hiện hành vi bất thường, hệ thống sẽ kích hoạt cảnh báo tự động qua Telegram với nội dung được làm giàu thông tin.
- **Bước 7 - Truy vấn logs bằng AI:** Người quản trị sử dụng chức năng hỗ trợ truy vấn logs, nhập yêu cầu bằng ngôn ngữ tự nhiên (tiếng Việt). Hệ thống sử dụng mô hình ngôn ngữ lớn để sinh ra câu lệnh SPL tương ứng, giúp người dùng dễ dàng truy xuất các log liên quan đến sự cố.
- **Bước 8 - Phân tích logs bằng AI:** Người quản trị chọn một dòng log cụ thể và yêu cầu hệ thống phân tích. Mô hình ngôn ngữ lớn sẽ diễn giải nội dung log, giải mã các đoạn mã thực thi ẩn, xác định các thực thể an ninh và đưa ra đánh giá mức độ rủi ro.
- **Bước 9 - Điều tra sự cố bằng AI:** Khi một cảnh báo được kích hoạt, người quản trị kích hoạt chức năng điều tra sự cố. Hệ thống tự động tổng hợp toàn bộ dữ liệu

liên quan, tái tạo dòng thời gian tấn công, ánh xạ hành vi với ma trận MITRE ATT&CK và đề xuất các bước ứng phó ban đầu.

Như vậy, kịch bản thực nghiệm bao phủ toàn bộ quy trình từ tấn công đến phát hiện, truy vấn, phân tích và điều tra sự cố. Các mục tiếp theo sẽ trình bày chi tiết kết quả thực hiện cho từng bước trong kịch bản.

3.2.2. Mô phỏng tấn công

Kỹ thuật T1219 - Remote Access Software cho phép kẻ tấn công sử dụng các công cụ truy cập từ xa hợp pháp để thiết lập kênh điều khiển và thực thi lệnh từ xa. Quá trình thực hiện trong thực nghiệm được ghi nhận như sau:



- Tiến hành triển khai kết nối qua RDP (Port 3389)

```
Administrator: Command Prompt
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\system32>powershell.exe -NoProfile -ExecutionPolicy Bypass -Command "Write-Host 'Simulating Level RMM Installation for Splunk Log...'; iex (New-Object Net.WebClient).DownloadString('https://downloads.level.io/install_windows.ps1')"
Simulating Level RMM Installation for Splunk Log...
* Downloading level-windows-amd64.exe
* Installing level-windows-amd64.exe
- Starting the agent...
* Level Agent has failed to install.

WARNING: expected argument for flag `/k, /key', but got option `/a'
```

- Thực hiện thao tác chạy mã độc trên CMD.
- Kiểm tra vị trí file log trên anydesk bằng image C:\Users\Demo LHG\AppData\Roaming\AnyDesk\ad.trace

- Phiên kết nối bắt đầu được thiết lập.

```

590 10:19:55.253 back 4740 8232 win_app.backend - Backend is starting up.
591 10:19:55.254 back 4740 8232 app.named_process_starter - Using existing control (2960).
592 10:19:55.258 back 4740 8232 win_app.backend - Using global namespace.
593 10:19:55.263 back 4740 5448 win_app.backend - Incoming IPC connection.
594 10:19:55.263 back 4740 8232 win_app.backend - Backend startup finished.
595 10:19:55.265 back 4740 8228 sif - Updating license info.
596 10:19:55.265 back 4740 8228 sif - License info updated: free-1
597 10:19:55.265 back 4740 8228 win_app.backend - Backend process login finished.
598 10:19:55.265 back 4740 5432 win_app.msg_loop - Waiting for events.
599 10:19:55.290 back 4740 1044 win_app.desktop_switcher - Desktop change: --> Default
600 10:19:55.313 back 4740 5448 app.backend_session - Initializing session.
601 10:19:55.313 back 4740 8232 app.backend_session - Starting sockets.
602 10:19:55.314 back 4740 5448 win_app.backend - Session count: 1
603 10:19:55.315 back 4740 1044 win_app.desktop_switcher - Suspending capture.
604 10:19:55.315 back 4740 1044 capture.fallback_capture - Switching to none.
605 10:19:55.315 back 4740 1044 capture.fallback_capture - Switching to none cancelled.
606 10:19:55.315 back 4740 1044 win_app.desktop_switcher - Initiating the desktop switch.
607 10:19:55.316 back 4740 2952 app.backend_session - Incoming session request: (1591338903
608 10:19:55.316 back 4740 2952 app.backend_session - Remote caps: sound_src, sound_sink, key
609 10:19:55.316 back 4740 2952 app.backend_session - Remote OS: Windows, Connection flags: d
610 10:19:55.316 back 4740 2952 app.backend_session - Remote version: 9.7.3
611 10:19:55.316 back 4740 2952 app.backend_session - Trying to register a new backend sessio
612 10:19:55.382 back 4740 5432 internal - Could not get module file name (psapi 1
613 10:19:55.446 back 4740 2952 app.backend_session - Requesting to restore permissions.
614 10:19:55.447 back 4740 8228 app.backend_session - Restoring permissions. (action: 3; stat
615 10:19:55.447 back 4740 8228 app.backend_session - Initializing permissions.
616 10:19:55.451 back 4740 8228 app.backend_session - Requesting to store permissions.
617 10:19:55.463 back 4740 8228 app.backend_session - 9 times: Requesting to store permis
618 10:19:55.463 back 4740 8228 app.backend_session - Cannot set permission 10 to on. Permiss
619 10:19:55.464 back 4740 8228 app.backend_session - Requesting to store permissions.
620 10:19:55.478 back 4740 8228 app.backend_session - 8 times: Requesting to store permis
621 10:19:55.478 back 4740 8228 app.backend_session - Checking capabilities.
622 10:19:55.485 back 4740 8228 app.backend_session - Acknowledging session.
623 10:19:55.486 back 4740 5432 winapp.gui.permissions_panel - Selecting Profile: default, hasPw: N
624 10:19:55.488 back 4740 8228 desktop_auto_adjust - Route changes to activate Windows.

```

- Phát hiện câu lệnh đổi password bằng cmd.

```

541 gcp - Storing offer timestamp (fallback).
542 gcp - Store result: 1 (0x00000000)
543 win_app.frontend - Disallowing Google Chrome offer (user already decided).
544 ad_app.control - Received GCP response (2, 1, 15, 0).
545 win_app.frontend - Google Chrome installation complete (1).
546 win_app.frontend - Disallowing Google Chrome offer (user already decided).
547
548 main - * AnyDesk Windows Startup *
549 main - * Version 9.7.3 (release/win_9.7.3 ee48af62bd3621f3512caa3ca0ed803bf578595f)
550 main -
551 main - Command Line params: "C:\Program Files (x86)\AnyDesk\AnyDesk.exe" --set-password
552 main - Process started at 2026-05-14. PID 4568. OS is Windows 10 (64 bit)
553 impl_selector - using sse2 (intrinsics)
554 base.data.config_application - Adding GPO defaults layer.
555 base.data.config_application - Adding GPO overrides layer.
556 base.data.config_application - Adding GPO defaults layer.
557 base.data.config_application - Adding GPO overrides layer.
558 main - Setting a new password from cmd line.

```

- Ngay sau khi yêu cầu kết nối, file log ghi nhận AnyDesk tự động chọn profile truy cập không giám sát thay vì hiện bảng hỏi người dùng.


```

661 admin_settings 2904 7460 impl_selector - using sse2 (intrinsics)
662 admin_settings 2904 7460 base.data.config_application - Adding GPO defaults layer.
663 admin_settings 2904 7460 base.data.config_application - Adding GPO overrides layer.
664 admin_settings 2904 7460 base.data.config_application - Adding GPO defaults layer.
665 admin_settings 2904 7460 base.data.config_application - Adding GPO overrides layer.
666 admin_settings 2904 7460 gui.gui_app - DPI scaling: 1.00 / 1.00
667 admin_settings 2904 7460 gui.gui_app - Using std-resolution bitmaps.
668 admin_settings 2904 7460 sif - Initializing license info.
669 admin_settings 2904 7460 sif - License info updated: free-1
670 admin_settings 2904 7460 base.signal.hub - 1 threads.
671 admin_settings 2904 2896 gui.global_settings_app - Global settings startup.
672 admin_settings 2904 2896 app.named_process_starter - Using existing control (2960).
673 admin_settings 2904 7460 wgf.msg_loop - Waiting for events.
674 admin_settings 2904 6108 sif - Dropping out of date license response.
675 admin_settings 2904 7460 internal - Could not get module file name (psapi 1).
676 admin_settings 2904 7460 one_time_password - Resetting 2FA key.
677 admin_settings 2904 7460 winapp.gui.permissions_panel - Selecting Profile: _unattended_access, hasPw: Y
678 admin_settings 2904 7460 winapp.gui.permissions_panel - Selecting Profile: _unattended_access, hasPw: Y
679 * *
680 back 7028 8240 main - * AnyDesk Windows Startup *
681 back 7028 8240 main - * Version 9.7.3 (release/win_9.7.3 ee48af62bd3621f351
682 back 7028 8240 main -
683 back 7028 8240 main - Command Line params: "C:\Program Files (x86)\AnyDesk\
684 back 7028 8240 main - Process started at 2026-05-14. PID 7028. OS is Window
685 back 7028 8240 impl_selector - using sse2 (intrinsics)
686 back 7028 8240 base.data.config_application - Adding GPO defaults layer.
687 back 7028 8240 base.data.config_application - Adding GPO overrides layer.

```

- Phát hiện capture và nhập từ bàn phím:

info 2026-05-14 10:21:07.489 back 7028 564
desk_rt.capture_component - Starting capture.

info 2026-05-14 10:21:07.866 back 7028 7832
desk_rt.capture_component - Keyboard layout changed to: en_us

- Phát hiện có phiên kết nối truyền tải file từ 2 phía:

```

1086 2312 564 winapp.gui.permissions_panel - 9 times: Selecting Profile: _unattended_access, hasPw: N
1087 2312 564 audio.core_audio_capture - Buffer sizes do not match (3840, 4224).
1088 2312 564 audio.core_audio_capture - Audio loopback capture initialized. ch=2, bps=16, sps=48000 Hz,
1089 2312 1608 winapp.gui.permissions_panel - Selecting Profile: _unattended_access, hasPw: N
1090 2312 564 app.backend_session - Session started (ok).
1091 2312 4244 app.backend_session - Received new lock account parameters (0).
1092 2312 8832 desk_rt.encoder - Format changed to 1024x768 (X8R8G8B8).
1093 2312 1608 winapp.gui.permissions_panel - Selecting Profile: _unattended_access, hasPw: N
1094 2960 8028 ad_app.control - Session count: (1, 1).
1095 2312 8264 winapp.gui.permissions_panel - 1 times: Selecting Profile: _unattended_access, hasPw: N
1096 2312 8264 app.backend_session - Received a sysinfo request.
1097 2312 4268 desk_rt.capture_component - Keyboard layout changed to: en_us
1098 2312 5160 app.backend_session - Received a thumbnail request.
1099 2312 5160 app.backend_session - The current thumbnail is already up-to-date.
1100 2960 6892 base.task_queue - Waiting for commands.
1101 2960 8028 app.tunnel_ft_session - Invalid progress code (0).
1102 2960 6892 app.prepare_task - Preparing files in 'C:\Users\Demo LHG\AppData\Roaming\AnyDesk'.
1103 2960 6892 app.local_file_transfer - Preparation of 1 files completed (io_ok).
1104 2960 8028 app.tunnel_ft_session - Invalid progress code (0).
1105 2312 1304 app.backend_session - The user has requested a connection quit.

```

3.2.3. Cấu hình Rules và tạo Alert

splunk>enterprise Apps ▼ Thợ Splunk ▼ Messages ▼ Settings ▼ Activity ▼ Help ▼ Find

Search Analytics Datasets Reports Alerts Dashboards Modules Search & Reporting

warning rdp Save Save As View Create Table View Close

```
index=os_win ((source=*WinEventLog:Security EventCode=4624 Logon_Type=10) OR (source=*WinEventLog:Microsoft-Windows-Sysmon/Operational
EventCode=3 DestinationPort=3389))
| eval Source_IP=coalesce(src_ip, SourceIp)
| eval Target_User=coalesce(user, User)
| eval SourceUser=coalesce(SourceHostname, User)
| stats dc(EventCode) as Event_Count values(EventCode) as Triggered_Events values(SourceUser) as SourceUser values(Target_User) as
Users_Logged_In earliest(_time) as First-Seen by host Source_IP
| where Event_Count=2
| table host Source_IP SourceUser Triggered_Events Users_Logged_In
```

Time range: All time

✓ 44 events (before 6/1/26 9:56:25.000 PM) No Event Sampling Job

Events (44) Patterns Statistics (1) Visualization

Show: 100 Per Page Format Preview: On

host	Source_IP	SourceUser	Triggered_Events	Users_Logged_In
DESKTOP-E8L3LK3	192.168.214.1	NGUYENKIEN	3	Demo_LHG
			4624	NOT_TRANSLATED
				NT_AUTHORITY\NETWORK SERVICE

- Xây dựng lệnh truy vấn phát hiện các phiên đăng nhập RDP kết hợp với kết nối mạng đến cổng 3389.

Edit alert

Schedule & trigger conditions Trigger actions

* Alert type

Scheduled Real-time

* Schedule

Run On Cron Schedule

Cron Expression

e.g. 0 18 *** (every day at 6PM). [Learn more](#)

* Time range

Last 1 minute

Expires after

24 hour(s)

* Trigger alert when

Number of results is greater than 0

Cancel Save

- Alert typed : để cron
- Exp: * * * * *: mỗi phút chạy tác vụ 1 lần
- Time range: khoảng thời gian tìm kiếm logs

Edit alert

24

hour(s)

* Trigger alert when

Number of results

is greater than

0

* Trigger frequency

Once

For each result

Turn on throttle

☒

If throttle is on, then after an alert is triggered, subsequent alerts are suppressed for the duration of the throttle period. [Throttle alerts documentation](#)

Suppress results containing field value

Message

Suppress triggering for

60

second(s)

Cancel

Save

- Trigger alert when: điều kiện kích hoạt alert ở đây chỉ cần lớn hơn 0 logs là nổ, nếu rules là brute-force hoặc Ddos thì dễ theo ngưỡng.
- Trigger fre: để for each result và để trường message vào để lọc alert trùng lặp vì trường message bao gồm thời gian etc nên tránh được việc trùng cảnh báo.
- Suppress : khoảng time delay giữa các cảnh báo

Edit alert

Alert title

warning rdp

Description

Schedule & trigger conditions

Trigger actions

+ Add actions

▼

Add to Triggered Alerts

Severity

High

>

Telegram app

Cancel

Save

- Chỉnh thêm cảnh báo vào phần trigger alert và telegram.

Edit alert

☒ New rendering method: On

Title
Alert title. (Use to send field values.)

Message
Message that will be sent to Telegram. (Use to send field values.)

Severity
Severity level for the alert.

Bot token
Telegram Bot API token.

Chat ID

- Message : thêm các trường để khi đọc cảnh báo có thể thấy được các trường quan trọng như host, SourceUser và Users_Logged_In...

Edit alert

Bot token
Telegram Bot API token.

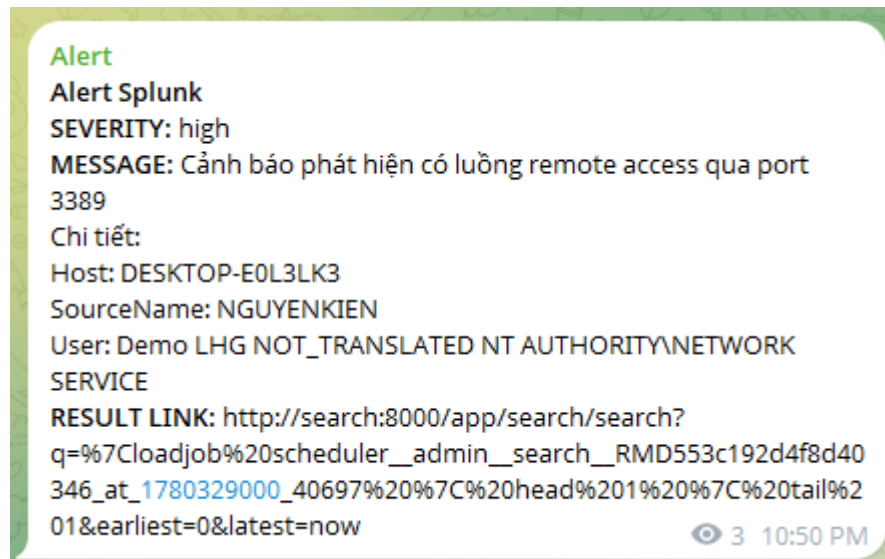
Chat ID
Single chat ID (private / group / supergroup). Example:

Chat mode
For superchats with topics the message will be sent to the specified topic (message_thread_id).

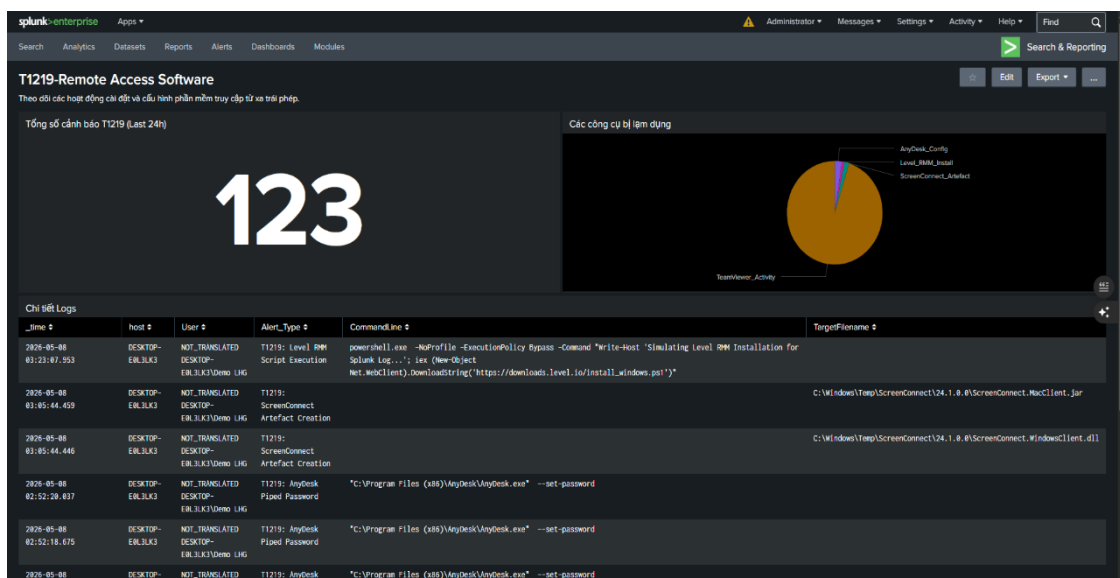
Topic ID or link
Only used when Chat mode = Superchat. You can pass a numeric ID or a link containing / .

Link to Results
Attach Splunk results link to the message.

- Thêm các token bot và chat ID để bot có thể gửi cảnh báo qua đoạn chat



- Kết quả cho thấy, khi alert nổ ra, tin nhắn cảnh báo đã gửi thành công đến Telegram, nội dung cảnh báo chứa các trường quan trọng để xác định chính xác các thông tin quan trọng của cảnh báo.



Ý nghĩa và logic khi tạo Dashboard:

- Đếm tổng số lượng sự kiện khớp với bộ rule T1219 trong 24 giờ qua. Khi số cảnh báo tăng đột biến bất thường sẽ bắt đầu quy trình ứng phó sự cố.
- Giúp nhận diện nhanh mức độ nghiêm trọng và loại trừ False Positive. Ví dụ: Nếu công ty quy định nv được phép dùng AnyDesk, việc thấy AnyDesk có thể chỉ là cảnh báo mức độ thấp thì cần xác minh lại. Nhưng nếu tự nhiên biểu đồ xuất hiện một mảng của TeamViewer là phần mềm công ty không bao giờ dùng, đó gần như chắc chắn là một cuộc tấn công thật và cần ưu tiên xử lý ngay lập tức.

- Khi đã xác định có sự cố từ 2 Panel trên, người điều tra viên cần bằng chứng. Thay vì phải tốn thời gian gõ lại các câu lệnh SPL phức tạp để truy tìm log, bảng này hiện thị sẵn các thông tin như host, User, CommandLine, TargetFilename

3.2.4. AI hỗ trợ truy vấn logs

The screenshot shows the Splunk Enterprise interface with the 'Hỗ trợ truy vấn SPL' (SPL Query Assistant) panel. The input text is: 'phát hiện truy cập từ xa qua giao thức rdp kết hợp sysmon'. The AI Assistant has generated the following SPL query:

```
(index=os_win sourcetype="wineventlog" source="WinEventLog:Security" EventCode=4624 Logon_Type=10) OR (index=os_win sourcetype="XmlWinEventLog" source="WinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=3 Image="*mstsc.exe") OR (index=os_win sourcetype="XmlWinEventLog" source="WinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=3 DestinationPort=3389) | search NOT (user="*" OR user="NT AUTHORITY\SYSTEM" OR user="NT AUTHORITY\NETWORK SERVICE" OR user="NT AUTHORITY\LOCAL SERVICE" OR Image="*\\SplunkUniversalForwarder\\*") | table _time host user Logon_Type CommandLine SourceIp DestinationIp DestinationPort ProcessId SourceHostname
```

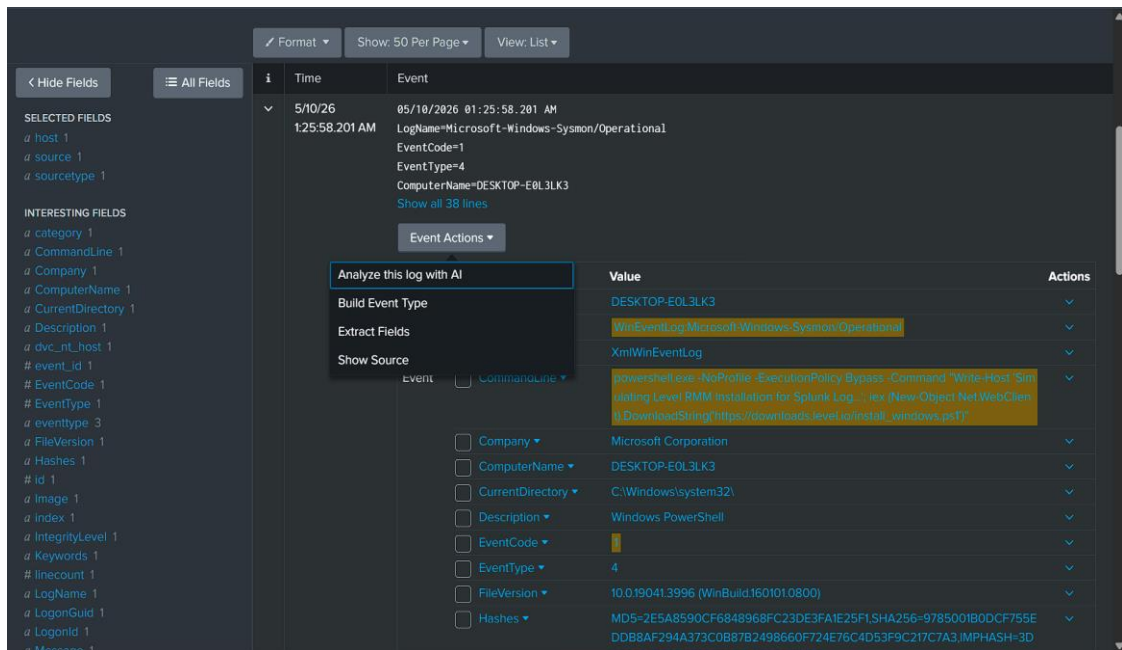
- Người dùng nhập câu hỏi, hệ thống sinh câu lệnh SPL hoàn chỉnh.

The screenshot shows the Splunk Enterprise interface with the 'warning rdp' search. The generated SPL query is displayed in a text box, and the results are shown in a table with 44 events. The table columns are: _time, host, user, Logon_Type, CommandLine, SourceIp, DestinationIp, DestinationPort, ProcessId, and SourceHostname.

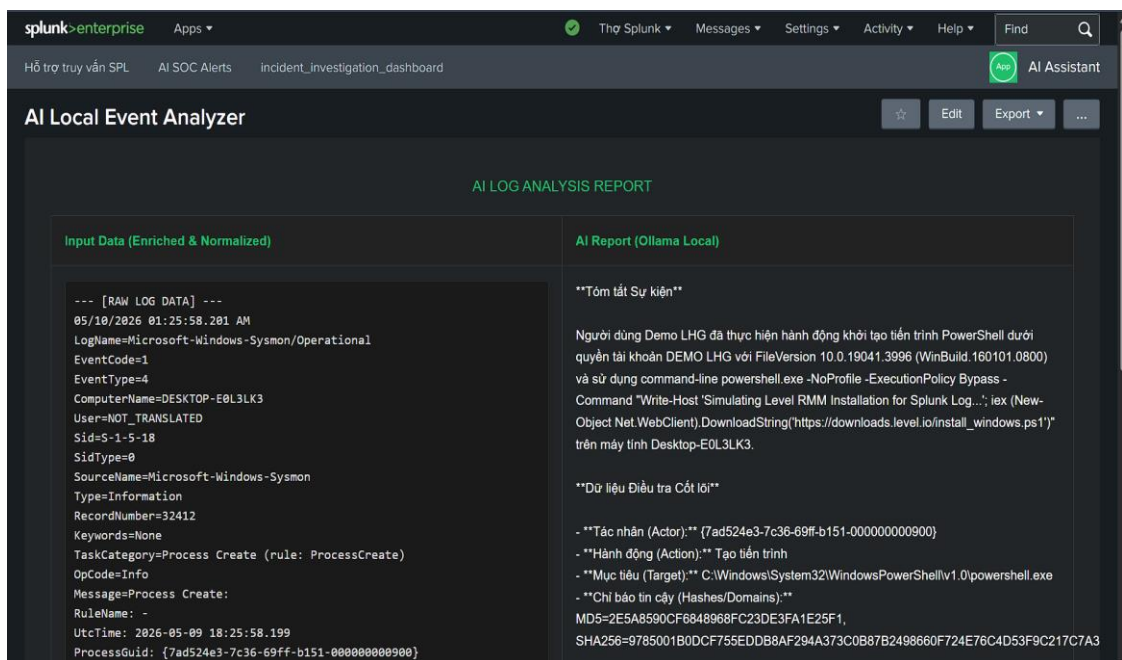
_time	host	user	Logon_Type	CommandLine	SourceIp	DestinationIp	DestinationPort	ProcessId	SourceHostname
2026-05-31 00:30:46.944	DESKTOP-EBL3LK3				192.168.214.1	192.168.214.145	3389	456	NGUYENKIEN
2026-05-31 00:30:46.944	DESKTOP-EBL3LK3				192.168.214.1	192.168.214.145	3389	456	NGUYENKIEN
2026-05-31 00:30:46.711	DESKTOP-EBL3LK3	Demo	10						
2026-05-31 00:30:46.711	DESKTOP-EBL3LK3	Demo	10						

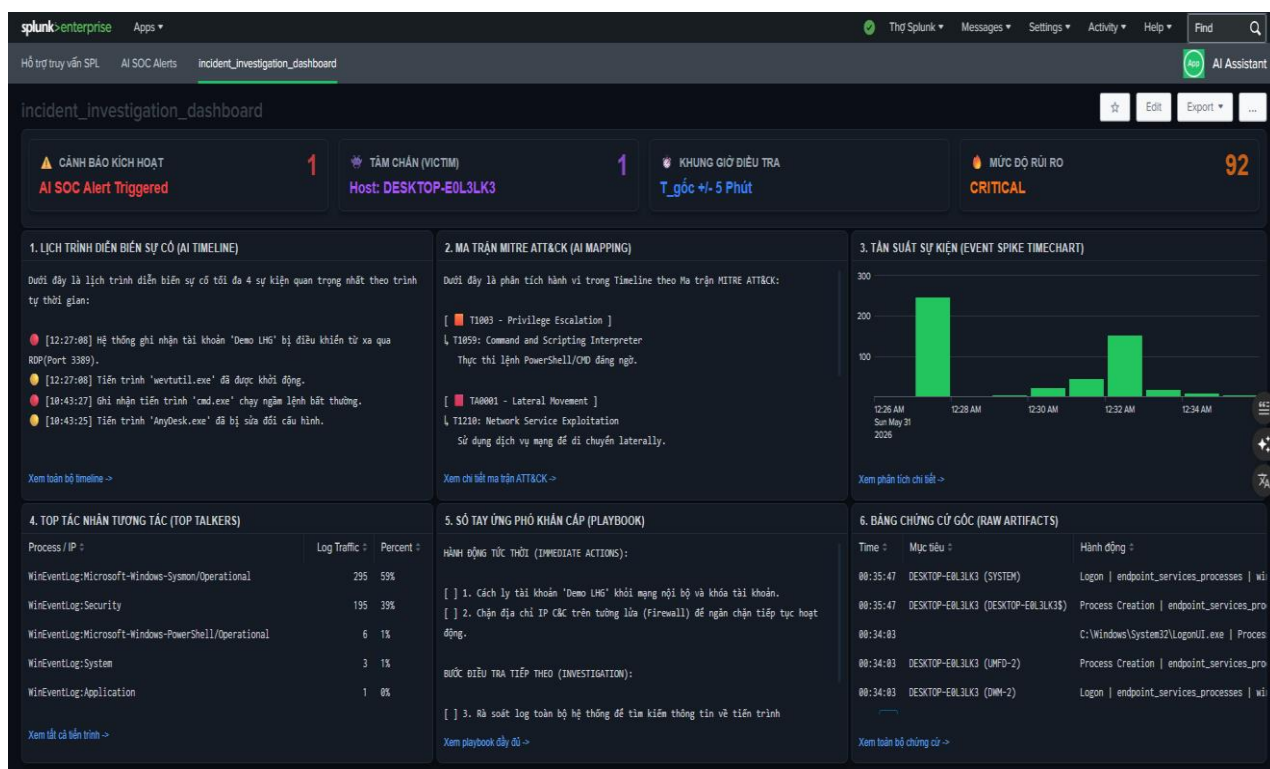
- Câu lệnh SPL được sinh ra có thể sao chép và sử dụng trực tiếp trên thanh tìm kiếm của Splunk, hỗ trợ người dùng không rành về cú pháp.

3.2.5. AI hỗ trợ phân tích logs



- Giao diện Splunk hiển thị thao tác khi muốn phân tích 1 dòng log





- Bảng dashboard thể hiện các trường quan trọng hỗ trợ các nhà phân tích khi có cảnh báo xuất hiện như lịch trình diễn biến sự cố, ánh xạ cảnh báo theo ma trận MITRE ATT&CK, biểu đồ thể hiện tần suất sự kiện, top các tác nhân trong cảnh báo, và bằng chứng gốc, kèm theo đề xuất sổ tay ứng phó khẩn cấp.

3.3. Đánh giá kết quả

Hệ thống Splunk tích hợp AI đã được triển khai thành công và vận hành ổn định trong môi trường thực nghiệm. Dữ liệu nhật ký từ Windows, Linux và pfSense được thu thập về Splunk một cách liên tục, đảm bảo nguồn dữ liệu cho các hoạt động phát hiện và phân tích.

Quá trình mô phỏng tấn công T1219 sử dụng AnyDesk đã được ghi nhận đầy đủ bởi Sysmon và Windows Event Log. Rule SPL phát hiện AnyDesk với tham số --set-password đã hoạt động đúng, phát hiện chính xác sự kiện tấn công. Cảnh báo được gửi qua Telegram trong vòng vài giây, bao gồm các thông tin như host, user và dòng lệnh thực thi. Hệ thống không phát sinh cảnh báo sai đối với các hoạt động AnyDesk thông thường.

Với chức năng truy vấn, AI đã sinh câu lệnh SPL đúng cú pháp và sử dụng đúng tên index, sourcetype từ câu hỏi tiếng Việt. Tuy nhiên, AI đôi khi tạo ra truy vấn với tên trường không chính xác nếu câu hỏi không rõ ràng, do đó cần kiểm tra lại trước khi thực thi.

Với chức năng phân tích log, AI diễn giải được ý nghĩa sự kiện AnyDesk khởi chạy với tham số --set-password là hành động thiết lập backdoor. Người dùng nhận

được bản tóm tắt bằng tiếng Việt mà không cần kiến thức sâu về Sysmon. Tuy nhiên, AI vẫn có thể xuất hiện hallucination khi tự thêm thông tin không có trong log.

Với chức năng điều tra, AI đã tổng hợp log xung quanh thời điểm tấn công, xây dựng timeline và ánh xạ sang MITRE ATT&CK với các Tactic như Initial Access (RDP), Persistence (AnyDesk backdoor) và Command and Control. Tuy nhiên, timeline có thể không đầy đủ nếu log bị thiếu hoặc AI suy diễn sai mối quan hệ giữa các sự kiện.

Bảng 3.4. So sánh hiệu quả giữa thực hiện thủ công và sử dụng AI hỗ trợ

Tiêu chí	Thực hiện thủ công	Với AI hỗ trợ
Thời gian thực hiện	- Mất từ vài phút đến vài chục phút tùy vào độ phức tạp và kinh nghiệm của người dùng. - Điều tra sự cố có thể mất 15 đến 30 phút.	- Giảm xuống còn vài phút. - Điều tra sự cố hoàn thành từ 3 đến 5 phút tùy thuộc vào mô hình.
Yêu cầu kỹ năng	- Cần thành thạo SPL, hiểu cấu trúc log và các khung tham chiếu như MITRE ATT&CK. - Người mới gặp nhiều khó khăn	- Chỉ cần biết đặt câu hỏi bằng ngôn ngữ tự nhiên. - Người mới có thể sử dụng ngay mà không cần đào tạo sâu.
Độ chính xác	- Phụ thuộc vào kinh nghiệm và kiến thức của người thực hiện. - Nếu là chuyên gia thì độ chính xác rất cao.	- Nhanh nhưng không bảo đảm chính xác tuyệt đối, có thể xảy ra tình trạng hallucination. - Với các yêu cầu phức tạp cần có chuyên gia kiểm chứng.
Tính nhất quán	- Kết quả không đồng đều giữa những người thực hiện khác nhau. - Cùng một tác vụ nhưng mỗi người lại có một cách khác nhau.	- Cùng một tác vụ, AI cho kết quả tương đối giống nhau. - Đảm bảo chất lượng đồng đều.
Bảo mật dữ liệu	- Dữ liệu không rời khỏi hệ thống nội bộ.	- Nếu dùng API bên ngoài, log có thể bị gửi ra ngoài. - Có rủi ro bảo mật dữ liệu nhạy cảm.

KẾT LUẬN

Trong bối cảnh khối lượng dữ liệu nhật ký và số lượng sự kiện bảo mật ngày càng gia tăng, việc khai thác hiệu quả dữ liệu từ các hệ thống SIEM đã trở thành yêu cầu quan trọng đối với công tác giám sát và vận hành an toàn thông tin. Tuy nhiên, quá trình truy vấn, phân tích và điều tra log thường đòi hỏi nhiều thời gian cũng như kiến thức chuyên môn. Trên cơ sở đó, đề tài đã nghiên cứu và xây dựng hệ thống Splunk tích hợp AI nhằm hỗ trợ người dùng trong quá trình khai thác và phân tích dữ liệu nhật ký.

Về mặt lý thuyết, đề tài đã nghiên cứu các kiến thức nền tảng liên quan đến SIEM, Splunk Enterprise, mô hình ngôn ngữ lớn (LLM), kỹ thuật Prompt Engineering và phương pháp Retrieval-Augmented Generation (RAG). Đồng thời, đề tài cũng phân tích khả năng ứng dụng của LLM trong lĩnh vực vận hành an toàn thông tin, làm cơ sở cho việc xây dựng các chức năng hỗ trợ bởi AI.

Về mặt triển khai, đề tài đã xây dựng thành công hệ thống Splunk tích hợp AI với ba chức năng chính gồm hỗ trợ truy vấn logs bằng ngôn ngữ tự nhiên, hỗ trợ phân tích logs và hỗ trợ điều tra sự cố. Kết quả thực nghiệm cho thấy việc kết hợp khả năng xử lý dữ liệu của Splunk với năng lực hiểu ngữ cảnh của mô hình ngôn ngữ lớn giúp người dùng tiếp cận và khai thác dữ liệu log thuận tiện hơn, đồng thời giảm đáng kể khối lượng công việc phân tích thủ công.

Bên cạnh đó, kết quả thực nghiệm cũng cho thấy việc tích hợp AI vào nền tảng SIEM là khả thi và có tiềm năng hỗ trợ hiệu quả cho các hoạt động truy vấn, phân tích và điều tra dữ liệu bảo mật. AI có khả năng chuyển đổi yêu cầu ngôn ngữ tự nhiên thành truy vấn SPL, diễn giải nội dung log và hỗ trợ tổng hợp thông tin phục vụ quá trình điều tra sự cố.

Mặc dù đạt được các mục tiêu đề ra, hệ thống vẫn còn một số hạn chế như phụ thuộc vào chất lượng dữ liệu đầu vào, nội dung prompt và mô hình AI được sử dụng. Ngoài ra, phạm vi thực nghiệm mới tập trung vào một số kịch bản tiêu biểu và chưa được đánh giá trên môi trường thực tế với quy mô lớn. Trong tương lai, hệ thống có thể được mở rộng theo hướng xây dựng cơ sở tri thức chuyên biệt cho lĩnh vực an toàn thông tin, tối ưu cơ chế RAG và tích hợp với các nền tảng tự động hóa nhằm nâng cao hiệu quả vận hành của trung tâm điều hành an ninh (SOC).

TÀI LIỆU THAM KHẢO

- Habibzadeh, A. F. (2025). *Large Language Models for Security Operations Centers: A Comprehensive Survey*. arXiv. Retrieved from <https://arxiv.labs.arxiv.org/html/2509.10858>
- RedHat. (2026). *What are large language models*. Red Hat. Retrieved from <https://www.redhat.com/en/topics/ai/what-are-large-language-models>
- SigmaHQ. (2025). *Sigma Rules*. Sigma detection format. Retrieved from <https://sigmahq.io/docs/basics/rules.html>
- Splunk. (2018). *Splunk Fundamental 1*. Retrieved from <https://freebootcamp.net/wp-content/uploads/2024/02/Splunk-Fundamentals-Powerpoint-1.pdf>
- Splunk. (2019). *Splunk Fundamental 2*. Retrieved from <https://fr.scribd.com/document/938023745/Splunk-8-0-Fundamentals-Part-2>
- Splunk. (2026, 05 15). *About the search language*. (Cisco) Retrieved from <https://help.splunk.com/en/splunk-cloud-platform/search/search-manual/10.4.2604/search-overview/about-the-search-language>
- Strom, B. E. (2020). *MITRE ATT&CK: Design and Philosophy*. MITRE Corporation. Retrieved from https://attack.mitre.org/docs/ATTACK_Design_and_Philosophy_March_2020.pdf

PHỤ LỤC

1. Cài đặt Splunk

```
wget -O splunk-10.4.0-f798d4d49089-linux-amd64.tgz  
"https://download.splunk.com/products/splunk/releases/10.4.0/linux/splunk-  
10.4.0-f798d4d49089-linux-amd64.tgz"
```

2. Cài đặt Splunk Enterprise

- Giải nén và lưu vào thư mục /opt

```
Tar -xvzf splunk-<version>.tgz -C /opt
```

- Chạy Splunk

```
/opt/splunk/bin/splunk start --accept-license
```

- Thiết lập user admin tại lần đầu khởi động

3. Enable web, enable ssl

- Mở port 443 với UFW (hoặc với Firewallld)

```
sudo ufw allow 443/tcp  
hoặc  
sudo firewall-cmd --add-port=443/tcp --permanent  
sudo firewall-cmd --reload
```

- Chạy câu lệnh để enable web

```
sudo nano /opt/splunk/etc/system/local/web.conf  
[settings]  
enableSplunkWebSSL = true  
httpport = 443
```

4. Import License

- Qua giao diện Web:

```
Truy cập Splunk Web: https://<splunk_server>:8000  
Đăng nhập với tài khoản admin  
Vào Settings > Licensing  
Nhấn Add license  
Tải file .lic (license file) và nhấn Install
```

- Qua dòng lệnh CLI:

```
/opt/splunk/bin/splunk add licenses /path/to/your.lic -auth admin:admin123
```

5. Cấu hình indexer clustering

a) Cấu hình cơ bản Indexer

- Mở cổng 9997 để nhận log

Cách 1: mở bằng câu lệnh

```
/opt/splunk/bin/splunk enable listen 9997 -auth admin:admin123
```

Cách 2: mở bằng giao diện web

Setting → Forwarding and receiving → Receive data → Configure receiving
→ Add new 9997

b) Cấu hình từng indexer

```
nano /opt/splunk/etc/system/local/server.conf
[replication_port://9000]
[clustering]
master_uri = https://192.168.214.164:8089
mode = peer
pass4SymmKey = 12345678
```

c) Cấu hình master cluster

```
nano /opt/splunk/etc/system/local/server.conf
[clustering]
cluster_label = cluster1
mode = manager
pass4SymmKey = 12345678
replication_factor = 2
```

6. Cấu hình Deployment Server

- Bật Deployment Server trên DS

```
/opt/splunk/bin/splunk enable deploy-server -auth admin:<pass>
/opt/splunk/bin/splunk restart
```

7. Đẩy App và cấu hình từ Deployer, Master Cluster, Deployment Server

- Tạo cấu hình index trên CM

```
- cd /opt/splunk/etc/manager-apps
mkdir -p pfsense_index/local
nano pfsense_index/local/indexes.conf
[pfsense]
homePath = $SPLUNK_DB/pfsense/db
coldPath = $SPLUNK_DB/pfsense/coldddb
thawedPath = $SPLUNK_DB/pfsense/thaweddb
repFactor = auto
[os_win]
homePath = $SPLUNK_DB/os_win/db
coldPath = $SPLUNK_DB/os_win/coldddb
thawedPath = $SPLUNK_DB/os_win/thaweddb
repFactor = auto
[os_nix]
homePath = $SPLUNK_DB/os_nix/db
coldPath = $SPLUNK_DB/os_nix/coldddb
thawedPath = $SPLUNK_DB/os_nix/thaweddb
```

```
repFactor = auto
```

- Đẩy xuống các indexer

```
/opt/splunk/bin/splunk apply cluster-bundle --answer-yes --auth  
admin:admin123
```

8. Cấu hình thu thập log pfsense gửi đến HF

- Cấu hình HF

```
nano /opt/splunk/etc/system/local/inputs.conf  
[udp://514]  
sourcetype = syslog  
index = pfsense  
connection_host = 10.3.82.99 ( địa chỉ LAN của pfsense)  
disabled = 0
```